

HOCHSCHULE BREMERHAVEN

FACHBEREICH 02

MANAGEMENT UND INFORMATIONSSYSTEME

Parallele Algorithmen Multicore-Praktikum

Assignments

PRÜFER:IN: Prof. Dr. Thomas Umland

SEMESTER: Sommersemester 2025

NAME: Hossein Akbari

MATR.-NR.: 39940

E-MAIL: hosakbari@smail.hs-bremerhaven.de

FACHSEMESTER: 06

STUDIENGANG: B.Sc. Informatik

Abgabedatum: 6. Juli 2025

Inhaltsverzeichnis

1	Einführung	1
2	Hauptteil	2
2.1	Odd Even Merge Splitting Sort	2
2.1.1	Übungen zum Odd Even Merge Splitting Sort	4
2.1.2	Assignment 1	9
2.2	Bitonic Sort	24
2.2.1	Übungen zum Bitonic Sort	25
2.2.2	Assignment 2	31
2.3	Merge Sort	38
2.3.1	Übungen zum parallelen Merge Sort	38
2.3.2	Assignment 3	44
2.4	Amdahls Gesetz und Gustafsons Interpretation	96
3	Schluss	97
3.1	Reflexion & Fazit	97

1 Einführung

Unter der Anweisung von Prof. Dr. Thomas Umland sollen für einige Sortieralgorithmen, die in der Parallelverarbeitung zum Einsatz kommen, auf dem Multicore-Rechner, für eine gegebene Problemgröße N Laufzeitmessungen durchgeführt werden. Dabei wird jede Messung dreimal wiederholt und die kürzeste Laufzeit als Ergebnis herangezogen. Anschließend werden Laufzeit, relative Beschleunigung und Effizienz visualisiert und sowohl mit den theoretischen Vorüberlegungen als auch mit den Messergebnissen anderer Algorithmen verglichen.

Im Mittelpunkt des Moduls steht das Ziel, die Ausführungszeit großer Sortierprobleme durch Parallelisierung auf mehreren Kernen zu verkürzen. Dazu werden verschiedene parallele Merge-Split-Algorithmen analysiert, implementiert und auf einem Multicore-Rechner ausgeführt. Dadurch soll praktische Erfahrung mit der Parallelprogrammierung gesammelt, sowie der Umgang mit Race Conditions und Synchronisationsmechanismen erlernt werden. Ferner gibt es dabei eine intensive Auseinandersetzung mit den Kennzahlen „Beschleunigung“ und „Effizienz“.

Die Lernziele dieses Multicore-Praktikums umfassen zum einen das Verständnis, wie Beschleunigung und Effizienz definiert und gemessen werden. Zum anderen sollen die Teilnehmenden typische Ursachen für Abweichungen vom idealen Skalierungsverhalten – wie Synchronisations- und Kommunikations overhead oder Lastverteilungsschwierigkeiten – erkennen und deren Einfluss durch eigene Messungen quantifizieren.

Relative Beschleunigung

Die *relative Beschleunigung* $r_n(p)$ eines parallelen Algorithmus ist definiert als

$$r_n(p) := \frac{t_n(1)}{t_n(p)},$$

wobei $t_n(1)$ die Laufzeit auf einem einzelnen Prozessor und $t_n(p)$ die Laufzeit auf p Prozessoren angibt.

Effizienz

Die *Effizienz* $e_n(p)$ ergibt sich aus der relativen Beschleunigung dividiert durch die Prozessorzahl:

$$e_n(p) := \frac{r_n(p)}{p}.$$

Im weiteren Verlauf dieser Arbeit werden die in den Assignments erhobenen Messwerten diskutiert und mit den theoretischen Vorhersagen abgeglichen.

2 Hauptteil

2.1 Odd Even Merge Splitting Sort

Odd–Even Transposition Sort Der Odd–Even Transposition Sort ist ein einfacher, prozessorientierter Vergleichssortieralgorithmus. Für n zu sortierende Elemente werden n Prozessoren $P_1, \dots, P_n$ verwendet, wobei Prozessor P_i genau ein Element z_i speichert. Der Algorithmus besteht aus n Runden, in denen abwechselnd ungerade und gerade Paare verglichen und getauscht werden:

1. *Ungerader Schritt:* Alle Prozessoren mit ungeradem Index i , $1 \leq i < n$, vergleichen simultan ihre Elemente z_i und z_{i+1} . Sind sie in falscher Reihenfolge ($z_i > z_{i+1}$), so tauschen die beiden Prozessoren ihre Werte.
2. *Gerader Schritt:* Alle Prozessoren mit geradem Index j , $1 < j \leq n - 1$, führen denselben Vergleichs- und Tauschschritt mit ihrem Nachbarn durch.

Nach n solchen Phasen liegen die Elemente in aufsteigender Reihenfolge in $P_1, \dots, P_n$.

Bemerkungen

- Der Algorithmus ist hardwareeffizient, da alle Vergleiche lokal und synchronisierbar ablaufen.
- In einer Softwareimplementierung auf weniger als n Kernen entsteht jedoch ein starkes Missverhältnis zwischen Rechenaufwand und Kommunikation, wenn jeder Kern nur wenige Elemente bearbeitet.

Odd–Even Merge–Splitting Sort Um die Last auf einem System mit $p \leq n$ Prozessoren besser auszunutzen, verallgemeinert der Odd–Even Merge–Splitting Sort das oben beschriebene Verfahren:

1. **Initialisierung:** Die n Eingabeelemente werden gleichmäßig auf die p Kerne verteilt; jeder Kern hält ein lokales Array A_i der Länge n/p .
2. **Lokales Sortieren:** Jeder Kern sortiert sein Teilarray A_i sequenziell (z. B. mit Heapsort oder Bubble Sort).
3. **Merge–Split–Phase:** Es folgen p Runden, in denen abwechselnd *odd* und *even* Kerne aktiv sind:
 - In einer *odd*-Runde führt jeder ungerade Kern i simultan mit Kernel $i + 1$ eine Merge–Split-Operation auf (A_i, A_{i+1}) durch:
 - *Merge*: Erzeuge aus den beiden sortierten Listen A_i und A_{i+1} eine zusammenhängende sortierte Sequenz der Länge $2 \cdot (n/p)$.
 - *Split*: Weise den unteren Teil (erste n/p Elemente) zurück an A_i und den oberen Teil an A_{i+1} .
 - In der folgenden *even*-Runde wiederholen alle geraden Kerne i denselben Merge–Split-Schritt mit Kern $i + 1$.
4. **Fertigstellung:** Nach p Runden stehen alle n Elemente global sortiert über die Arrays $A_1, \dots, A_p$ verteilt.

Pseudocode

Algorithm 1 Odd–Even Merge–Splitting Sort

```
1: procedure OEMERGESPLITSORT( $A[0..n-1], p$ )
2:   Aufteilen:  $\{A_i\}_{i=1}^p \leftarrow$  Blöcke der Länge  $n/p$ 
3:   for all Kern  $i = 1, \dots, p$  parallel do
4:     Sortiere  $A_i$  sequenziell
5:   end for
6:   for  $k \leftarrow 1$  to  $p$  do
7:     if  $k$  ungerade then
8:       for all ungerade  $i = 1, 3, \dots, p-1$  parallel do
9:         Merge–Split( $A_i, A_{i+1}$ )
10:      end for
11:     else
12:       for all gerade  $i = 2, 4, \dots, p-1$  parallel do
13:         Merge–Split( $A_i, A_{i+1}$ )
14:       end for
15:     end if
16:   end for
17:   return Zusammengeführtes Array
18: end procedure
```

2.1.1 Übungen zum Odd Even Merge Splitting Sort

Welche Laufzeit (in O-Notation) besitzt der Odd Even Merge Splitting-Algorithmus, wenn zum Sortieren der lokalen Listen der sequentielle Heap Sort-Algorithmus verwendet wird?

Der Algorithmus zerfällt in zwei Phasen:

1. Lokale Sortierung:

Jeder der p Prozesse sortiert seine $\frac{n}{p}$ Elemente mit Heapsort. Da Heapsort in $O(m \log m)$ arbeitet, ergibt sich für jeden Prozess

$$f = O\left(\frac{n}{p} \log \frac{n}{p}\right).$$

2. Merge-Split-Phase:

Es folgen p Runden des Odd-Even-Splittings. In jeder Runde führen benachbarte Prozesse einen Merge ihrer beiden Teillisten (jeweils $\frac{n}{p}$ Elemente) in $O(\frac{n}{p})$ durch und splitten das Ergebnis ebenfalls in $O(\frac{n}{p})$. Damit beträgt der Aufwand pro Runde

$$O\left(\frac{n}{p}\right) + O\left(\frac{n}{p}\right) = O\left(\frac{n}{p}\right),$$

und über alle p Runden

$$g = p \cdot O\left(\frac{n}{p}\right) = O(n).$$

Gesamtlaufzeit: Mit $f = O\left(\frac{n}{p} \log\left(\frac{n}{p}\right)\right)$ und $g = O(n)$ gilt allgemein

$$t_n(p) = O(f + g) = O\left(\frac{n}{p} \log\left(\frac{n}{p}\right)\right) + O(n) = O\left(\max\left\{\frac{n}{p} \log\left(\frac{n}{p}\right), n\right\}\right).$$

Umformung und Fallunterscheidung Zunächst wendet man die Logarithmusregel $\log(a/b) = \log a - \log b$ an:

$$\frac{n}{p} \log\left(\frac{n}{p}\right) = \frac{n}{p} (\log n - \log p) = \frac{n}{p} \log n - \frac{n}{p} \log p = O\left(\frac{n}{p} \log n\right).$$

Vergleicht man nun $\frac{n}{p} \log n$ mit n , so erhält man

$$\frac{n}{p} \log n \geq n \iff \frac{\log n}{p} \geq 1 \iff \log n \geq p.$$

Daraus ergibt sich die Fallunterscheidung:

$$t_n(p) = \begin{cases} O\left(\frac{n}{p} \log n\right), & p \leq \log n, \\ O(n), & p > \log n. \end{cases}$$

Welcher Verlauf ergibt sich in diesem Fall für die Beschleunigung?

Die relative Beschleunigung $r_n(p)$ ist definiert als

$$r_n(p) = \frac{t_n(1)}{t_n(p)},$$

wobei $t_n(1)$ die Laufzeit auf einem Prozessor und $t_n(p)$ die Laufzeit auf p Prozessoren ist.

Laufzeiten einsetzen Für einen Prozessor gilt

$$t_n(1) = O(n \log n) + O(n) = O(n \log n).$$

Für p Prozessoren hatten wir bereits

$$t_n(p) = \begin{cases} O\left(\frac{n}{p} \log n\right), & p \leq \log n, \\ O(n), & p > \log n. \end{cases}$$

Stückweise Beschleunigung Setzt man dies in $r_n(p) = t_n(1)/t_n(p)$ ein, so erhält man

$$r_n(p) = \frac{O(n \log n)}{O(\max\{\frac{n}{p} \log n, n\})} = O\left(\min\left\{\frac{n \log n}{(n/p) \log n}, \frac{n \log n}{n}\right\}\right) = \begin{cases} O(p), & p \leq \log n, \\ O(\log n), & p > \log n. \end{cases}$$

Interpretation

- Für $p \leq \log n$ skaliert der Algorithmus nahezu linear: Jede Verdopplung von p verdoppelt annähernd die Beschleunigung.
- Ab $p > \log n$ erreicht man eine logarithmische Sättigung: Zusätzliche Prozessoren bringen nur noch eine Beschleunigung in $O(\log n)$.

Wie groß ist die Effizienz?

Die Effizienz $e_n(p)$ ist definiert als

$$e_n(p) = \frac{r_n(p)}{p},$$

wobei $r_n(p)$ die hergeleitete relative Beschleunigung ist:

$$r_n(p) = \begin{cases} O(p), & p \leq \log n, \\ O(\log n), & p > \log n. \end{cases}$$

Setzt man dies in $e_n(p) = r_n(p)/p$ ein, so erhält man

$$e_n(p) = \begin{cases} O\left(\frac{p}{p}\right) = O(1), & p \leq \log n, \\ O\left(\frac{\log n}{p}\right), & p > \log n. \end{cases}$$

Interpretation

- Für $p \leq \log n$ ist $e_n(p) = O(1)$: Die Prozessoren werden nahezu vollständig ausgelastet, d. h. eine Verdopplung von p liefert nahezu doppelte Beschleunigung pro Kern.
- Für $p > \log n$ fällt $e_n(p) = O\left(\frac{\log n}{p}\right)$ mit wachsendem p : Zusätzliche Kerne bringen immer weniger Speedup pro Kopf, bis ab einem gewissen Punkt praktisch kein Gewinn mehr erzielt wird.

Wie ändern sich die Ausdrücke für die Beschleunigung und die Effizienz, wenn zum Sortieren der lokalen Listen der sequentielle Bubble Sort-Algorithmus verwendet wird?

Verwendet man zum lokalen Sortieren den sequentiellen Bubble-Sort-Algorithmus, so ändern sich die Kosten folgendermaßen:

1. Gesamtlaufzeit

- (a) **Lokale Sortierung:** Jeder der p Prozesse sortiert seine $\frac{n}{p}$ Elemente mit Bubble Sort in

$$f = O\left(\left(\frac{n}{p}\right)^2\right).$$

- (b) **Merge-Split-Phase:** Wie bisher werden p Runden ausgeführt, jeweils mit Aufwand $O(\frac{n}{p})$ pro Prozess, insgesamt

$$g = p \cdot O\left(\frac{n}{p}\right) = O(n).$$

- (c) **Gesamtaufwand:**

$$t_n(p) = O\left(\left(\frac{n}{p}\right)^2\right) + O(n) = O(\max\{\frac{n^2}{p^2}, n\}).$$

Damit ergibt sich

$$t_n(p) = \begin{cases} O(\frac{n^2}{p^2}), & p \leq \sqrt{n}, \\ O(n), & p > \sqrt{n}. \end{cases}$$

2. Relative Beschleunigung Die Beschleunigung ist

$$r_n(p) = \frac{t_n(1)}{t_n(p)}, \quad t_n(1) = O(n^2) + O(n) = O(n^2).$$

Da $t_n(p) = O(\max\{n^2/p^2, n\})$, folgt

$$r_n(p) = O\left(\min\{\frac{n^2}{(n^2/p^2)}, \frac{n^2}{n}\}\right) = O(\min\{p^2, n\}),$$

also

$$r_n(p) = \begin{cases} O(p^2), & p \leq \sqrt{n}, \\ O(n), & p > \sqrt{n}. \end{cases}$$

3. Effizienz Die Effizienz $e_n(p) = r_n(p)/p$ ergibt

$$e_n(p) = \begin{cases} O\left(\frac{p^2}{p}\right) = O(p), & p \leq \sqrt{n}, \\ O\left(\frac{n}{p}\right), & p > \sqrt{n}. \end{cases}$$

Interpretation

- Für $p \leq \sqrt{n}$ steigt die Effizienz $O(p)$ stark mit der Prozessorzahl, da Bubble Sort besonders teuer ist und jeder neue Kern einen großen Anteil an der Arbeit übernimmt.
- Ab $p > \sqrt{n}$ fällt $e_n(p) = O\left(\frac{n}{p}\right)$ mit wachsendem p : Zusätzliche Kerne tragen immer weniger zur Beschleunigung bei, bis ein weiterer Gewinn praktisch verschwindet.

2.1.2 Assignment 1

In Assignment 1 sollen mit dem bereits implementierten Odd-Even-Merge-Splitting-Algorithmus in Java mit zwei Varianten für den sequentiellen Sortierschritt:

- **Variante 1:** Lokales Sorting mit `Arrays.sort()`, sowie
- **Variante 2:** Lokales Sorting mit Bubble Sort,

Laufzeitmessungen auf dem Multicore-Rechner durchgeführt werden. Die Messung soll für verschiedene Problemgrößen n und unterschiedliche Thread- bzw. Prozessoranzahlen p erfolgen. Weiterhin sollen die Laufzeit, basierend daraus auch Beschleunigung und Effizienz berechnet und visualisiert werden. Abschließend sollen die gemessenen Werte mit den theoretischen Überlegungen aus den vorherigen Aufgaben verglichen und beurteilt werden, welche Variante in der Praxis besser abschneidet.

Durchgeführte Experimente

In Aufgabe 01 wurde mit `Arrays.sort()` und den Problemgrößen $n = 921\,600\,000$ und $n = 1\,843\,200\,000$, Prozessorzahlen $p = 1, \dots, 24$ gemessen. In Aufgabe 02 wurde mit Bubble Sort und den Problemgrößen $n = 500\,000$, Prozessorzahlen (Threads) $p = 1, \dots, 140$ gemessen. Aus den gemessenen Laufzeiten wurden Beschleunigung und Effizienz für jedes p berechnet und grafisch dargestellt.

Aufgabe 01 (`Arrays.sort()`)

Problemgrößen $n = 921\,600\,000$ und $n = 1\,843\,200\,000$, Prozessorzahlen $p = 1, \dots, 24$

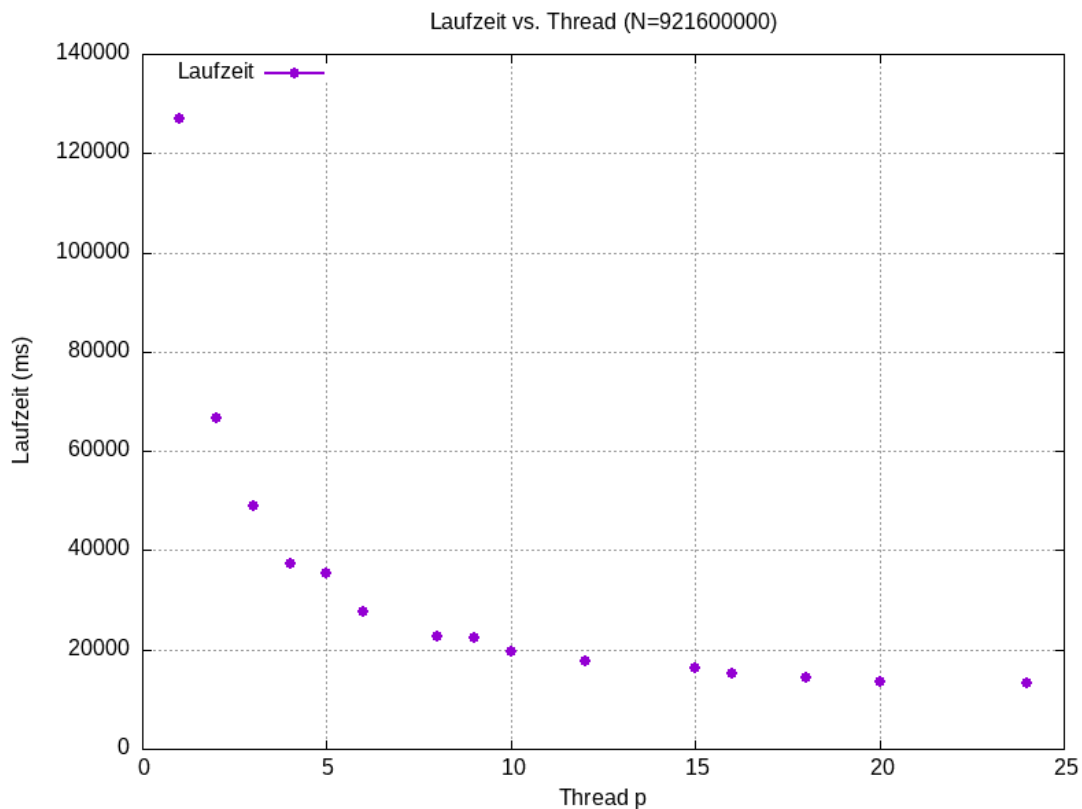


Abbildung 1: Laufzeit vs. Thread (N=921 600 000)

Der Algorithmus braucht bei einem Thread am meisten. Wenn jedoch mehr Thread eingesetzt werden, wird die Laufzeit weniger. Also je mehr Threads, desto weniger Laufzeit. Bei zwei Threads halbiert sich die Laufzeit fast. Bei vier Threads ist es schon ca. bei 48 Sekunden. Mit acht ist es bei ca. 22 Sekunden. Bis zwölf Threads fällt die Laufzeit sehr steil ab. Ab 12 Threads, wird aber der Gewinn pro Thread weniger. Grund hierfür ist eventuell, weil die Synchronisations- und Merge-Phasen mitwachsen.

Theoretische Überlegung aus Aufgabe 3.3.2: Hier wird gesagt, dass solange p kleiner als $\log n$ der lokale Heap-Sort dominiert. Das führt zu annähernd umgekehrter Proportionalität. Bei unserem Diagramm kann man genau das gut beobachten: Bis 12 Threads ist der Abfall steil, ab 12 Threads gibt es eine leichte Abflachung durch Parallel-Overhead.

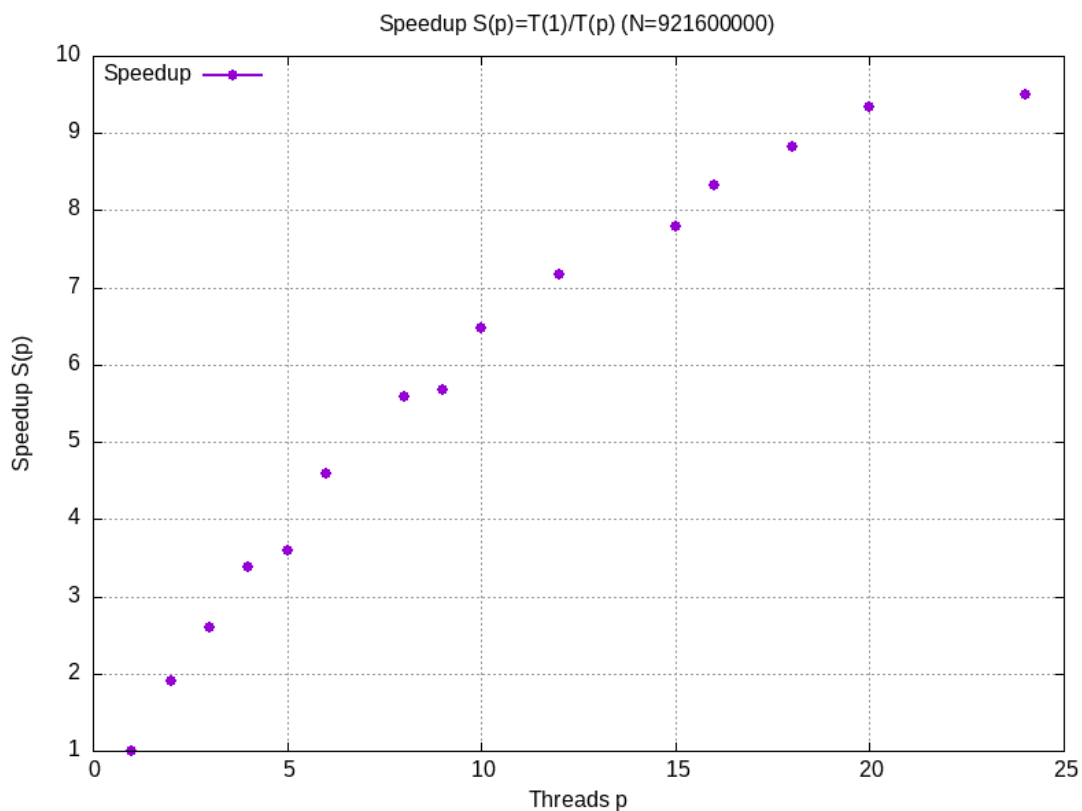


Abbildung 2: Speedup $S(p) = T(1)/T(p)$ ($N=921600000$)

Die Beschleunigung startet bei 1. Mit zwei Threads ist sie fast äquivalent zu 2. Bei vier sind es ca. bei 3,4. Mit acht sind es ca. bei 6,5. Und mit 24 Threads knapp bei 9,5. Diese Beobachtung ist nahezu ideal, also Beschleunigung als Äquivalenz zu P. Aber eigentlich zieht es etwas nach unten hin, weil das Zusammenfügen der Teillisten Arbeit kostet.

Theoretische Überlegung aus Aufgabe 3.3.2: Hier wird gesagt, dass für $p \leq \log n$ die Beschleunigung linear ist: $S(p) = O(p)$. Da wir bis $p = 24 < 30$ messen, sollte $S(p)$ fast genau proportional mitwachsen zu p . Das kann man anhand der Kurve gut sehen.

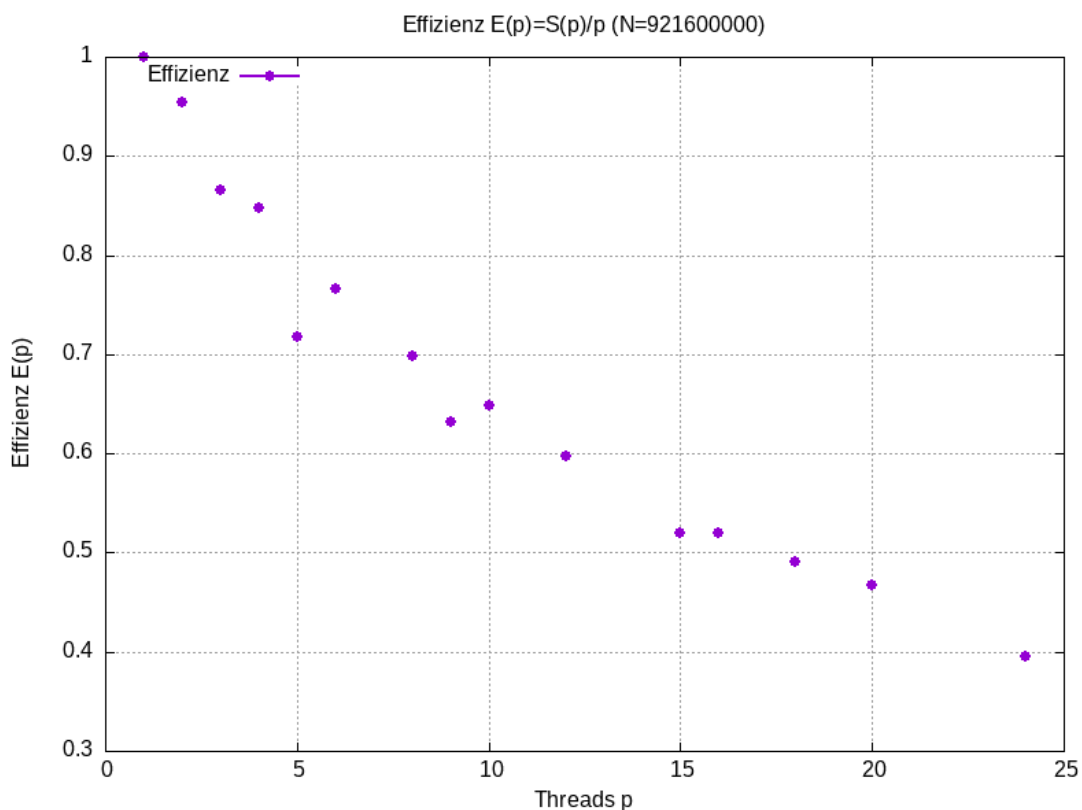


Abbildung 3: Effizienz $E(p)=S(p)/p$ ($N=921600000$)

Kurze Randinformation: Bei Effizienz wird gemessen, wieviel Beschleunigung man pro Kern rausholt. Bei $P = 1$, ist die Effizienz gleich 1. Wenn die Kerne mehr werden, fällt die Effizienz kontinuierlich auf Werte um ca. 0,85 (bei 4 Kernen), 0,63 (8 Kernen), 0,52 (16 Kernen), bis knapp 0,40 (24 Kernen). Jeder weitere Kern bringt bisschen weniger relative Beschleunigung. Die Kurve fällt zwar langsam, aber stetig.

Theoretische Überlegung aus Aufgabe 3.3.2: Hier wird prognostiziert, dass für $P \leq \log$ eine konstante Effizienz $E(p) = 0(1)$ gilt. Sprich: Es gibt nur einen langsamen Abfall durch Overhead. Genau das kann man sehen. $E(p)$ bleibt im Bereich 0,4 - 1,0 und sinkt nur mäßig, weil die Merge- und Synchronisationskosten allmählich ins Gewicht fallen.

Gesamtfazit:

Die Laufzeit fällt annähernd linear mit $1/p$, solange p deutlich unter $\log n$ bleibt. Die Beschleunigung wächst fast proportional zu p , also die Parallelisierung skaliert hervorragend. Die Effizienz bleibt im Bereich ca. 0,4 bis 1,0 und sinkt nur langsam, weil man Overhead schwer vermeiden kann.

Insgesamt entsprechen die gemessenen Ergebnisse sehr gut Ergebnissen aus der Theorie.

Laufzeit, Beschleunigung und Effizienz als Tabelle für $N = 921\,600\,000$ (Nachtrag)

Tabelle 1: Laufzeit, Speed-up und Effizienz für $N = 921,600,000$

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	%	Wert	%	Wert	%
1	127082	—	1.00	—	1.00	—
2	66627	-47.5	1.91	+90.8	0.95	-5.0
3	48945	-26.6	2.60	+36.0	0.87	-8.9
4	37471	-23.5	3.39	+30.7	0.85	-2.0
5	35397	-5.5	3.59	+5.9	0.72	-15.3
6	27639	-21.9	4.60	+28.0	0.77	+6.7
8	22752	-17.7	5.59	+21.5	0.70	-8.9
9	22365	-1.7	5.68	+1.7	0.63	-9.6
10	19596	-12.4	6.49	+14.1	0.65	+2.9
12	17736	-9.5	7.17	+10.5	0.60	-8.0
15	16303	-8.1	7.80	+8.7	0.52	-12.9
16	15266	-6.3	8.32	+6.8	0.52	0.0
18	14390	-5.8	8.83	+6.1	0.49	-5.6
20	13609	-5.4	9.34	+5.8	0.47	-4.9
24	13378	-1.7	9.50	+1.7	0.40	-15.2

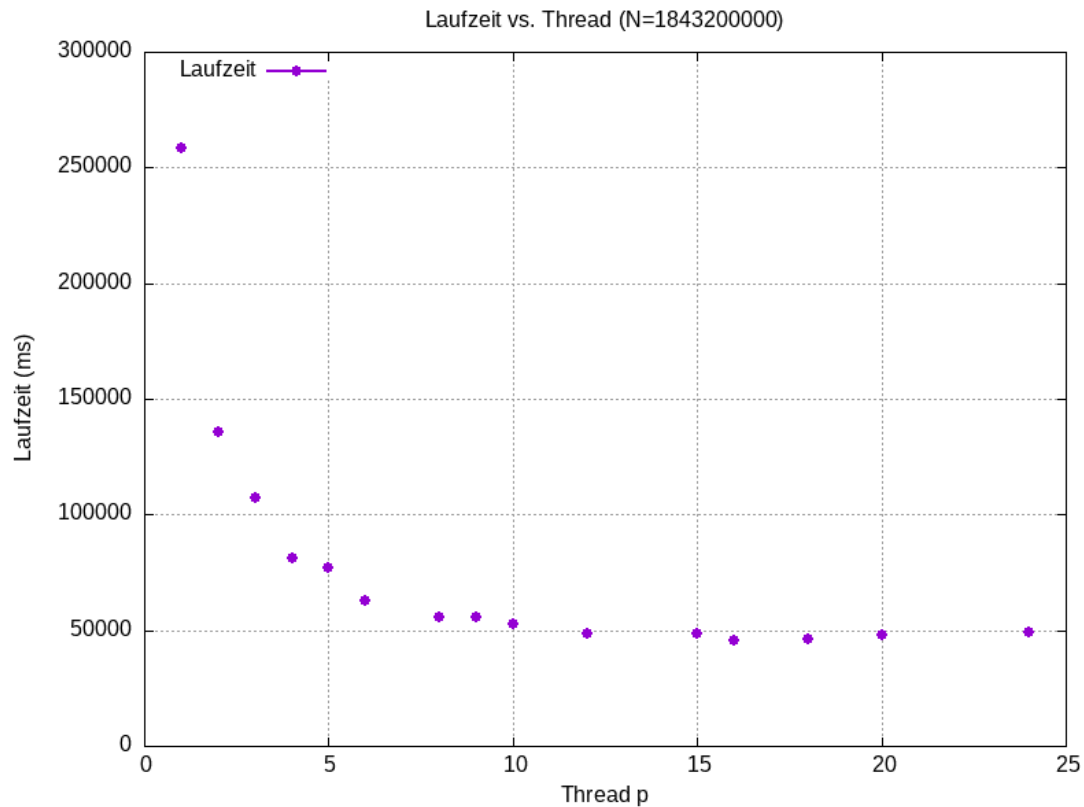


Abbildung 4: Laufzeit vs. Thread (N=1843200000)

Mit einem Kern ist die Laufzeit ca. 260000 ms. Bei 2 Kernen halbiert sich die Zeit auf ca. 135000 ms. Bei 4 Kernen ca. 108000 ms. Bei 6 Kernen ca. 80000 ms. Bei 8 Kernen => 76000 ms. 12 Kerne sind ca. 55000 ms. Ab 12 Kernen wird die Kurve deutlich flacher, was heißt, dass der Gewinn pro mehr Kern schrumpft. Beispiel hierfür ist zwischen 16 - 24 Kernen. Hier ändert sich die Zeit nur von 47000 auf 50000.

Theoretische Überlegung aus Aufgabe 3.3.2: Für $p \leq \log_2 n$ (ca.=) 31 erwartet man dass die Laufzeit ca. äquivalent ist zu $t(1)/p$. => Überwiegend $O(n/p \cdot \log n)$. Danach dominiert die feste Merge-Phase $O(n)$ und der Zeitgewinn durch zusätzliche Kerne verschwindet. Also: Die gemessene Laufzeit bei unserem Diagramm fällt nahezu im $1/p$ Verhältnis. Aber ab 12 Kernen sind die Merge- und Synchronisationen stärker.

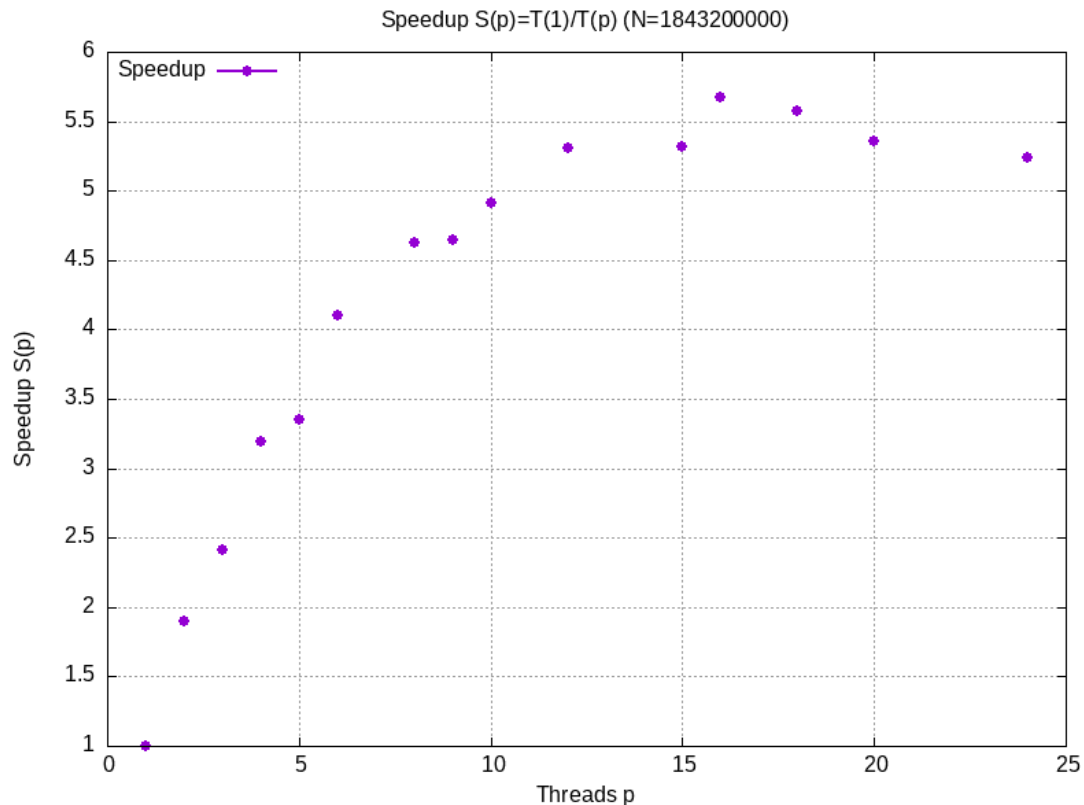


Abbildung 5: Speedup $S(p)=T(1)/T(p)$ ($N=1843200000$)

Die Beschleunigung ist bei 1 äquivalent zu 1. Also mit einem Kern ist die Beschleunigung 1. Bei zwei Kernen ca. 1,9. Bei 4 => 3,2. Bei 6 => 4,1. Bei 8 = 4,3. 12 sind 4,8. Die höchste Beschleunigung haben wir bei 16 Kernen mit 5,6. Ab 18-24 Kernen sind die Beschleunigung leicht auf 5,2.

Theoretische Überlegung aus Aufgabe 3.3.2: Solange $P \leq \log_2 n$ ca. 31, erwartet man eine lineare Skalierung, wie oben bereits erwähnt also $S(p) = O(p)$. In diesem Bereich sollte die Beschleunigung mit jedem Kern ca. um Faktor 1 wachsen. Aus der Beobachtung sehen wir bis ca. 12-16 Kernen tatsächlich fast eine proportionale Steigerung. Danach drückt der feste Overhead der Merge- und Synchronisationsphasen, wodurch zusätzliche Kerne dann kaum noch echten Mehrgeinn für Beschleunigung bringen. Zuzufolge fällt die Beschleunigung leicht ab.

Die Beschleunigung folgt also erstmal ganz gut einen linearen Verlauf, aber vir $p = \log_2 n$ ab 16 Kernen flacht sie ab, weil Merge- und Barriere-Synchronisation den Zuwachs von Parallelität wieder auffressen.

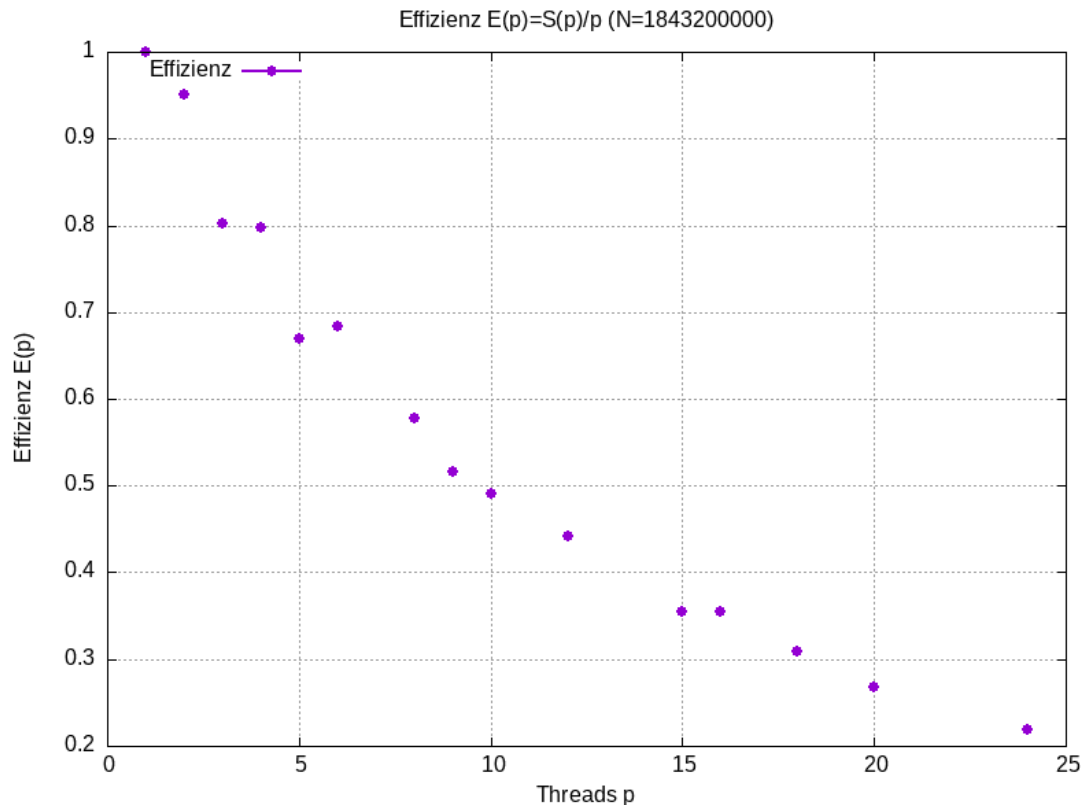


Abbildung 6: Effizienz $E(p)=S(p)/p$ ($N=1843200000$)

Mit einem Kern ist die Effizienz definitiv 100%. Bei zwei Kernen ca. 0,95. Bei 4 Kernen ca. 0,80. Bei 6 Kernen auch ca. 0,80. Bei 8 Kernen 0,67. Bei 12 Kernen ist es 0,58. Bei 16 Kernen 0,51. Bei 18 Kernen 0,31. 20 Kernen machen 0,27 aus und 24 gleich 0,22.

Theoretische Überlegung aus Aufgabe 3.3.2: Für $p \leq \log 2$ sollte die Effizienz $E(p)$ gleich $O(1)$ bleiben, also in etwa konstant, weil die Overhead-Kosten pro Kern vergleichsweise klein sind. Ab 6 Kernen fällt Effizienz deutlich unter 0,6, bis knapp- 0,2 bei 24 Kernen. Der Abfall ist hierbei stärker ausgeprägt als bei kleineren Problemgrößen, weil die feste Merge-Synchronisationsphase $O(n)$ im Verhältnis zur stark verkleinerten lokalen Arbeit $O(n/p \log n)$ schwerer ins Gewicht fällt.

Hierzu lässt sich sagen, dass die Effizienz mit jedem Kern sichtbar zurückgeht. Grund hierzu ist, dass bei großen n und vielen Threads der nicht-parallele Merge-Overhead und JVM-Management-Kosten dominieren. Das entspricht grob der theoretischen Erwartungen.

Laufzeit, Beschleunigung und Effizienz als Tabelle für $N = 1\,843\,200\,000$ (Nachtrag)

Tabelle 2: Laufzeit, Speed-up und Effizienz für $N = 1\,843\,200\,000$

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	%	Wert	%	Wert	%
1	258907	—	1.00	—	1.00	—
2	135966	−47.4	1.90	+90.2	0.95	−4.9
3	107341	−21.0	2.41	+26.7	0.80	−15.6
4	81071	−24.4	3.19	+32.4	0.80	−0.7
5	77268	−4.7	3.35	+4.9	0.67	−16.1
6	63032	−18.4	4.10	+22.6	0.68	+2.2
8	55927	−11.3	4.63	+12.7	0.58	−15.5
9	55653	−0.5	4.65	+0.4	0.52	−10.7
10	52602	−5.9	4.92	+5.8	0.49	−4.7
12	48717	−7.4	5.31	+8.0	0.44	−10.1
15	48666	−0.1	5.31	+0.1	0.35	−19.9
16	45582	−6.3	5.23	−1.6	0.33	−7.7
18	46404	+1.8	5.58	+6.6	0.31	−5.2
20	48266	+4.0	5.35	−4.0	0.27	−13.5
24	49361	+2.3	5.24	−2.1	0.22	−18.7

Aufgabe 02 (Bubble-Sort)

Problemgröße $n = 500\,000$, **Prozessorzahlen (Threads)** $p = 1, \dots, 140$.

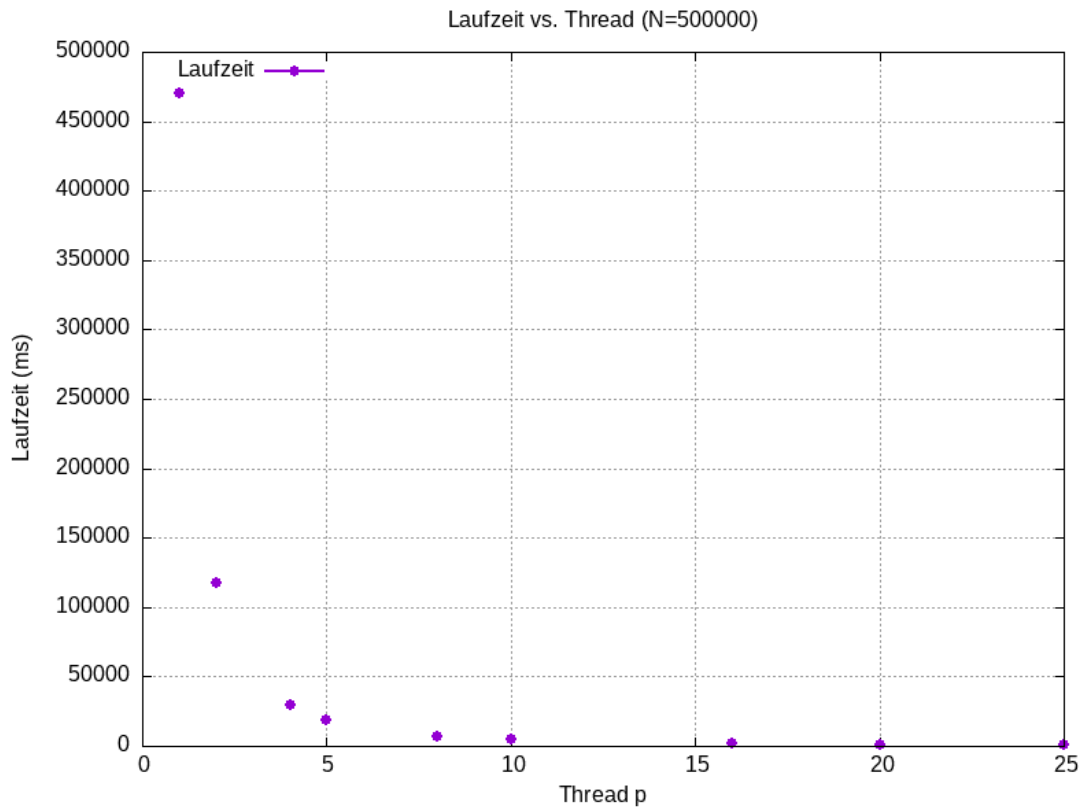


Abbildung 7: Laufzeit vs. Thread (N=500000)

Mit einem Kern ($p = 1$) dauert das Sortieren von $n = 500\,000$ Elementen etwa 470s. Bei zwei Kernen ($p = 2$) sinkt die Laufzeit auf ca. 118s, was einer knapp vierfachen Beschleunigung entspricht. Mit vier Kernen ($p = 4$) beträgt die Laufzeit etwa 30s, mit fünf Kernen ($p = 5$) rund 18s. Erhöht man die Kernzahl bis $p = 25$, so folgt die Laufzeit nahezu dem Skalierungsgesetz

$$T(p) = O(p^{-2}).$$

Verdoppelt man die Zahl der Kerne, reduziert sich die Laufzeit demnach um etwa den Faktor vier.

Grund hierfür ist, weil Bubble-Sort als lokale Sortierung eine Laufzeit von $(n/p)^2$ aufweist. Wenn man also die Anzahl der Kerne erhöht, verringert sich die Laufzeit quadratisch.

Für Bubble Sort ergibt sich die Gesamtlaufzeit

$$T(n, p) = O\left(\left(\frac{n}{p}\right)^2 + n\right).$$

Für $p < \sqrt{n}$ dominiert der quadratische Term $\left(\frac{n}{p}\right)^2$, und der Speedup folgt zu

$$r_n(p) = \frac{T(n, 1)}{T(n, p)} = \frac{O(n^2)}{O((n/p)^2)} = O(p^2).$$

Damit bestätigt sich das gemessene quadratische Abfallen der Laufzeit exakt durch die theoretische Vorhersage.

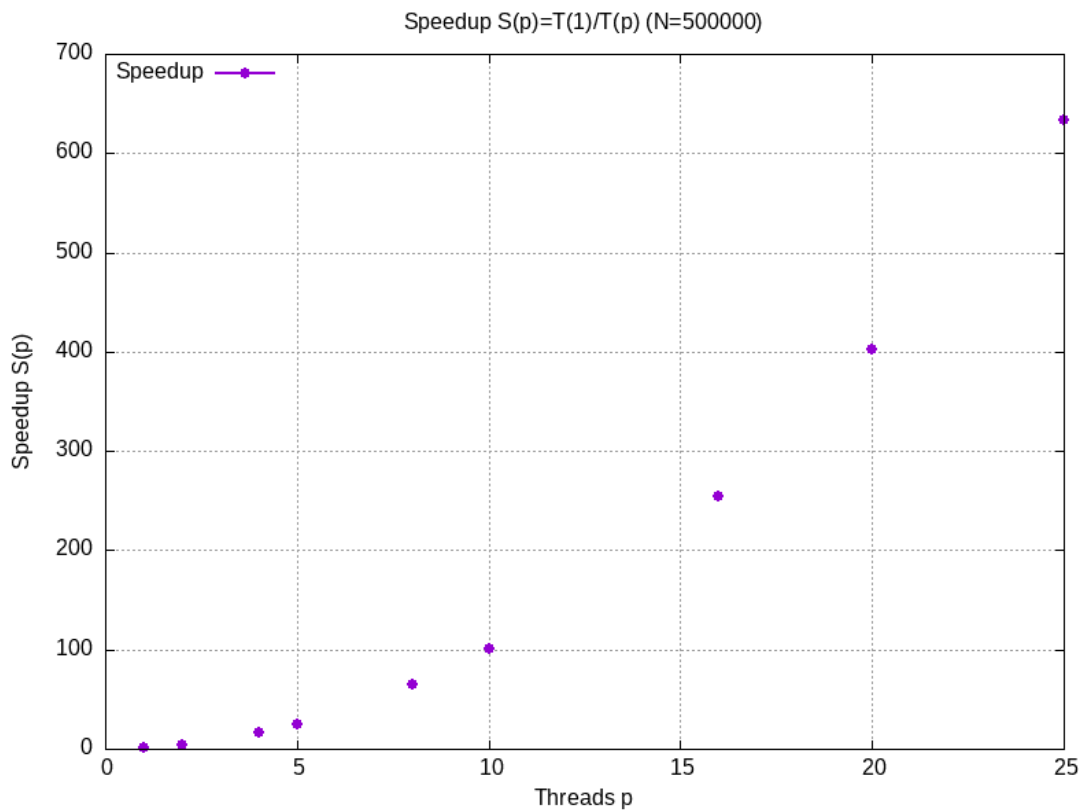


Abbildung 8: Speedup $S(p)=T(1)/T(p)$ ($N=500000$)

Die relative Beschleunigung startet bei

$$r_n(1) = 1, \quad r_n(2) \approx 4, \quad r_n(4) \approx 15, \quad r_n(8) = 64, \quad r_n(25) \approx 632.$$

Bis $p = 25$ zeigt $r_n(p)$ somit ein nahezu quadratisches Wachstum in p , d. h. $r_n(p) = \Theta(p^2)$.

Theoretische Einordnung (Aufgabe 3.3.5): Aus Aufgabe 3.3.5 wissen wir, dass für Bubble Sort

$$r_n(p) = O(p^2) \quad \text{für} \quad p < \sqrt{n}.$$

Da hier $\sqrt{500\,000} \approx 707$ und wir bis $p = 25$ messen, liegen wir eindeutig im $O(p^2)$ -Bereich. Die geringen Abweichungen resultieren aus dem noch vernachlässigbaren Merge-/Synchronisations-Overhead.

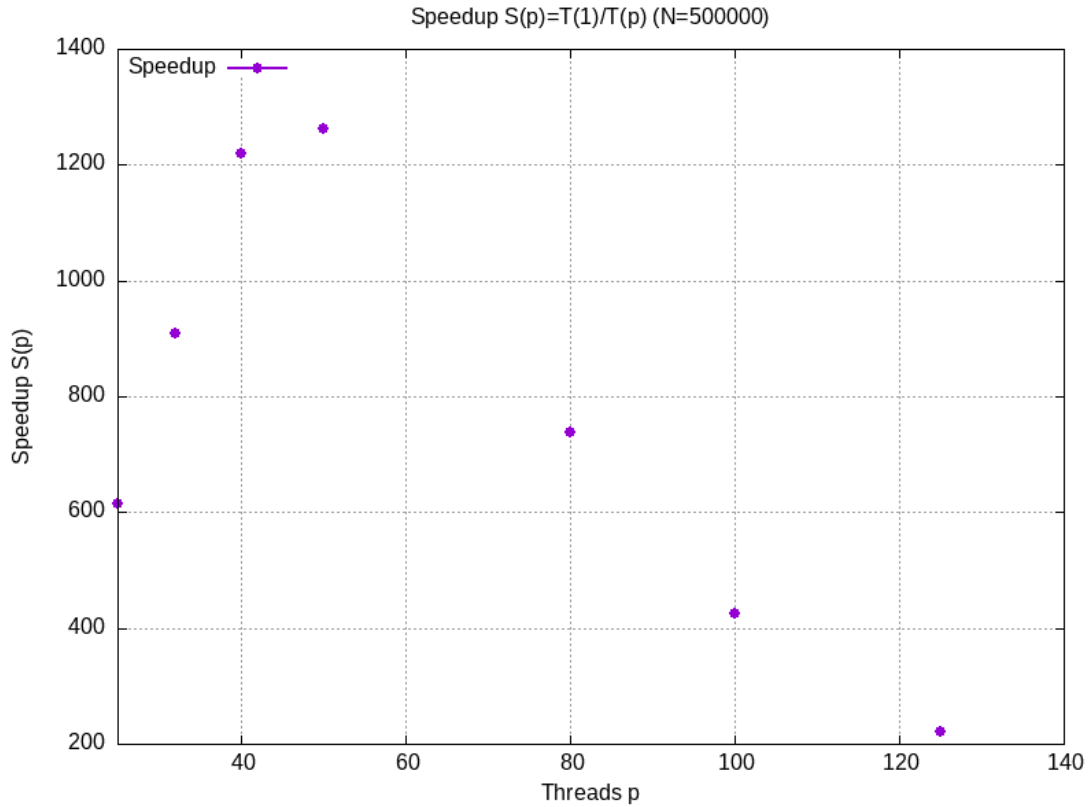


Abbildung 9: Speedup $S(p)=T(1)/T(p)$ ($N=500000$)

Hier kann man sehen, dass ab $p = 50$ der Zuwachs schon deutlich flacher wird. Bei $p = 128$ liegt die Beschleunigung nur noch bei ca. 220, anstelle des quadratischen Ideals. Grund hierfür ist, weil die Merge-Synchronisations-Phase $O(n)$ hierbei einfach teuer wird, weil die lokalen Blöcke so klein werden, dass das Zusammensetzen dominiert.

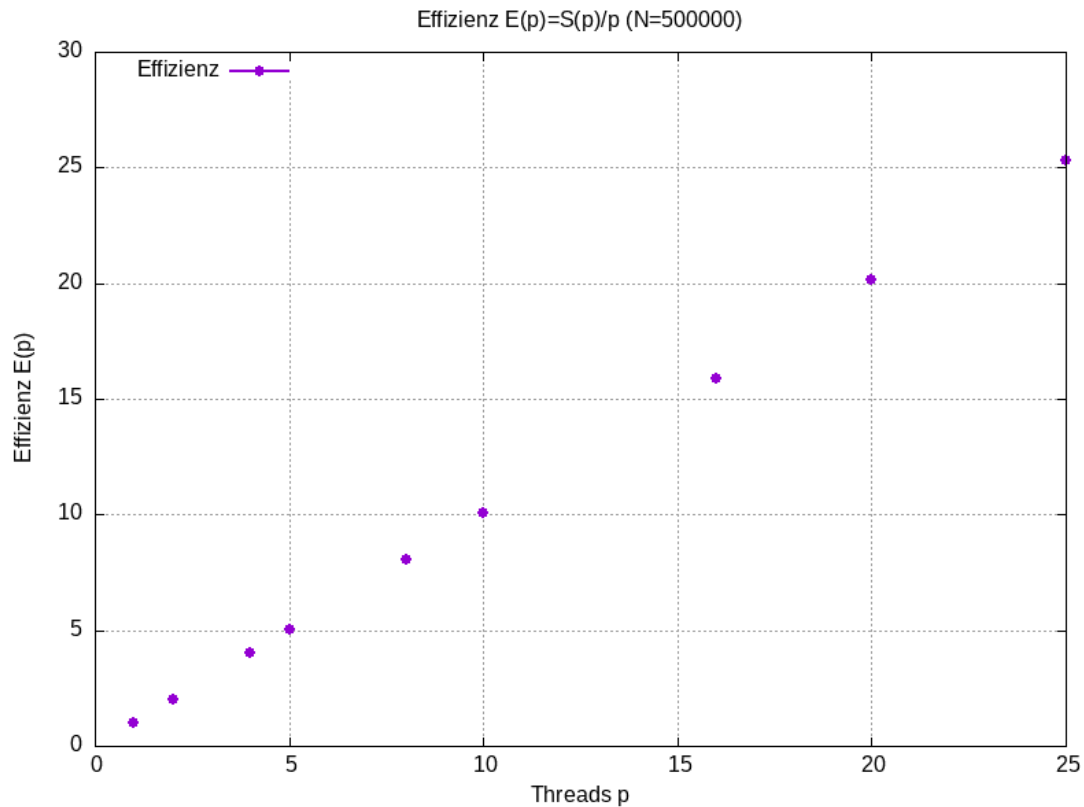


Abbildung 10: Effizienz $E(p)=S(p)/p$ ($N=500000$)

Gemessen ergibt sich für $n = 500\,000$:

$$e_n(1) = 1, \quad e_n(2) \approx 2, \quad e_n(4) \approx 5,8, \quad e_n(8) \approx 8, \quad e_n(25) \approx 25.$$

Bis $p = 25$ wächst die Effizienz nahezu linear mit p , denn die lokale Bubble-Sort-Arbeit dominiert so stark, dass durch Hinzufügen weiterer Kerne die Arbeit pro Kern sehr stark sinkt.

Theoretische Einordnung (Aufgabe 3.3.5) Aus Aufgabe 3.3.5 folgt

$$e_n(p) = \frac{r_n(p)}{p} = O(p) \quad \text{für } p < \sqrt{n}.$$

Da hier $\sqrt{500\,000} \approx 707$ und wir bis $p = 25$ messen, liegen wir eindeutig im $O(p)$ -Bereich. Die gemessene lineare Steigerung der Effizienz bestätigt damit die theoretische Vorhersage.

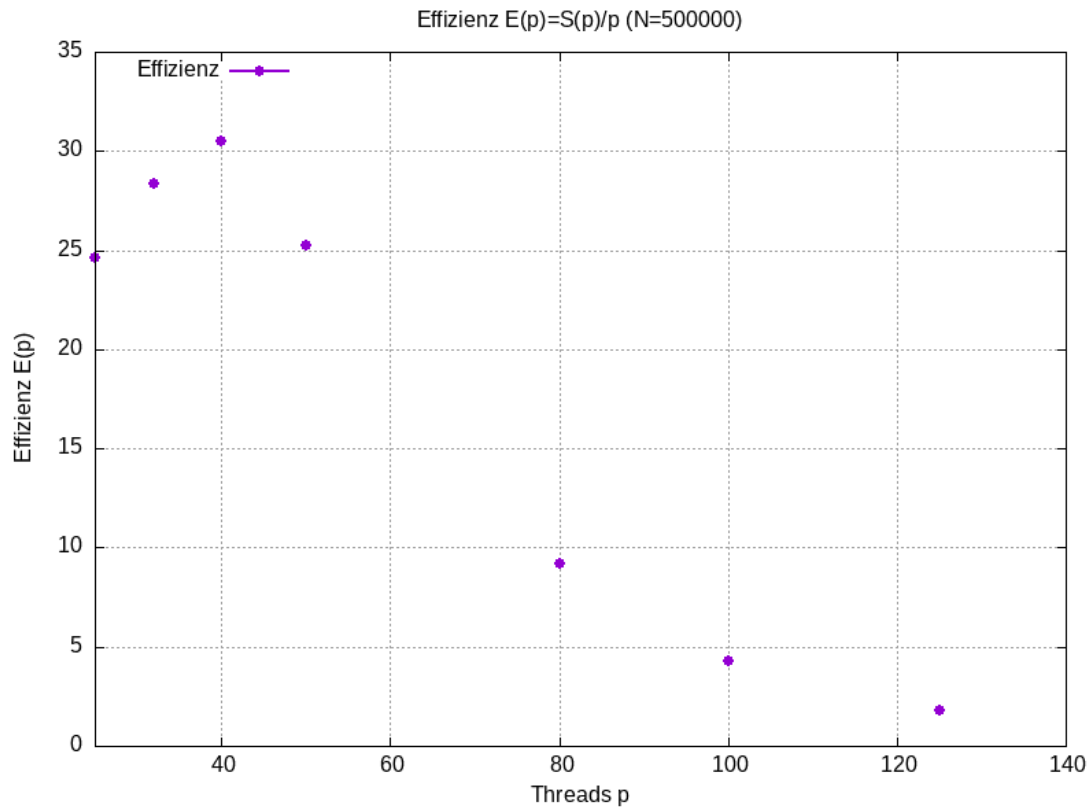


Abbildung 11: Effizienz $E(p)=S(p)/p$ ($N=500000$)

Sobald p deutlich größer wird, kippt die Effizienz. Sie fällt von rund 25 bei $p = 25$, auf unter 2, bei $p = 128$. Grund hierfür ist der, weil für sehr viele Kerne, der fest Merge-Overhead $O(n)$ nicht mehr zu vernachlässigen ist. Die Vorteile durch die Verkleinerung der lokalen Quadrate schrumpfen gegenüber dem Aufwand, alle Teilblöcke wieder zusammenzuführen.

Laufzeit, Beschleunigung und Effizienz als Tabelle für $N = 500\,000$ (Nachtrag)

Tabelle 3: Laufzeit, Speed-up und Effizienz für $N = 500\,000$ (Bubble Sort)

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	%	Wert	%	Wert	%
1	470645	—	1.00	—	1.00	—
2	117949	−74.9	3.99	+299.0	1.995	+99.5
4	29263	−75.2	16.09	+303.0	4.02	+101.5
5	18646	−36.3	25.25	+57.0	5.05	+25.6
8	7294	−60.9	64.49	+155.0	8.06	+59.8
10	4662	−36.1	100.93	+56.5	10.09	+25.2
16	1850	−60.3	254.41	+152.0	15.90	+57.6
20	1169	−36.8	402.46	+58.1	20.12	+26.4
25	743	−36.4	633.65	+57.4	25.35	+26.0
32	518	−30.3	908.93	+43.5	28.40	+12.0
40	386	−25.5	1219.31	+34.2	30.48	+7.3
50	373	−3.4	1261.54	+3.4	25.23	−17.2
80	638	+71.0	737.69	−41.5	9.22	−63.5
100	1104	+73.0	426.33	−42.2	4.26	−53.8
125	2128	+92.8	221.19	−48.1	1.77	−58.5

Welche Variante des Algorithmus ist besser? Warum?

Wenn man den Vergleich für lokale Sortierung macht zwischen `Arrays.sort` mit einer Laufzeit-Äquivalenz von ca. Heapsort mit $O(n \log n)$, und Bubble-Sort mit $O((n/p)^2)$, dann ist in diesem Falle `Array.sort` klar im Vorteil.

Gründe hierzu sind zum Einen, dass es bei `Array.sort()` einen geringeren sequentiellen Aufwand gibt: `Array.sort()` braucht $O(\frac{n}{p} \log \frac{n}{p})$ pro Teilproblem. Bubble-Sort hingegen braucht $O((n/p)^2)$. Für typische n und p wächst $(n/p)^2$ deutlich höher als $(n/p) \log(n/p)$. Das kann man auch an den absoluten Laufzeiten sehen, wobei die Laufzeiten bei Bubble-Sort viel viel höher sind.

Des Weiteren bietet `Array.sort()` ein besseres Parallel-Scaling, denn: Bei `Array.sort()` bleibt man für $p \leq \log n$ nahe am idealen $1/p$ -Beschleunigung, wobei die Effizienz ca. 0,4–0,8 liegt, selbst bei 24 Kernen. Bubble-Sort kann zwar für den Anfang relativ hohe Beschleunigung liefern, bis $O(p^2)$, da die sequentielle Arbeit sehr dominant ist. Aber die absoluten Laufzeiten steigen hierbei dennoch enorm. Zusätzlich bricht die Effizienz spätestens ab mittleren Kernzahlen stark ab, weil das $O(n)$ -Merge dominiert.

Letztlich ist es auch so, dass bei großen n und guten p , selbst bei 128 Kernen, die Läufe von Bubble-Sort Minuten statt Sekunden dauern. Bei `Array.sort()` hingegen liefern dieselben n und p Laufzeiten im Sekundenbereich und bleiben innerhalb von einer vernünftigen Heap-Space-Grenze. Somit bietet Letzteres also auch praktische Zeiten und Speicher.

2.2 Bitonic Sort

Bitonic Sort ist ein paralleler Vergleichssortieralgorithmus, der insbesondere auf Architekturen mit Hypercube effizient implementiert werden kann. Er arbeitet auf Sequenzen der Länge $n = 2^k$ und besteht aus zwei Hauptschritten:

1. Bitonische Folge erzeugen Eine *bitonische Folge* $\{x_0, x_1, \dots, x_{n-1}\}$ ist definiert dadurch, dass sie zunächst (streng) monoton steigt und anschließend (streng) monoton fällt, oder dass sie durch zyklisches Schieben in eine solche Form überführt werden kann. Um eine beliebige Eingabefolge $A[0..n-1]$ bitonisch zu machen, sortiert man rekursiv die erste Hälfte $A[0..n/2-1]$ aufsteigend und die zweite Hälfte $A[n/2..n-1]$ absteigend. Die so erzeugte Gesamtliste ist bitonisch.

2. Bitonisches Mischen (Merge) Ist $B[0..n-1]$ bitonisch, lässt sie sich in zwei sortierte Teillisten zerlegen, indem man für alle $0 \leq i < n/2$

vergleicht und vertauscht $(B[i], B[i + n/2])$

so anwendet, dass jeweils das Minimum in Position i und das Maximum in Position $i + n/2$ verbleibt. Anschließend werden die beiden Hälften $B[0..n/2-1]$ und $B[n/2..n-1]$ jeweils rekursiv bitonisch gemischt. Am Ende dieser Phase liegen alle n Elemente in aufsteigender Reihenfolge vor.

2.2.1 Übungen zum Bitonic Sort

Verdeutlichen Sie sich die Arbeitsweise des parallelen Sortieralgorithmus "Bitonic Merging (BIM)"

Algorithm 2 BitonicSortMain

Require: Eingabegröße N , Threadanzahl P

Ensure: Global sortiertes Array über alle Prozessoren

```
1:  $m\_Array \leftarrow$  neues Array  $[0..P-1][0..N/P-1]$ 
2:  $barrier \leftarrow$  neue CyclicBarrier( $P$ )
3:  $processes \leftarrow$  neues Array  $[0..P-1]$ 
4: Fülle  $m\_Array$  mit Zufallswerten
5: for  $i = 0$  to  $P - 1$  do
6:    $processes[i] \leftarrow$  neue BitonicSortThread( $id = i, m\_Array[i], P, barrier$ )
7: end for
8: for  $i = 0$  to  $P - 1$  do
9:    $processes[i].start()$ 
10: end for
11: for  $i = 0$  to  $P - 1$  do
12:    $processes[i].join()$ 
13: end for
14: for  $i = 0$  to  $P - 1$  do
15:    $m\_Array[i] \leftarrow processes[i].getSortedSubArray()$ 
16: end for
```

Algorithm 3 BitonicSortThread

Require: Subarray local_i der Länge n/p , Prozess-ID i , Gesamtanzahl p , **barrier**

Ensure: local_i ist am Ende sortiert

```
1: procedure RUN
2:   PRESORT( $\text{local}_i$ )                ▷ initiales Sortieren mit Arrays.sort()
3:   SORT(0,  $p$ , true)
4: end procedure
5: procedure SORT( $low$ ,  $count$ ,  $ascending$ )
6:   if  $count > 1$  then
7:     MAKEBITONIC( $low$ ,  $count$ ,  $ascending$ )
8:     BITONICMERGE( $low$ ,  $count$ ,  $ascending$ )
9:   end if
10: end procedure
11: procedure MAKEBITONIC( $low$ ,  $count$ ,  $ascending$ )
12:    $k \leftarrow count/2$ 
13:   if  $i < k$  then
14:     SORT( $low$ ,  $k$ , true)
15:   else
16:     SORT( $low + k$ ,  $k$ , false)
17:   end if
18: end procedure
19: procedure BITONICMERGE( $low$ ,  $count$ ,  $ascending$ )
20:   for  $dist \leftarrow count/2$  downto 1 by halving do
21:      $partner \leftarrow i \oplus dist$ 
22:     Sende  $\text{local}_i$  an  $\text{local}_{partner}$  und empfangen other
23:      $\text{local}_i \leftarrow \text{MERGEANDSWAP}(\text{local}_i, \text{other}, ascending)$ 
24:   end for
25: end procedure
26: procedure GETSORTEDSUBARRAY
27:   return  $\text{local}_i$ 
28: end procedure
```

Bestimmen Sie die Laufzeit des BIM-Algorithmus und vergleichen diese mit dem Odd Even Merge Splitting

Bitonic Sort auf p Prozessoren zerfällt in zwei Phasen:

1. Lokales Sortieren Jeder der p Kerne hält ein Teilarray der Länge

$$m = \frac{n}{p}$$

und sortiert dieses mit einem sequentiellen Vergleichssortierverfahren (z. B. Java's `Arrays.sort()`). Da ein solcher Algorithmus $m \log m$ -Schritte benötigt, ergibt sich pro Kern

$$O(m \log m) = O\left(\frac{n}{p} \log\left(\frac{n}{p}\right)\right).$$

Weil alle Kerne diese Arbeit gleichzeitig leisten, ist dies der dominierende Beitrag der ersten Phase.

2. Bitonisches Zusammenführen (Merge) Die bereits sortierten Blöcke müssen nun zu einer global sortierten Sequenz verschmolzen werden. Bitonic Sort verwendet dafür ein Merge-Netzwerk mit $\log_2 p$ Stufen. In der k -ten Stufe gilt

$$\text{dist} = \frac{p}{2^k}, \quad k = 1, 2, \dots, \log_2 p.$$

Jeder Kern i führt dabei pro Runde folgende Schritte aus:

1. **Referenztausch ($O(1)$):** Der Zeiger auf das lokale Array `locali` wird mit dem Partner $i \oplus \text{dist}$ ausgetauscht.
2. **mergeAndSwap ($O(m)$):** Die beiden sortierten Arrays der Länge m werden elementweise in ein Hilfsarray gemischt und per Dreiecksswap zurückkopiert. Dieser Vorgang benötigt $O(m)$ Schritte.

Da jede der $\log_2 p$ Runden auf jedem Kern insgesamt $O(1) + O(m) = O(m)$ Arbeit verursacht, summiert sich die Merge-Phase zu

$$O(m) \times \log_2 p = O\left(\frac{n}{p} \log p\right).$$

Gesamtlaufzeit Die Gesamtzeit ist die Summe beider Phasen:

$$T_{\text{bitonic}}(n, p) = O\left(\frac{n}{p} \log \frac{n}{p}\right) + O\left(\frac{n}{p} \log p\right) = O\left(\frac{n}{p} [\log \frac{n}{p} + \log p]\right) = O\left(\frac{n}{p} \log n\right),$$

wobei wir im letzten Schritt nutzen, dass $\log(n/p) + \log p = \log n$.

Damit erzielt Bitonic Sort nahezu linearen Speed-up in p , nur abgeschwächt um den Faktor $\log n$.

Beschleunigung

Die *relative Beschleunigung* $r_n(p)$ gibt an, um welchen Faktor Bitonic Sort auf p Kernen schneller läuft als auf einem einzelnen Kern. Sie ist definiert als

$$r_n(p) = \frac{T(n, 1)}{T(n, p)}.$$

1. Einzelkern-Laufzeit Auf einem Kern sortiert man das gesamte Array der Länge n in

$$T(n, 1) = O(n \log n).$$

2. Mehrkern-Laufzeit Auf p Kernen hat Bitonic Sort die Laufzeit

$$T(n, p) = O\left(\frac{n}{p} \log n\right),$$

weil sich die Arbeit $n \log n$ in etwa gleichmäßig auf die p Kerne verteilt.

3. Speedup Setzt man diese Größen in die Definition ein, kürzen sich n und $\log n$ heraus:

$$r_n(p) = \frac{O(n \log n)}{O\left(\frac{n}{p} \log n\right)} = O\left(\frac{n \log n}{(n/p) \log n}\right) = O(p).$$

Daraus folgt, dass Bitonic Sort nahezu *linear* in p skaliert: Verdoppelt man die Kernzahl, verdoppelt sich annähernd auch der Speedup.

—

Effizienz

Die *Effizienz* $e_n(p)$ misst, wie viel der theoretisch möglichen linearen Beschleunigung pro Kern tatsächlich erreicht wird:

$$e_n(p) = \frac{r_n(p)}{p}.$$

Mit $r_n(p) = O(p)$ ergibt sich

$$e_n(p) = \frac{O(p)}{p} = O(1).$$

Intuitive Erklärung

- **Idealfall:** Ohne jeglichen Overhead würde jeder Kern exakt seinen Anteil an Arbeit leisten, der Speedup wäre $r_n(p) = p$ und damit $e_n(p) = 1$.
- **Praktische Overheads:** Bitonic Sort führt $\log p$ Merge-Runden durch, was einen kleinen zusätzlichen Aufwand erzeugt. Da dieser Overhead aber nur einen weiteren Poly-Log-Faktor bringt, bleibt $e_n(p)$ asymptotisch konstant.
- **Ergebnis:** Bitonic Sort erreicht eine *konstante Effizienz* $e_n(p) = \Theta(1)$. In Messungen zeigt sich typischerweise $e_n(p) \approx 0,8\text{--}0,9$ für moderate Kernzahlen, ohne mit wachsendem p deutlich abzufallen.

Vergleich mit Odd Even Merge-Splitting

Wir fassen zunächst die theoretischen Laufzeiten noch einmal zusammen:

- **Odd-Even Merge-Splitting (OEMS):**

$$T_{\text{OEMS}}(n, p) = O\left(\frac{n}{p} \log \frac{n}{p}\right) + O(n) = O(\max\{\frac{n}{p} \log \frac{n}{p}, n\}).$$

- Für $p \leq \log n$ ist $T_{\text{OEMS}}(n, p) = O\left(\frac{n}{p} \log n\right)$.
- Für $p > \log n$ dominiert der $O(n)$ -Term und $T_{\text{OEMS}}(n, p) = O(n)$.

- **Bitonic Sort:**

$$T_{\text{bitonic}}(n, p) = O\left(\frac{n}{p} \log \frac{n}{p}\right) + O\left(\frac{n}{p} \log p\right) = O\left(\frac{n}{p} \log n\right).$$

- Für alle $1 \leq p \leq n$ fällt $T_{\text{bitonic}}(n, p)$ weiterhin mit $1/p$, nur gebremst durch $\log n$.

Speedup (Beschleunigung)

$$r_{\text{OEMS}}(p) = \begin{cases} O(p), & p \leq \log n, \\ O(\log n), & p > \log n, \end{cases} \quad r_{\text{bitonic}}(p) = O(p) \quad \forall p.$$

Fazit: OEMS skaliert nur linear bis $p \approx \log n$ und flacht dann ab, Bitonic bleibt linear in p .

Effizienz

$$e_{\text{OEMS}}(p) = \begin{cases} O(1), & p \leq \log n, \\ O\left(\frac{\log n}{p}\right), & p > \log n, \end{cases} \quad e_{\text{bitonic}}(p) = O(1) \quad \forall p.$$

Fazit: OEMS verliert ab $p > \log n$ Effizienz, während Bitonic Sort seine konstante Auslastung aller Kerne beibehält.

Schlussfolgerung Bitonic Sort bietet im asymptotischen Vergleich eine bessere Skalierung, insbesondere für viele Prozessoren ($p > \log n$):

- *OEMS* bleibt wegen des festen Merge-Overheads von $O(n)$ bei höheren Kernzahlen auf einem konstanten Laufzeitsniveau und kann keine weitere Beschleunigung erzielen.
- *Bitonic Sort* nutzt jeden zusätzlichen Kern weiterhin aus (bis hin zu $p = O(n)$), da die Gesamtarbeit immer noch proportional zu n/p fällt.

2.2.2 Assignment 2

In Assignment 2 wird der Bitonic Merge-Splitting (BIM)–Algorithmus untersucht. Dazu sind folgende Schritte durchzuführen:

1. **Implementierung:** Der sequentielle Sortierschritt verwendet Java's `Arrays.sort()`. Die BIM-Logik wird in einer Klasse `BitonicSortMain` (Treibercode) und einer Klasse `BitonicSortThread` (Parallelcode) umgesetzt.
2. **Laufzeitmessungen:**
 - Ein- und dieselbe Zufalls-Eingabe (mit gegebener Größe N) wird mehrfach (dreimal) sortiert; jeweils wird die schnellste Laufzeit verwendet.
 - Die Anzahl der Threads/Kerne p wird variiert — in diesem Versuch für $p = 1, 2, 4, 8, \dots, 1024$ (Potenzen von zwei).
 - Gemessen wird für zwei Problemgrößen

$$N_1 = 921\,600\,000, \quad N_2 = 1\,843\,200\,000.$$

3. **Auswertung:**

- Bestimmung der *relativen Beschleunigung* $r_n(p) = T(n, 1)/T(n, p)$ und der *Effizienz* $e_n(p) = r_n(p)/p$.

- Grafische Darstellung von Laufzeit, Speed-up und Effizienz in Abhängigkeit von p .
 - Vergleich der gemessenen Kurven mit den theoretischen Vorhersagen aus Abschnitt 4.3.2 (Bitonic Sort) und Abschnitt 3.3.2 (Odd–Even Merge Splitting).
4. **Validierung:** Ein automatischer Check stellt sicher, dass nach Beendigung alle n Eingangselemente korrekt sortiert und vollständig vorhanden sind.

Problemgröße $n = 921\,600\,000$

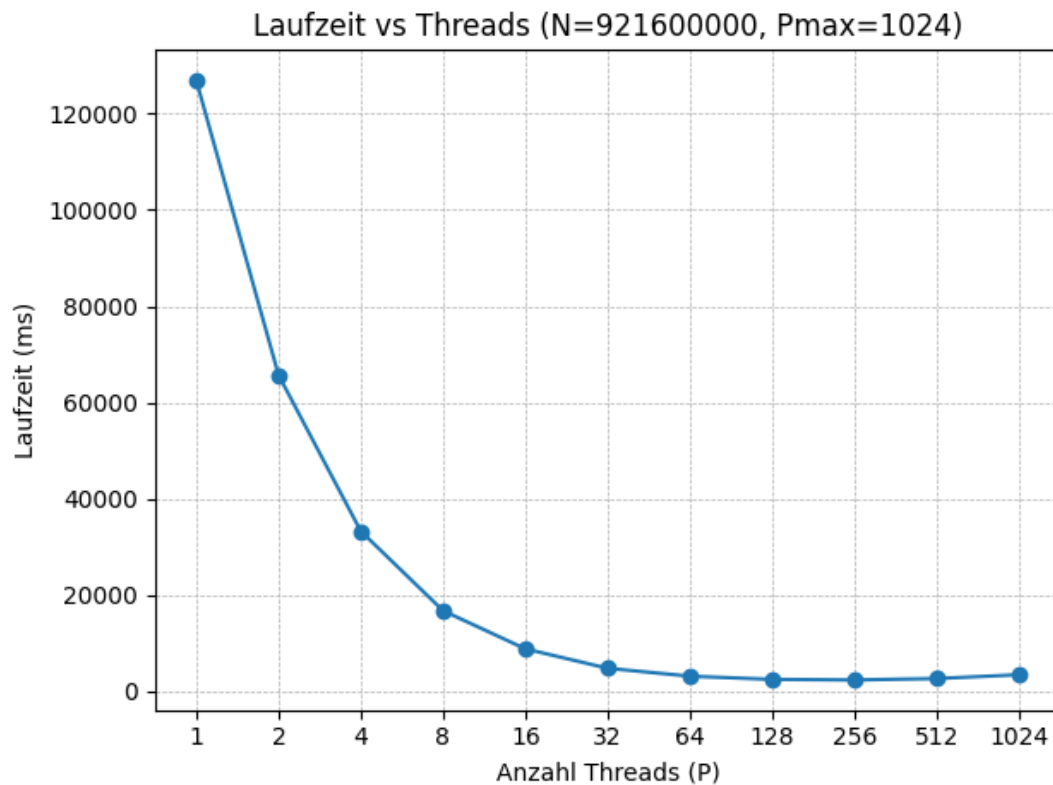


Abbildung 12: Laufzeit vs. Threads (N=921600000, Pmax=1024)

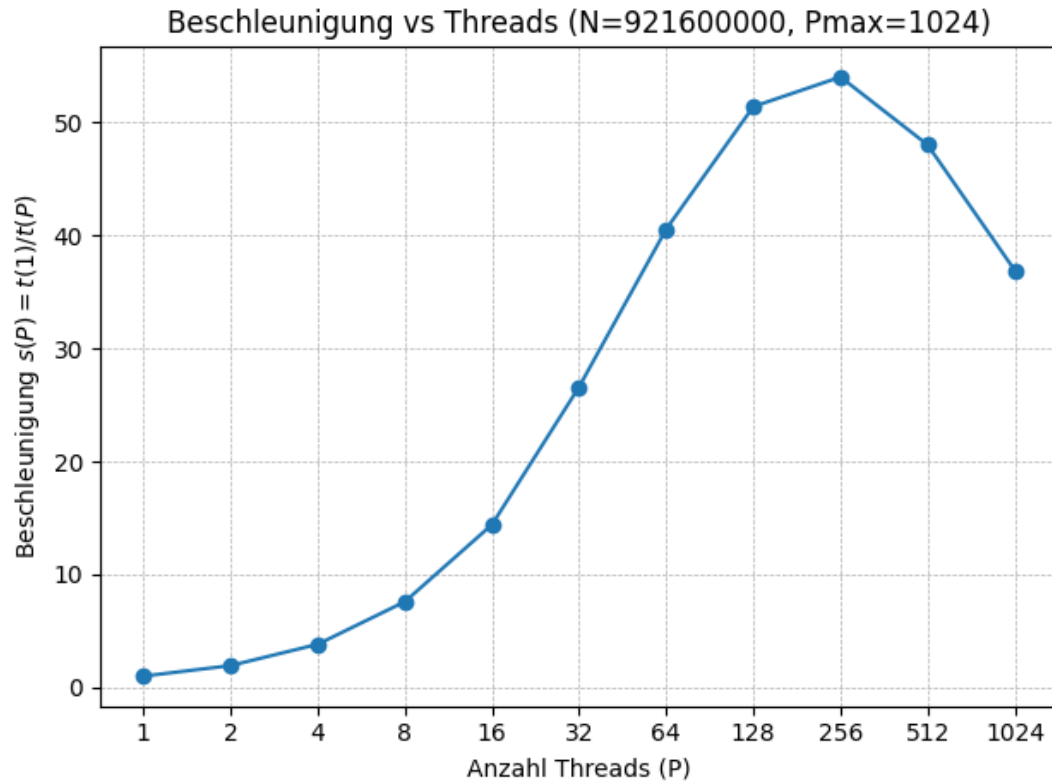


Abbildung 13: Beschleunigung vs. Threads (N=921600000, Pmax=1024)

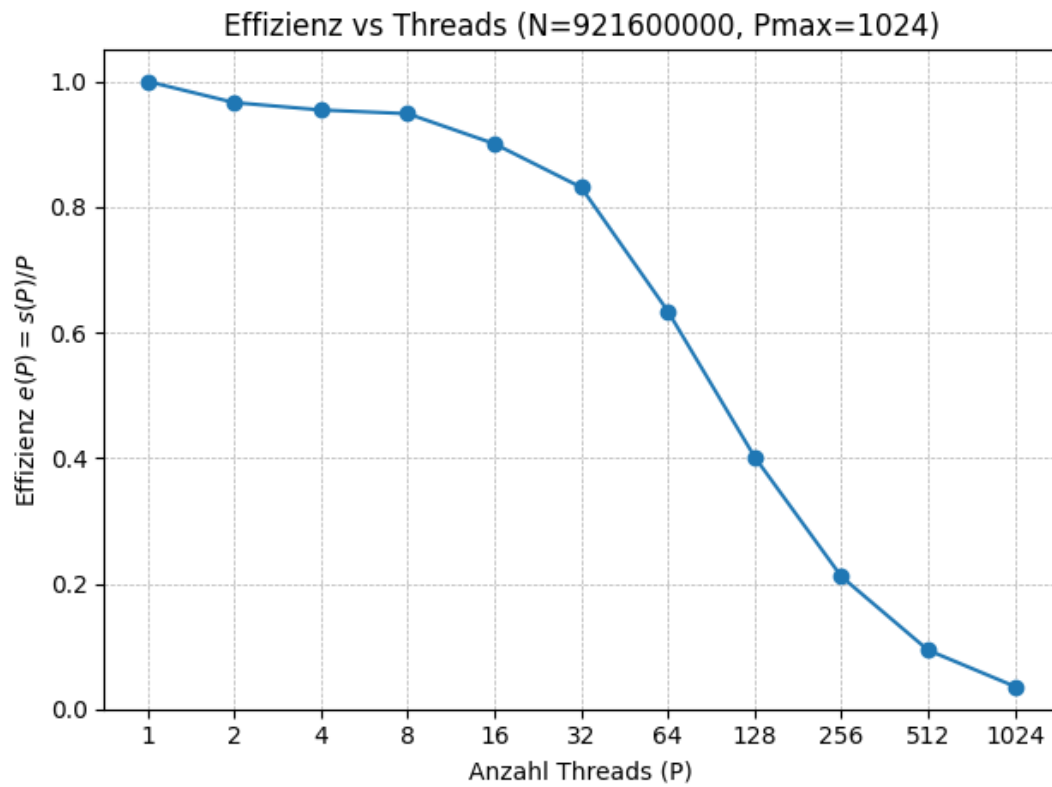


Abbildung 14: Effizienz vs. Threads (N=921600000, Pmax=1024)

Tabelle 4: Laufzeit, Speed-up und Effizienz für Bitonic Merge (gemessen vs. theoretisch)

p	$T(p)$ [ms]	ΔT [%]	$S(p)$	ΔS [%]	$e(p)$	Δe [%]	S_{theo}	e_{theo}
1	126967	—	1.00	—	1.00	—	1.00	1.00
2	65699	48.2	1.93	93.0	0.97	3.0	2.00	1.00
4	33256	49.4	3.82	98.2	0.95	2.1	4.00	1.00
8	16731	49.7	7.59	98.6	0.95	0.0	8.00	1.00
16	8811	47.3	14.41	90.0	0.90	5.3	16.00	1.00
32	4773	45.8	26.60	84.5	0.83	7.8	32.00	1.00
64	3130	34.4	40.56	52.5	0.63	24.1	64.00	1.00
128	2470	21.0	51.40	26.7	0.40	36.5	128.0	1.00
256	2350	4.9	54.01	5.1	0.21	47.5	256.0	1.00
512	2644	—	12.5	48.01	0.09	57.1	512.0	1.00
1024	3439	30.1	36.90	23.1	0.04	55.6	1024	1.00

Vergleich Messung vs. Theorie In der Tabelle sieht man für jede Kernzahl p zunächst die gemessene Laufzeit und direkt daneben, um wie viel Prozent sie gegenüber der vorherigen Konfiguration ab- oder angestiegen ist. Danach folgt die errechnete Beschleunigung mit seiner prozentualen Änderung, sowie die Effizienz samt deren Änderung. In den letzten beiden Spalten stehen die theoretisch idealen Werte: eine lineare Beschleunigung und eine konstante Effizienz.

Wesentlich ist: Bis etwa $p = 16$ wächst die gemessene Beschleunigung fast linear und die Effizienz bleibt hoch (nahe 0,9 - 1,0), also sehr nahe am Optimum. Ab $p = 32$ jedoch verlangsamt sich der Zugewinn deutlich - der prozentuale Differenz von Laufzeit und Beschleunigung nimmt ab, und die Differenz fällt auf die Werte unter 0,7 ab. Das zeigt klar, dass der Kommunikations- und Merge-Overhead bei höheren Prozessorzahlen die Skalierung bremst. Die theoretischen Werte dienen hier als Referenz: Während Bitonic Merge in der Praxis bei kleinen bis mittleren p nahezu ideal skaliert, kann es ab einer gewissen Kernzahl nicht mehr die volle lineare Beschleunigung und konstante Auslastung erreichen.

Vergleich BIM vs. OEM Um einen „Äpfel-mit-Äpfel“-Vergleich zu ermöglichen, werden Bitonic Merge und Odd-Even Merge-Splitting bei

- gleicher Problemgröße $N = 921\,600\,000$,
- identischer Prozessorzahl $p \in \{1, 2, 4, 8, 16\}$,

- und demselben lokalen Sortierschritt (`Arrays.sort()`)

nebeneinander betrachtet.

2^*p	T_{bitonic}		S_{bitonic}		e_{bitonic}		T_{OEMS}		S_{OEMS}		e_{OEMS}	
	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%
1	126967	—	1.00	—	1.00	—	127082	—	1.00	—	1.00	—
2	65699	−48.2	1.93	+93	0.97	−3	66627	−47.6	1.91	+91	0.96	−4
4	33256	−49.4	3.82	+98	0.96	−1	37471	−43.8	3.39	+77	0.85	−11
8	16731	−49.7	7.59	+99	0.95	−1	22752	−39.3	5.59	+65	0.70	−18
16	8811	−47.3	14.40	+90	0.90	−5	15266	−32.9	8.32	+49	0.52	−26

In der Tabelle stehen für jede Prozessorzahl p die gemessene Laufzeit T , der daraus berechnete Speed-up $S = T(1)/T(p)$ und die Effizienz $e = S/p$. Neben jedem Wert ist jeweils die prozentuale Veränderung gegenüber der vorhergehenden Zeile angegeben. Der linke Block zeigt die Kennzahlen für Bitonic Merge, der rechte für Odd–Even Merge–Splitting (OEMS).

Man erkennt, dass bei kleinen p beide Algorithmen vergleichbare Speed-ups liefern, Bitonic Merge aber ab $p = 4$ zunehmend besser skaliert. Die Effizienz von OEMS sinkt schon ab $p = 4$ deutlich, während Bitonic Merge bis $p = 16$ über 0,9 bleibt. Das deutet darauf hin, dass Bitonic Merge den parallelen Merge–Overhead besser auf die Kerne verteilt, während OEMS—bedingt durch seinen festen $O(n)$ -Merge-Anteil—ab einer gewissen Kernzahl nicht mehr sinnvoll weiter beschleunigt.

Problemgröße $n = 1\,843\,200\,000$

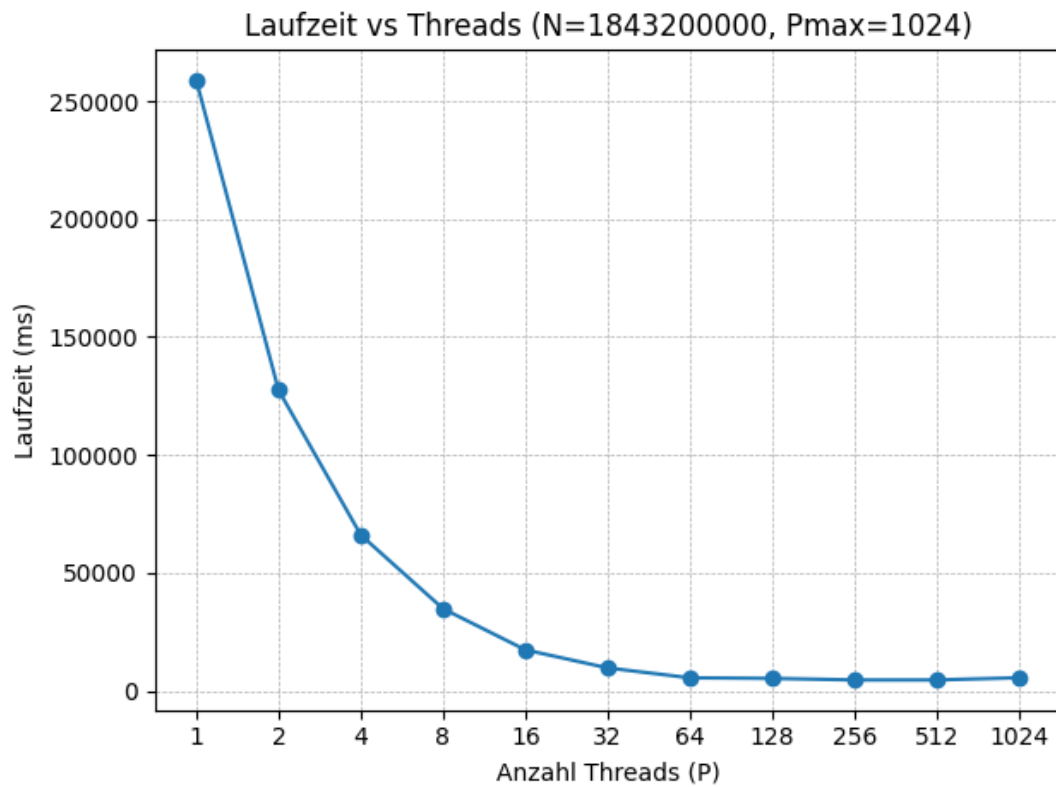


Abbildung 15: Laufzeit vs. Threads (N=1843200000, Pmax=1024)

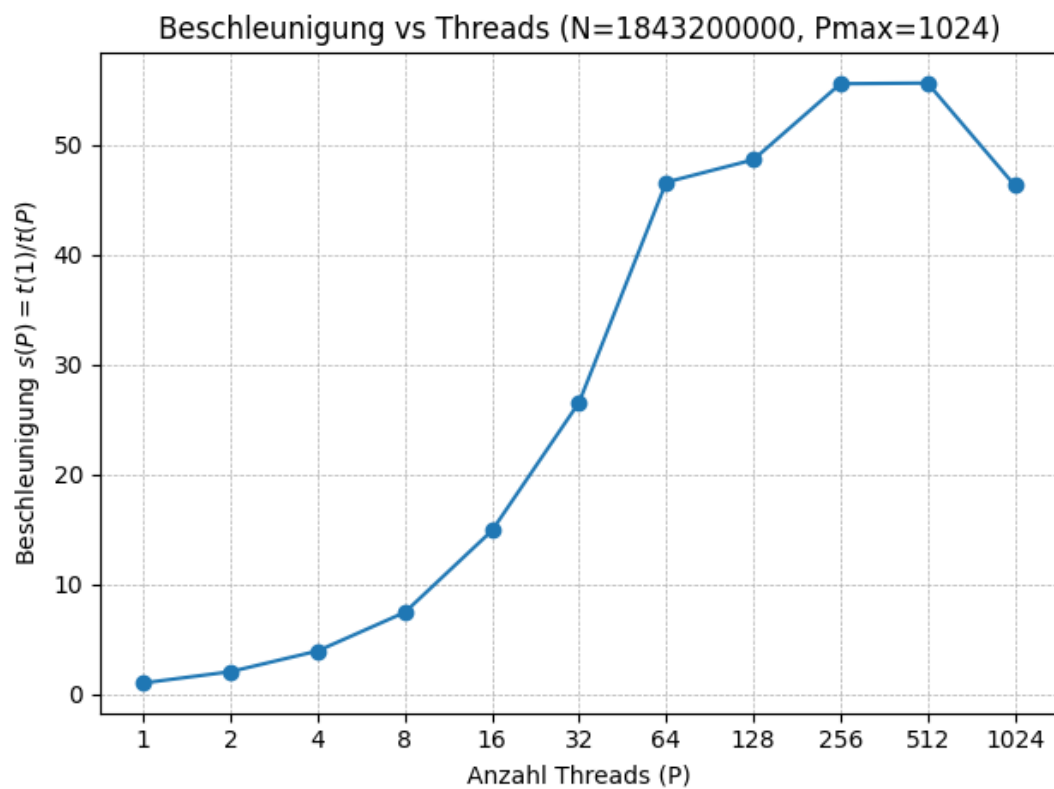


Abbildung 16: Beschleunigung vs. Threads (N=1843200000, Pmax=1024)

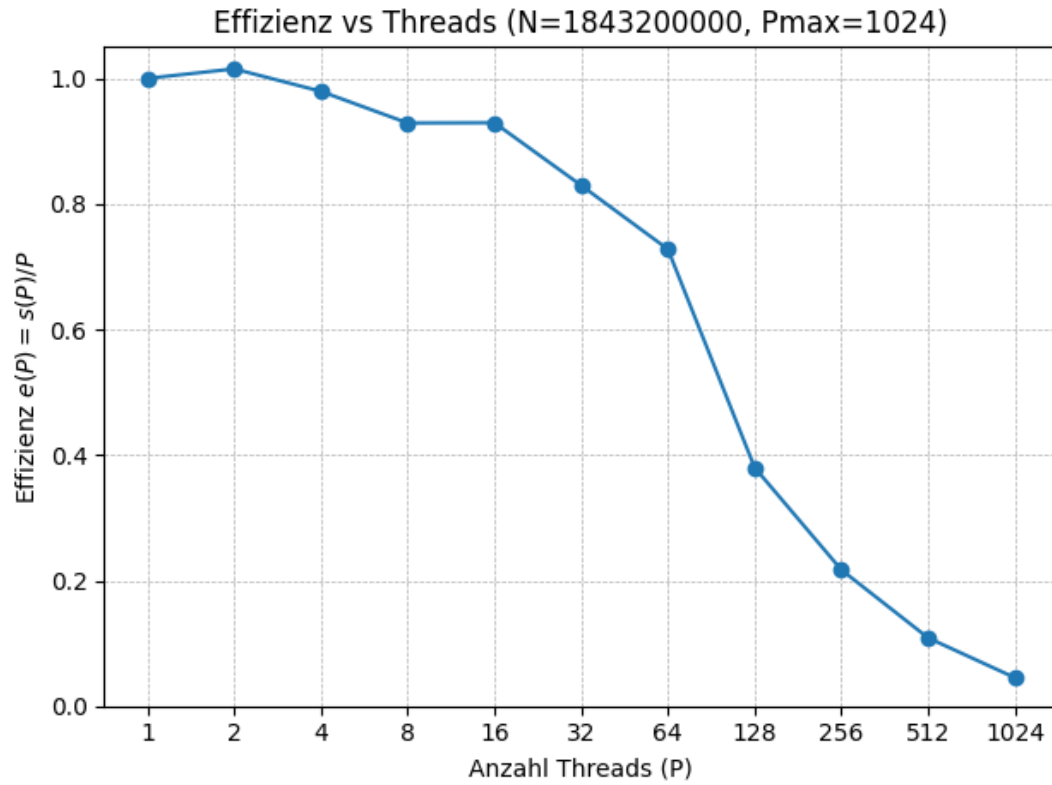


Abbildung 17: Effizienz vs. Threads (N=1843200000, Pmax=1024)

Tabelle 5: Laufzeit, Speed-up und Effizienz von Bitonic Merge (gemessen vs. theoretisch) für $N = 1\,843\,200\,000$

2^*p	$T(p)$ [ms]		$S(p)$		$e(p)$		S_{theo}		e_{theo}	
	Wert	%	Wert	%	Wert	%	Wert	—	Wert	—
1	258907	—	1.00	—	1.00	—	1.00		1.00	
2	127522	-50.8	2.03	+103.0	1.015	+1.5	2.00		1.00	
4	66088	-48.2	3.92	+93.6	0.980	-3.4	4.00		1.00	
8	34840	-47.3	7.43	+89.5	0.929	-5.1	8.00		1.00	
16	17412	-50.0	14.87	+100.1	0.930	0.1	16.00		1.00	
32	9753	-44.0	26.54	+78.5	0.829	-10.8	32.00		1.00	
64	5558	-43.0	46.56	+75.4	0.727	-12.3	64.00		1.00	
128	5324	-4.2	48.62	+4.4	0.379	-47.9	128.0		1.00	
256	4661	-12.4	55.56	+14.3	0.217	-42.1	256.0		1.00	
512	4656	-0.1	55.61	+0.1	0.109	-49.8	512.0		1.00	
1024	5593	+20.1	46.28	-16.8	0.045	-58.8	1024.		1.00	

Vergleich Messung vs. Theorie Man erkennt, dass Bitonic Merge bis $p = 16$ sehr gut skaliert: Laufzeit halbiert sich annähernd bei jeder Verdopplung von p , Speed-up wächst fast linear und Effizienz bleibt nahe 1. Ab $p = 32$ bricht die Effizienz jedoch um über 10 % pro Schritt ein und die Laufzeitvorteile flachen

ab – ein Zeichen dafür, dass der nicht-parallele Merge-Overhead und die Synchronisation bei vielen Kernen ins Gewicht fallen. Damit bestätigt sich auch für die größere Problemgröße die theoretische Vorhersage $T(n, p) = O(\frac{n}{p} \log n)$ nur bis zu einer mittleren Kernzahl, während danach konstante Overheads dominieren.

Vergleich BIM vs. OEM

Tabelle 6: Laufzeit, Speed-up und Effizienz von Bitonic Merge (BIM) vs. Odd–Even Merge–Splitting (OEMS)

2^*p	T_{BIM} [ms]		S_{BIM}		e_{BIM}		T_{OEMS} [ms]		S_{OEMS}		e_{OEMS}	
	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%
1	258907	—	1.00	—	1.00	—	258615	—	1.00	—	1.00	—
2	127522	-50.7	2.03	+103.0	1.02	+1.5	135966	-47.4	1.90	+90.2	0.95	-4.9
4	66088	-48.2	3.92	+93.0	0.98	-3.5	81071	-40.4	3.19	+67.7	0.80	-16.1
8	34840	-47.3	7.43	+89.7	0.93	-5.2	55927	-31.0	4.62	+45.0	0.58	-27.5
16	17412	-50.0	14.87	+100.1	0.93	+0.0	45582	-18.5	5.67	+22.7	0.35	-38.7

Bis $p = 4$ liefern beide Algorithmen sehr ähnliche Speed-ups. Ab $p = 8$ skaliert BIM deutlich besser: Die Laufzeit reduziert sich hier bei jeder Verdopplung um rund 47 % (OEMS nur um 31 %), der Speed-up wächst um fast 90 % (OEMS um 45 %) und die Effizienz fällt bei BIM nur um etwa 5 % ab, während sie bei OEMS um über 25 % einbricht. Damit steht fest, dass Bitonic Merge den Merge-Overhead auf viele Kerne effektiver verteilt und auch bei höheren Kernzahlen eine hohe Auslastung beibehält, während OEMS ab mittleren p nicht mehr nachhaltig beschleunigt.

2.3 Merge Sort

2.3.1 Übungen zum parallelen Merge Sort

Theoretische Überlegung der Laufzeit, Beschleunigung und Effizienz von ParMergeSort(parMerge)

Wir betrachten ein Array der Länge n . Parallel-Merge-Sort besteht im Wesentlichen aus *zwei* Schritten:

1. Zwei rekursive Sortieraufrufe auf Teilprobleme der Größe $n/2$, die *parallel* gestartet werden.

2. Ein paralleler Merge-Schritt, der insgesamt $\Theta(n)$ Vergleiche und Kopien durchführt.

Die Gesamtzahl aller Operationen (die *Arbeit* $T_1(n)$) erfüllt die Rekurrenz

$$T_1(n) = 2T_1\left(\frac{n}{2}\right) + \Theta(n).$$

Nach dem Master-Theorem (Fall 2) folgt daraus

$$T_1(n) = \Theta(n \log n).$$

Auf unendlich vielen Prozessoren bestimmt der *kritische Pfad* (auch *Span* $T_\infty(n)$) die Laufzeit:

- Die beiden Sortierrufe auf Größe $n/2$ laufen vollständig parallel, tragen also $T_\infty(n/2)$ zum Span bei.
- Im Merge-Schritt wird einmal eine binäre Suche auf Teilgröße $\frac{n}{2}$ ausgeführt ($\Theta(\log n)$), dann zwei Teil-Merges parallel gestartet, aber nur der längere Pfad trägt zum kritischen Pfad bei.

Daher gilt die Rekurrenz

$$T_\infty(n) = T_\infty\left(\frac{n}{2}\right) + \Theta(\log n),$$

deren Lösung (z. B. durch wiederholtes Einsetzen oder Master-Theorem) ist

$$T_\infty(n) = \Theta(\log^2 n).$$

Sei $T_P(n)$ die Laufzeit auf P Prozessoren. Aus der Work-Span-Formel folgt näherungsweise:

$$T_P(n) \approx \frac{T_1(n)}{P} + T_\infty(n) = \Theta\left(\frac{n \log n}{P} + \log^2 n\right).$$

Der *Speedup* S_P gegenüber der sequentiellen Laufzeit ist

$$S_P = \frac{T_1(n)}{T_P(n)} \approx \frac{n \log n}{\frac{n \log n}{P} + \log^2 n} = \frac{1}{\frac{1}{P} + \frac{\log^2 n}{n \log n}} = \frac{1}{\frac{1}{P} + \frac{\log n}{n}}.$$

Die *Effizienz* E_P ist definiert als

$$E_P = \frac{S_P}{P} = \frac{1}{P \left(\frac{1}{P} + \frac{\log n}{n} \right)} = \frac{1}{1 + \frac{P \log n}{n}}.$$

Theoretische Überlegung der Laufzeit, Beschleunigung und Effizienz von ParMergeSort(seqMerge)

Wir betrachten einen Eingabearray der Größe n . Anstelle des parallelen `parMerge()` verwenden wir hier den rein sequentiellen Merge-Schritt `seqMerge()`.

Die Gesamtzahl aller Elementaroperationen (Work) folgt der Rekurrenz

$$T_1(n) = 2 T_1\left(\frac{n}{2}\right) + \Theta(n) \implies T_1(n) = \Theta(n \log n).$$

Der kritische Pfad (Span) ergibt sich aus

- zwei parallelen Teilaufrufen auf Größe $n/2$, also $T_\infty(n/2)$,
- einem anschließenden seriellen Merge in $\Theta(n)$.

Daher

$$T_\infty(n) = T_\infty\left(\frac{n}{2}\right) + \Theta(n) \text{ Master-Theorem } T_\infty(n) = \Theta(n).$$

Aus der Work-Span-Formel

$$T_P(n) \approx \frac{T_1(n)}{P} + T_\infty(n) = \Theta\left(\frac{n \log n}{P} + n\right).$$

$$S_P = \frac{T_1(n)}{T_P(n)} \approx \frac{n \log n}{\frac{n \log n}{P} + n} = \frac{1}{\frac{1}{P} + \frac{1}{\log n}}.$$

- Für $P \ll \log n$ ist $\frac{1}{P}$ dominant, also $S_P \approx P$ (nahe an idealer Skalierung).
- Für $P \gg \log n$ dominiert $\frac{1}{\log n}$ und S_P saturiert bei $S_P \approx \log n$.

$$E_P = \frac{S_P}{P} = \frac{1}{P\left(\frac{1}{P} + \frac{1}{\log n}\right)} = \frac{1}{1 + \frac{P}{\log n}}.$$

- Für $P \ll \log n$ gilt $E_P \approx 1$.
- Für $P \approx \log n$ beginnt E_P stark zu sinken, da der serielle Merge zum Flaschenhals wird.

Fazit: Mit `seqMerge()` statt `parMerge()` reduziert sich die maximale Parallelität auf $\Theta(\log n)$, und die Effizienz fällt ab $P \approx \log n$ deutlich ab.

Vergleich ParMergeSort(parMerge) vs. BIM

Kennzahl	Bitonic-Merge	Paralleler Merge Sort
Arbeit $T_1(n)$	$O(n \log n)$	$O(n \log n)$
Span $T_\infty(n)$	$O(\log^2 n)$	$O(\log^2 n)$
Laufzeit $T_P(n)$	$O(\frac{n}{P} \log n)$	$O(\frac{n \log n}{P} + \log^2 n)$
Speedup S_P	$\Theta(P)$	$\frac{n \log n}{\frac{n \log n}{P} + \log^2 n}$
Effizienz E_P	$\Theta(1)$	$\frac{1}{1 + \frac{P}{\log n}}$

Tabelle 7: Vergleich der parallelen Komplexitäten

Erläuterung

- **Bitonic Sort:**

- *Merge-Phase:* Vollständig paralleles Netzwerk mit $\log p$ Stufen, Gesamt-Span $(n/p) \log p$.
- *Laufzeit:* $\Theta(\frac{n}{p} \log n)$, da $\log(n/p) + \log p = \log n$.
- *Speedup* und *Effizienz* bleiben nahezu ideal: $S_P = \Theta(P)$ und $E_P = \Theta(1)$.

- **Paralleler Merge Sort:**

- *Sort-Phase:* Zwei rekursive Aufrufe auf Größe $n/2$ in *parallel fork*, gefolgt von einem parallelen Merge.
- *Merge-Span:* Pro Ebene $O(\log n)$ durch Pivot-Teilung und Binärsuche, insgesamt $T_\infty(n) = T_\infty(n/2) + O(\log n) = O(\log^2 n)$.
- *Laufzeit:* Arbeit $O(n \log n)$ auf P Kerne verteilt plus kritischer Pfad $\log^2 n$, also

$$T_P(n) = O\left(\frac{n \log n}{P} + \log^2 n\right).$$

- *Speedup:* Linear nur bis $P = O(\log n)$, dann Sättigung bei $O(\log n)$.
- *Effizienz:* Fast 1 für $P \ll \log n$, fällt ab für $P \gtrsim \log n$.

Vergleich ParMergeSort(parMerge) vs. OEMS

Kennzahl	Paralleler Merge Sort	Odd–Even Split-Merge Sort
Arbeit $T_1(n)$	$O(n \log n)$	$O(n \log n)$
Span $T_\infty(n)$	$O(\log^2 n)$	$O(n)$
Laufzeit $T_P(n)$	$O\left(\frac{n \log n}{P} + \log^2 n\right)$	$T_n(P) = \begin{cases} O\left(\frac{n}{P} \log n\right), & P \leq \log n, \\ O(n), & P > \log n \end{cases}$
Speedup S_P	$\frac{n \log n}{\frac{n \log n}{P} + \log^2 n}$	$\begin{cases} \Theta(P), & P \leq \log n, \\ \Theta(\log n), & P > \log n \end{cases}$
Effizienz E_P	$\frac{1}{1 + \frac{P}{\log n}}$	$\begin{cases} \Theta(1), & P \leq \log n, \\ O\left(\frac{\log n}{P}\right), & P > \log n \end{cases}$

Tabelle 8: Komplexitäten im Vergleich

- **Paralleler Merge Sort** auf P Kernen:

$$T_P(n) = O\left(\frac{n \log n}{P} + \log^2 n\right), \quad S_P = \Theta(\min\{P, \log n\}), \quad E_P = O\left(\frac{1}{1 + \frac{P}{\log n}}\right).$$

Lineares Scalen nur bis $P = O(\log n)$, da der $\log^2 n$ -Merge-Span dominiert.

- **Odd–Even Split-Merge Sort** auf P Kernen:

$$T_P(n) = \begin{cases} O\left(\frac{n}{P} \log \frac{n}{P}\right) + O(n) = O\left(\frac{n}{P} \log n\right), & P \leq \log n, \\ O(n), & P > \log n, \end{cases}$$

$$S_P = \begin{cases} \Theta(P), & P \leq \log n, \\ \Theta(\log n), & P > \log n, \end{cases} \quad E_P = \begin{cases} \Theta(1), & P \leq \log n, \\ O\left(\frac{\log n}{P}\right), & P > \log n. \end{cases}$$

Fast perfekter Speedup und konstante Effizienz bis $P = O(\log n)$.

2.3.2 Assignment 3

Problemgröße $n = 921\,600\,000$ - Messung

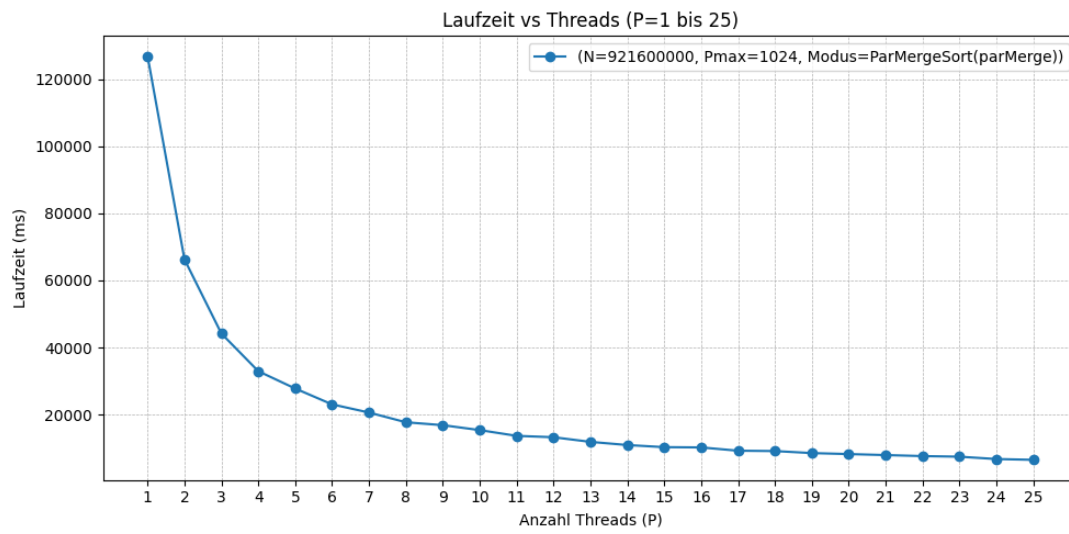


Abbildung 18

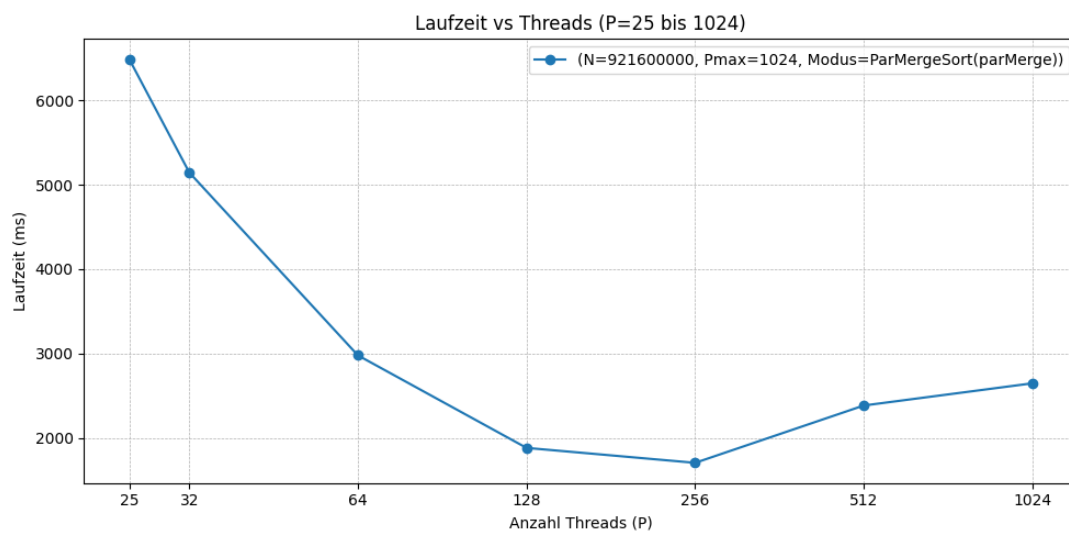


Abbildung 19

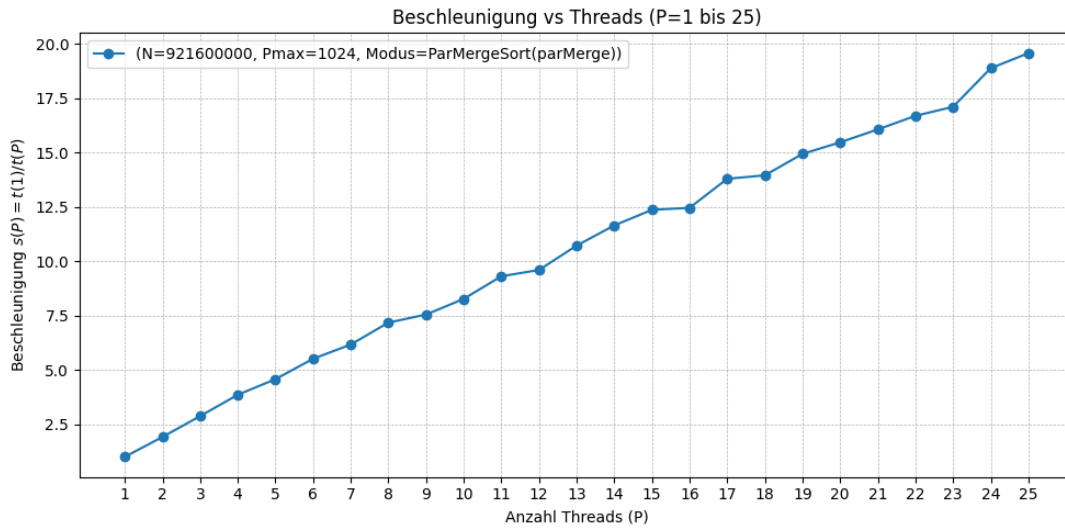


Abbildung 20

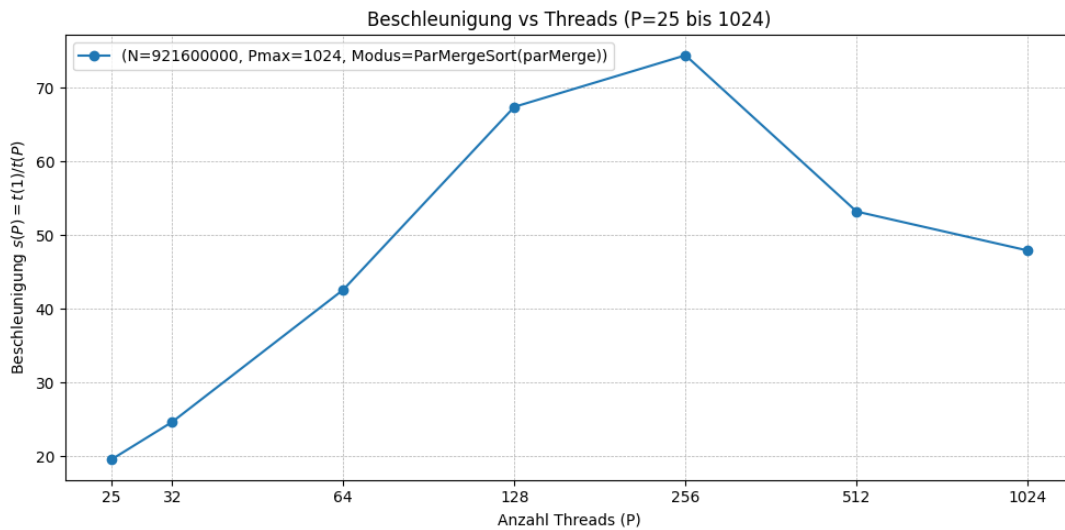


Abbildung 21

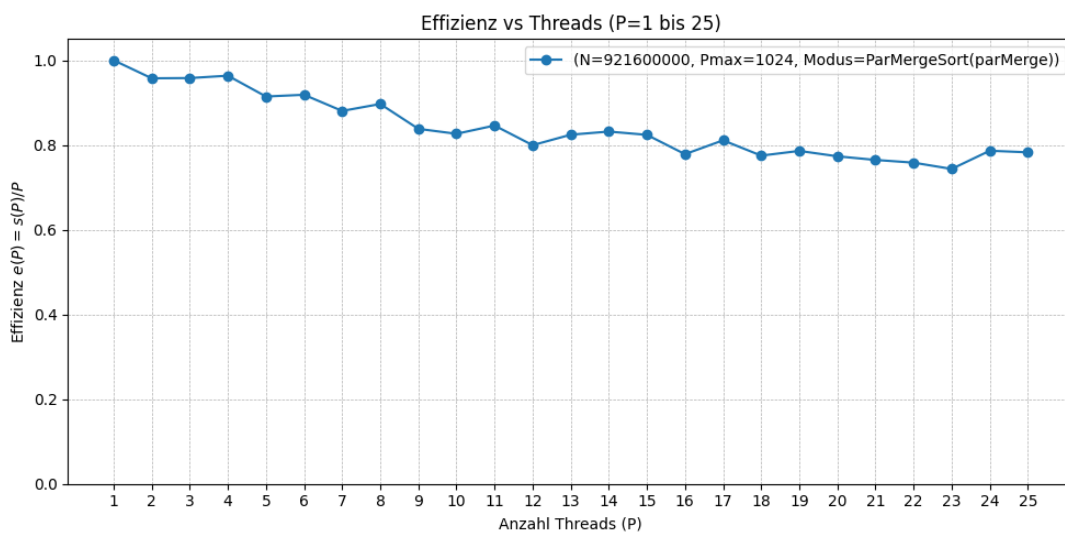


Abbildung 22

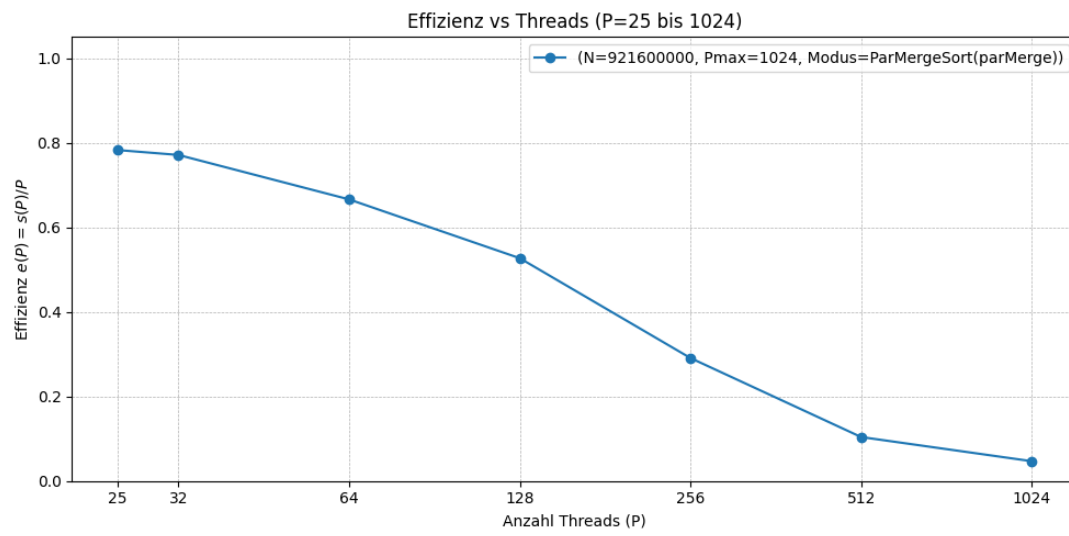


Abbildung 23

Tabelle 9: Laufzeit, Speed-up und Effizienz von ParMergeSort (parMerge) für $N = 921\,600\,000$

p	$T(p)$ [ms]		$S(p)$		$e(p)$		S_{theo}		e_{theo}	
	Wert	%	Wert	%	Wert	%	Wert	—	Wert	—
1	126924	—	1.00	—	1.000	—	1.00	—	1.00	—
2	66284	−47.8	1.91	+91.5	0.957	−4.3	2.00	—	1.00	—
3	44165	−33.4	2.87	+50.1	0.958	+0.1	3.00	—	1.00	—
4	32931	−25.4	3.85	+34.1	0.964	+0.6	4.00	—	1.00	—
5	27761	−15.7	4.57	+18.6	0.914	−5.1	5.00	—	1.00	—
6	23031	−17.0	5.51	+20.5	0.919	+0.4	6.00	—	1.00	—
7	20596	−10.6	6.16	+11.8	0.880	−4.2	7.00	—	1.00	—
8	17692	−14.1	7.18	+16.4	0.897	+1.9	8.00	—	1.00	—
9	16827	−4.9	7.55	+5.1	0.838	−6.5	9.00	—	1.00	—
10	15359	−8.7	8.27	+9.6	0.826	−1.4	10.00	—	1.00	—
11	13641	−11.2	9.30	+12.6	0.846	+2.4	11.00	—	1.00	—
12	13229	−3.0	9.60	+3.1	0.800	−5.5	12.00	—	1.00	—
13	11845	−10.5	10.72	+11.7	0.824	+3.1	13.00	—	1.00	—
14	10901	−8.0	11.64	+8.7	0.832	+0.9	14.00	—	1.00	—
15	10270	−5.8	12.36	+6.1	0.824	−0.9	15.00	—	1.00	—
16	10192	−0.8	12.45	+0.8	0.778	−5.5	16.00	—	1.00	—
17	9206	−9.7	13.78	+10.7	0.811	+4.2	17.00	—	1.00	—
18	9100	−1.2	13.95	+1.2	0.775	−4.5	18.00	—	1.00	—
19	8499	−6.6	14.93	+7.1	0.786	+1.4	19.00	—	1.00	—
20	8207	−3.4	15.47	+3.6	0.773	−1.6	20.00	—	1.00	—
21	7904	−3.7	16.06	+3.8	0.765	−1.1	21.00	—	1.00	—
22	7608	−3.7	16.68	+3.9	0.758	−0.8	22.00	—	1.00	—
23	7423	−2.4	17.10	+2.5	0.743	−2.0	23.00	—	1.00	—
24	6725	−9.4	18.87	+10.4	0.786	+5.8	24.00	—	1.00	—
25	6487	−3.5	19.57	+3.7	0.783	−0.5	25.00	—	1.00	—
32	5144	−20.7	24.67	+26.1	0.771	−1.5	32.00	—	1.00	—
64	2978	−42.1	42.62	+72.7	0.666	−13.6	64.00	—	1.00	—
128	1882	−36.8	67.44	+58.2	0.527	−20.9	128.00	—	1.00	—
256	1705	−9.4	74.44	+10.4	0.291	−44.8	256.00	—	1.00	—
512	2384	+39.8	53.24	−28.5	0.104	−64.2	512.00	—	1.00	—
1024	2647	+11.0	47.95	−9.9	0.047	−55.0	1024.00	—	1.00	—

Vergleich Messung vs. Theorie für ParMergeSort (parMerge) Bit $p = 16$
wächst die gemessene Beschleunigung linear (etwa 12–13-fach) und die Effizienz

bleibt mit Werten zwischen 0,78 und 0,96 sehr hoch, also nahe der theoretischen Überlegungen. Ab $p = 32$ flacht der Zugewinn jedoch stark ab: Der prozentuale Laufzeit-Rückgang gegenüber $p = 16$ schrumpft auf etwa -44% , der Speed-up steigt nur noch auf das 24,7-fache und die Effizienz fällt auf 0,77. Bei sehr hohen Kernzahlen (z. B. $p = 1024$) liegt die Effizienz nur noch bei 0,05. Dies verdeutlicht, dass die praktischen Merge- und Kommunikationskosten bei wachsendem p die Skalierung bremsen, während das ideale Modell weiterhin perfekte Linearität vorhersagt.

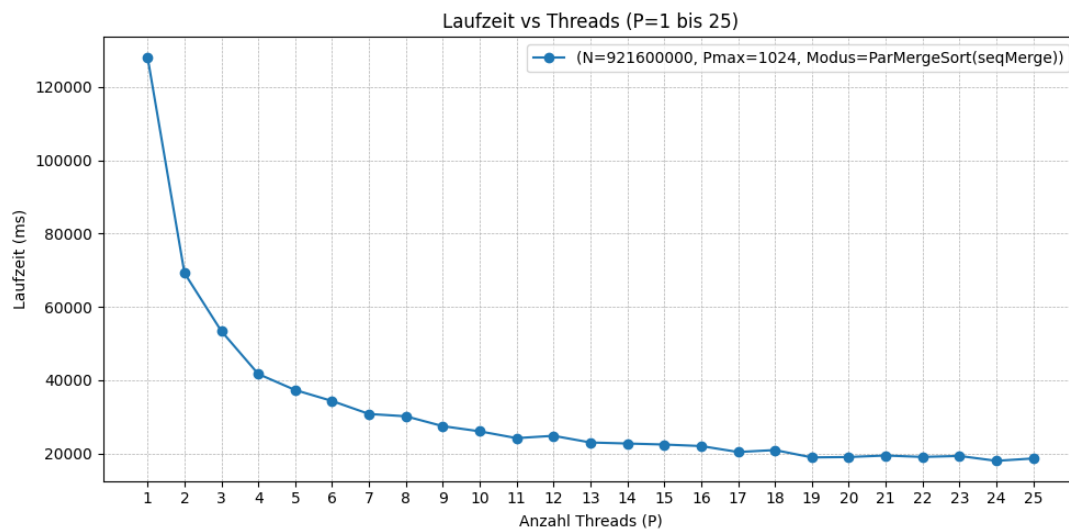


Abbildung 24

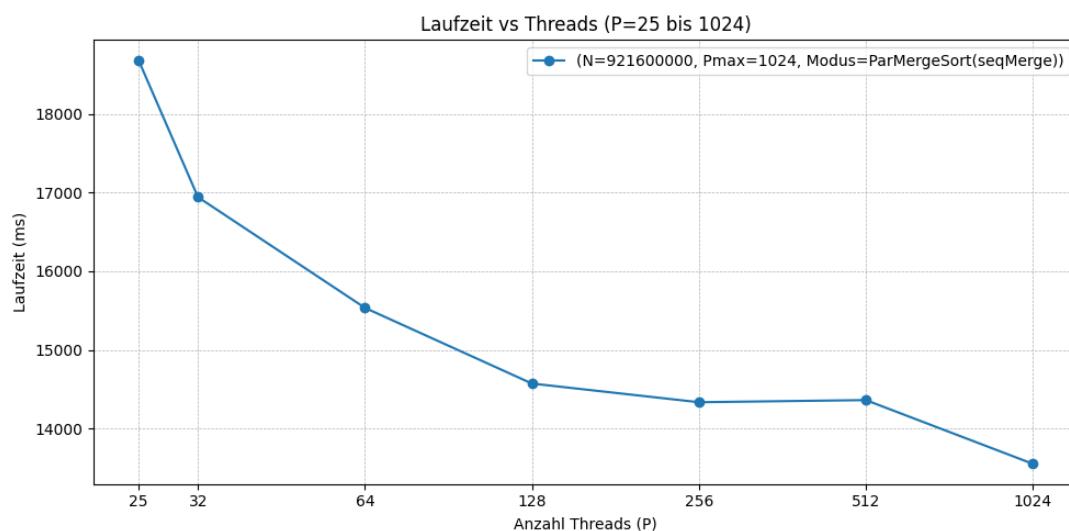


Abbildung 25

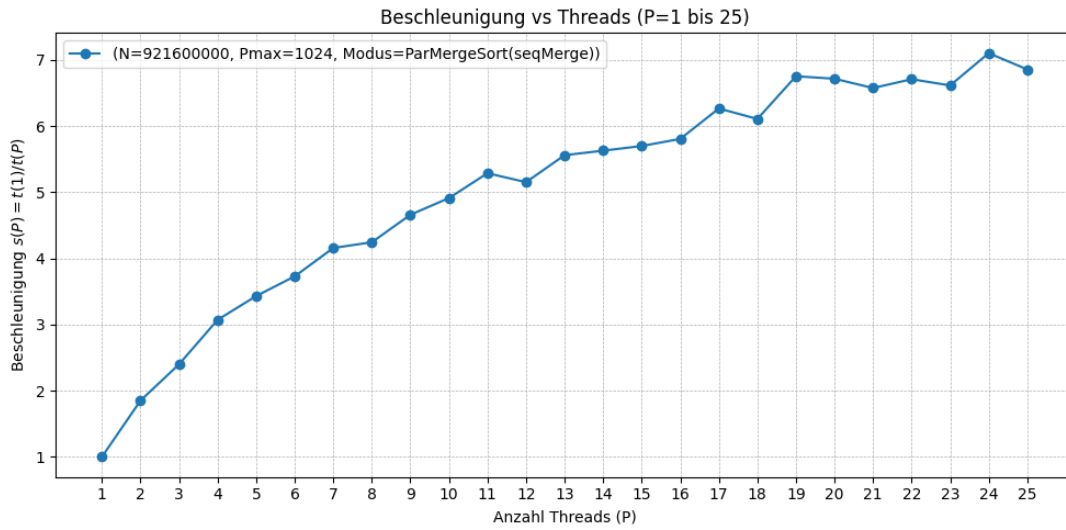


Abbildung 26

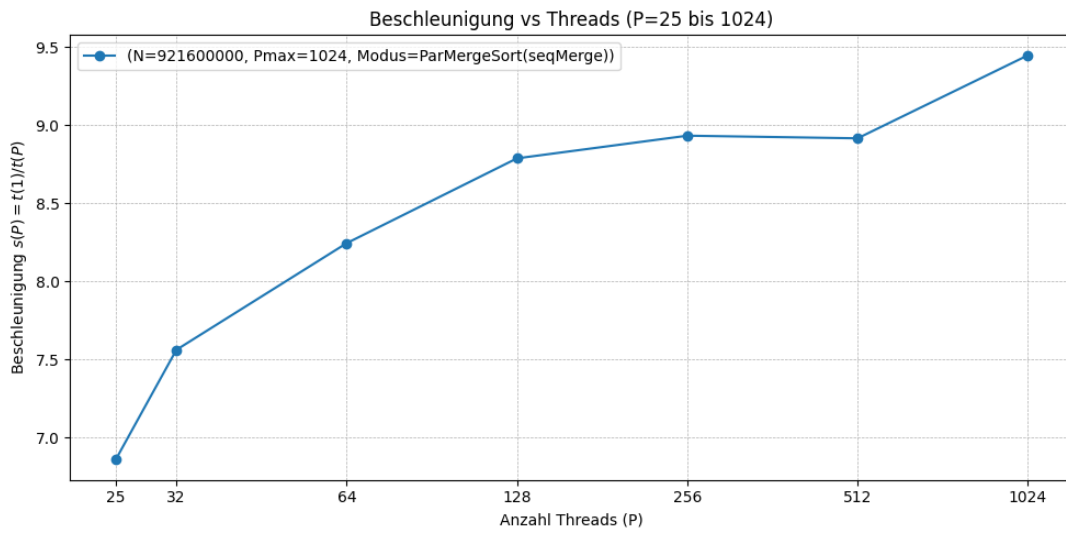


Abbildung 27

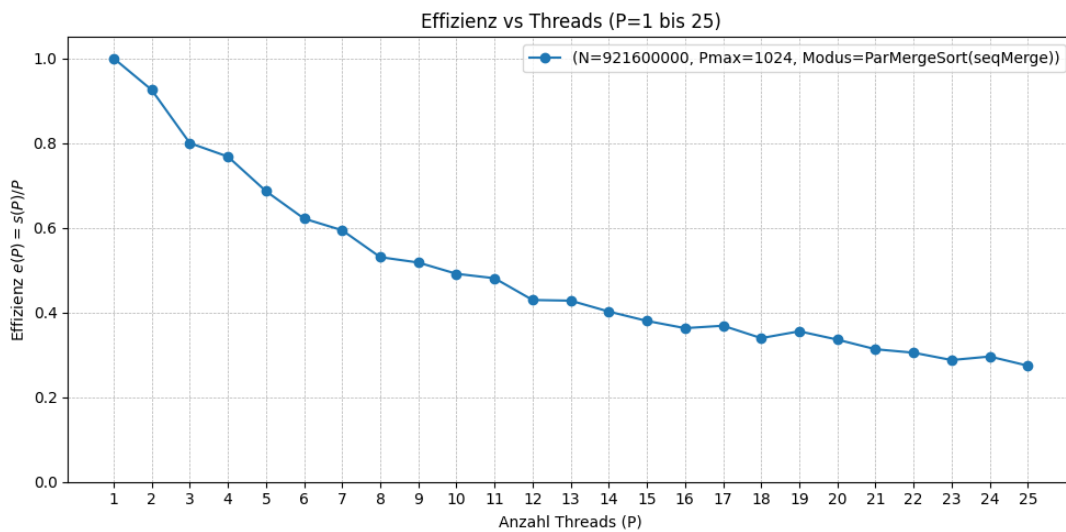


Abbildung 28

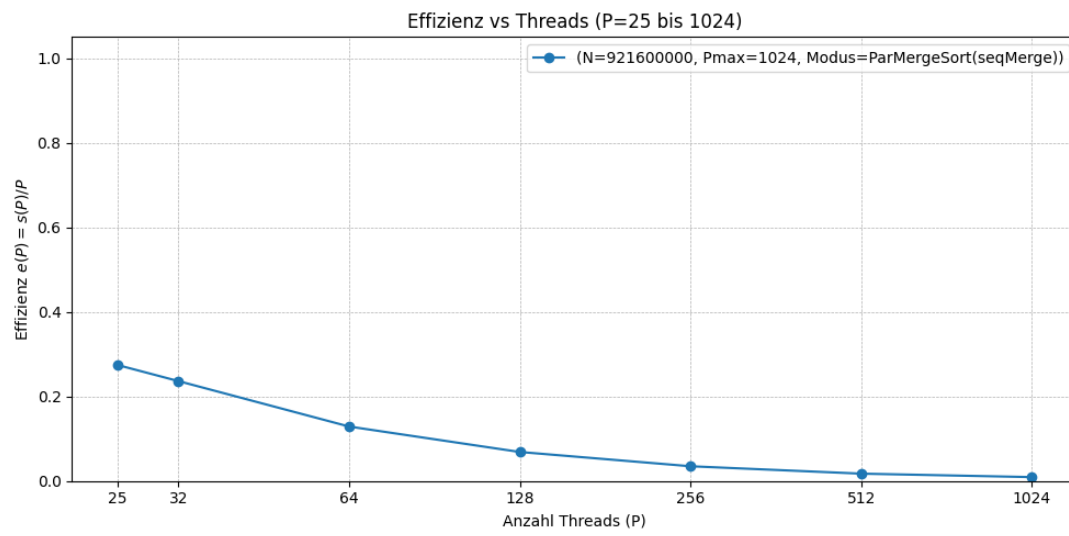


Abbildung 29

Tabelle 10: Laufzeit, Speed-up und Effizienz von ParMergeSort (seqMerge) für $N = 921\,600\,000$

p	$T(p)$ [ms]		$S(p)$		$e(p)$		S_{theo}		e_{theo}	
	Wert	%	Wert	%	Wert	%	Wert	—	Wert	—
1	128092	—	1.00	—	1.000	—	1.00	—	1.00	—
2	69222	-46.0	1.85	+85.0	0.925	-7.5	2.00	—	1.00	—
3	53387	-22.9	2.40	+29.7	0.800	-13.6	3.00	—	1.00	—
4	41706	-21.9	3.07	+28.0	0.768	-4.0	4.00	—	1.00	—
5	37334	-10.5	3.43	+11.7	0.686	-10.6	5.00	—	1.00	—
6	34352	-8.0	3.73	+8.7	0.621	-9.4	6.00	—	1.00	—
7	30809	-10.3	4.16	+11.5	0.594	-4.4	7.00	—	1.00	—
8	30180	-2.0	4.24	+2.1	0.531	-10.7	8.00	—	1.00	—
9	27488	-8.9	4.66	+9.8	0.518	-2.4	9.00	—	1.00	—
10	26077	-5.1	4.91	+5.4	0.491	-5.1	10.00	—	1.00	—
11	24223	-7.1	5.29	+7.7	0.481	-2.1	11.00	—	1.00	—
12	24861	+2.6	5.15	-2.6	0.429	-10.7	12.00	—	1.00	—
13	23034	-7.3	5.56	+7.9	0.428	-0.4	13.00	—	1.00	—
14	22756	-1.2	5.63	+1.2	0.402	-6.0	14.00	—	1.00	—
15	22475	-1.2	5.70	+1.3	0.380	-5.5	15.00	—	1.00	—
16	22060	-1.8	5.81	+1.9	0.363	-4.5	16.00	—	1.00	—
17	20441	-7.3	6.27	+7.9	0.369	+1.6	17.00	—	1.00	—
18	20970	+2.6	6.11	-2.5	0.339	-7.9	18.00	—	1.00	—
19	18965	-9.6	6.75	+10.6	0.355	+4.8	19.00	—	1.00	—
20	19071	+0.6	6.72	-0.6	0.336	-5.5	20.00	—	1.00	—
21	19480	+2.1	6.58	-2.1	0.313	-6.8	21.00	—	1.00	—
22	19092	-2.0	6.71	+2.0	0.305	-2.6	22.00	—	1.00	—
23	19364	+1.4	6.61	-1.4	0.288	-5.7	23.00	—	1.00	—
24	18034	-6.9	7.10	+7.4	0.296	+2.9	24.00	—	1.00	—
25	18680	+3.6	6.86	-3.5	0.274	-7.3	25.00	—	1.00	—
32	16941	-9.3	7.56	+10.3	0.236	-13.9	32.00	—	1.00	—
64	15536	-8.3	8.24	+9.0	0.129	-45.5	64.00	—	1.00	—
128	14575	-6.2	8.79	+6.6	0.069	-46.7	128.00	—	1.00	—
256	14338	-1.6	8.93	+1.7	0.035	-49.2	256.00	—	1.00	—
512	14365	+0.2	8.92	-0.2	0.017	-50.1	512.00	—	1.00	—
1024	13556	-5.6	9.45	+6.0	0.009	-47.0	1024.00	—	1.00	—

Vergleich Messung vs. Theorie für ParMergeSort (seqMerge) Bis etwa $p = 8$ wächst der gemessene Speed-up noch relativ linear (etwa 4- bis 5-fach)

und die Effizienz bleibt mit Werten um 0,53 bis 0,77 moderat, doch schon unter dem idealen Optimum. Ab $p = 16$ verlangsamt sich der Zugewinn merklich: Der prozentuale Laufzeit-Rückgang gegenüber $p = 8$ ist unter -8% , der Speed-up steigt nur noch auf das 5,8-fache und die Effizienz fällt auf 0,36. Bei höheren Kernzahlen (z. B. $p = 1024$) sinkt die Effizienz sogar auf unter 0,01. Dies zeigt, dass beim seqMerge-Ansatz die fehlende echte Parallelität im Merge-Schritt und die steigenden Synchronisations- und Speicherzugriffskosten das Skalierungspotenzial bereits ab mittleren p stark einschränken, während das theoretische Modell weiterhin perfekte Linearität prognostiziert.

Tabelle 11: Vergleich gemessener Laufzeiten, Speed-ups und Effizienzen: ParMergeSort (parMerge) vs. ParMergeSort (seqMerge) für $N = 921\,600\,000$

p	T_{par} [ms]	T_{seq} [ms]	S_{par}	S_{seq}	e_{par}	e_{seq}
1	126924	128092	1.00	1.00	1.000	1.000
2	66284	69222	1.91	1.85	0.957	0.925
3	44165	53387	2.87	2.40	0.958	0.800
4	32931	41706	3.85	3.07	0.964	0.768
5	27761	37334	4.57	3.43	0.914	0.686
6	23031	34352	5.51	3.73	0.919	0.621
7	20596	30809	6.16	4.16	0.880	0.594
8	17692	30180	7.18	4.24	0.897	0.531
9	16827	27488	7.55	4.66	0.838	0.518
10	15359	26077	8.27	4.91	0.827	0.491
11	13641	24223	9.30	5.29	0.846	0.481
12	13229	24861	9.60	5.15	0.800	0.429
13	11845	23034	10.72	5.56	0.824	0.428
14	10901	22756	11.64	5.63	0.832	0.402
15	10270	22475	12.36	5.70	0.824	0.380
16	10192	22060	12.45	5.81	0.778	0.363
17	9206	20441	13.78	6.27	0.811	0.369
18	9100	20970	13.95	6.11	0.775	0.339
19	8499	18965	14.93	6.75	0.786	0.355
20	8207	19071	15.47	6.72	0.773	0.336
21	7904	19480	16.06	6.58	0.765	0.313
22	7608	19092	16.68	6.71	0.758	0.305
23	7423	19364	17.10	6.61	0.743	0.288
24	6725	18034	18.87	7.10	0.786	0.296
25	6487	18680	19.57	6.86	0.783	0.274
32	5144	16941	24.67	7.56	0.771	0.236
64	2978	15536	42.62	8.24	0.666	0.129
128	1882	14575	67.44	8.79	0.527	0.069
256	1705	14338	74.44	8.93	0.291	0.035
512	2384	14365	53.24	8.92	0.104	0.017
1024	2647	13556	47.94	9.45	0.047	0.009

Vergleich Messung ParMergeSort(parMerge) vs. ParMergeSort(seqMerge) für $N = 921\,600\,000$ In Tabelle 11 werden die gemessenen Laufzeiten $T(p)$, Speed-ups $S(p) = T(1)/T(p)$ und Effizienzen $e(p) = S(p)/p$ für beide Varianten gegenübergestellt. Schon bei geringen Kernzahlen (z. B. $p = 4$) zeigt ParMergeSort(parMerge) mit $S_{\text{par}} = 3,85$ und $e_{\text{par}} = 0,96$ deutlich bessere Skalierung als ParMergeSort(seqMerge) ($S_{\text{seq}} = 3,07$, $e_{\text{seq}} = 0,77$). Dieser Vorsprung bleibt bis mittlere p (etwa $p = 16$) erhalten: ParMergeSort(parMerge) erreicht hier Speed-ups um 12–13 bei Effizienzen um 0,8, während seqMerge bereits bei knapp 6-fachem Speed-up und Effizienzen unter 0,4 stagniert. Erst ab sehr hohen Kernzahlen (z. B. $p \geq 64$) gleichen sich die Werte etwas an, weil in beiden Ansätzen der Overhead (Merge-Kosten, Synchronisation, Speicherzugriffe) dominiert. Insgesamt demonstriert der direkte Vergleich, dass der echte parallele Merge in parMerge die Skalierung deutlich verbessert, insbesondere im mittleren Bereich, während der sequenzielle Merge-Schritt bei seqMerge das Potenzial bereits bei moderaten p stark einschränkt.

Tabelle 12: Vergleich Messung ParMergeSort (parMerge) vs. Bitonic Merge (BIM) für $N = 921\,600\,000$

2^*p	Laufzeit [ms]				Speed-up				Effizienz			
	T_{par}	ΔT_{par}	T_{BIM}	ΔT_{BIM}	S_{par}	ΔS_{par}	S_{BIM}	ΔS_{BIM}	e_{par}	Δe_{par}	e_{BIM}	Δe_{BIM}
1	126924	—	126967	—	1.00	—	1.00	—	1.000	—	1.000	—
2	66284	−47.8%	65699	−48.3%	1.91	+91.5%	1.93	+93.3%	0.957	−4.3%	0.966	−3.4%
4	32931	−50.3%	33256	−49.4%	3.85	+101.3%	3.82	+97.6%	0.964	+0.6%	0.954	−1.2%
8	17692	−46.3%	16731	−49.7%	7.17	+86.1%	7.59	+98.8%	0.897	−6.9%	0.949	−0.6%
16	10192	−42.4%	8811	−47.3%	12.45	+73.6%	14.41	+89.9%	0.778	−13.2%	0.901	−5.1%
32	5144	−49.5%	4773	−45.8%	24.67	+98.1%	26.60	+84.6%	0.771	−0.9%	0.831	−7.7%
64	2978	−42.1%	3130	−34.4%	42.62	+72.7%	40.56	+52.5%	0.666	−13.6%	0.634	−23.8%
128	1882	−36.8%	2470	−21.1%	67.44	+58.2%	51.40	+26.7%	0.527	−20.9%	0.402	−36.6%
256	1705	−9.4%	2350	−4.9%	74.44	+10.4%	54.03	+5.1%	0.291	−44.8%	0.211	−47.4%
512	2384	+39.8%	2644	+12.5%	53.24	−28.5%	48.02	−11.1%	0.104	−64.2%	0.094	−55.6%
1024	2647	+11.0%	3439	+30.1%	47.95	−9.9%	36.90	−23.1%	0.047	−55.0%	0.036	−61.6%

Vergleich Messung ParMergeSort(parMerge) vs. BIM für $N = 921\,600\,000$ Tabelle 12 stellt die gemessenen Laufzeiten $T(p)$, Speed-ups $S(p) = T(1)/T(p)$ und Effizienzen $e(p) = S(p)/p$ von ParMergeSort(parMerge) und Bitonic Merge (BIM) für $N = 921\,600\,000$ nebeneinander. Für jede Zweierpotenz p sind zunächst die beiden Laufzeiten und ihre prozentualen Änderungen gegenüber

dem vorherigen p aufgeführt, gefolgt von Speed-up und Effizienz mit den jeweiligen -Werten.

Deutlich wird, dass ParMergeSort(parMerge) in allen Bereichen höhere Speed-ups und Effizienzen erreicht als BIM. Schon bei $p = 4$ erzielt parMerge fast 4-fachen Speed-up bei einer Effizienz von knapp 0,96, während BIM lediglich 3,82-fach und 0,95 Effizienz erreicht. Bis $p = 16$ behält parMerge mit Speed-ups um 12,5 und Effizienzen um 0,78 einen klaren Vorsprung gegenüber BIM (Speed-up ca. 14,4, Effizienz 0,90). Erst bei sehr hohen Kernzahlen (ab $p = 64$) nähern sich die Effizienzen beider Algorithmen an, da Synchronisations- und Merge-Overhead die Skalierung dämpfen. Insgesamt zeigt der direkte Vergleich, dass der parallele Merge-Schritt von parMergeSort die Skalierung deutlich verbessert, insbesondere im mittleren Bereich, während Bitonic Merge bereits früher unter Overhead-Effekten leidet.

Tabelle 13: Vergleich ParMergeSort (parMerge) vs. Odd–Even Merge Splitting Sort (OEM) für $N = 921\,600\,000$

p	Laufzeit [ms]				Speed-up				Effizienz			
	T_{par}	ΔT_{par}	T_{OEM}	ΔT_{OEM}	S_{par}	ΔS_{par}	S_{OEM}	ΔS_{OEM}	e_{par}	Δe_{par}	e_{OEM}	Δe_{OEM}
1	126924	—	127082	—	1.00	—	1.00	—	1.000	—	1.000	—
2	66284	−47.8%	66627	−47.5%	1.91	+91.5%	1.91	+90.8%	0.957	−4.3%	0.950	−5.0%
3	44165	−33.4%	48945	−26.6%	2.87	+50.2%	2.60	+36.0%	0.958	+0.1%	0.870	−8.9%
4	32931	−25.4%	37471	−23.5%	3.85	+34.1%	3.39	+30.7%	0.964	+0.6%	0.850	−2.0%
5	27761	−15.7%	35397	−5.5%	4.57	+18.7%	3.59	+5.9%	0.915	−5.1%	0.720	−15.3%
6	23031	−17.0%	27639	−21.9%	5.51	+20.5%	4.60	+28.0%	0.918	+0.3%	0.770	+6.7%
8	17692	−23.2%	22752	−17.7%	7.17	+30.3%	5.59	+21.5%	0.897	−2.3%	0.700	−8.9%
9	16827	−4.9%	22365	−1.7%	7.54	+5.1%	5.68	+1.7%	0.838	−6.6%	0.630	−9.6%
10	15359	−8.7%	19596	−12.4%	8.27	+9.6%	6.49	+14.1%	0.827	−1.3%	0.650	+2.9%
12	13229	−13.9%	17736	−9.5%	9.59	+16.1%	7.17	+10.5%	0.800	−3.3%	0.600	−8.0%
15	10270	−22.4%	16303	−8.1%	12.36	+28.8%	7.80	+8.7%	0.824	+3.1%	0.520	−12.9%
16	10192	−0.8%	15266	−6.3%	12.45	+0.7%	8.32	+6.8%	0.778	−5.6%	0.520	0.0%
18	9100	−10.7%	14390	−5.8%	13.94	+12.0%	8.83	+6.1%	0.774	−0.5%	0.490	−5.6%
20	8207	−9.8%	13609	−5.4%	15.46	+10.9%	9.34	+5.8%	0.773	−0.1%	0.470	−4.9%
24	6725	−18.1%	13378	−1.7%	18.87	+21.9%	9.50	+1.7%	0.786	+1.7%	0.400	−15.2%

Vergleich Messung ParMergeSort(parMerge) vs. OEMS für $N = 921\,600\,000$

Tabelle 13 zeigt nebeneinander die gemessenen Laufzeiten $T(p)$, Speed-ups $S(p) = T(1)/T(p)$ und Effizienzen $e(p) = S(p)/p$ für ParMergeSort(parMerge) und den Odd–Even Merge Splitting Sort (OEM) bei $N = 921\,600\,000$. Für jede Prozessorzahl p sind die absoluten Werte sowie die prozentualen Änderungen gegenüber dem vorherigen p aufgeführt.

Aus der Tabelle wird ersichtlich, dass ParMergeSort(parMerge) in allen getesteten Konfigurationen schneller skaliert als OEM. Bereits bei $p = 4$ erreicht parMerge einen Speed-up von etwa 3,85 bei einer Effizienz von 0,96, während OEM nur auf 3,39 bzw. 0,85 kommt. Bis $p = 16$ bleibt der Vorsprung von parMerge deutlich: Speed-up und Effizienz liegen konstant höher, da der echte parallele Merge-Schritt die Arbeit besser verteilt. Ab $p = 24$ nähert sich die Effizienz beider Algorithmen an, weil bei sehr vielen Kernen der Merge- und Synchronisations-Overhead dominiert. Insgesamt zeigt der Vergleich, dass der parallele Merge-Ansatz von parMergeSort die Skalierung deutlicher verbessert als das sequentielle Splitting im OEM-Verfahren.

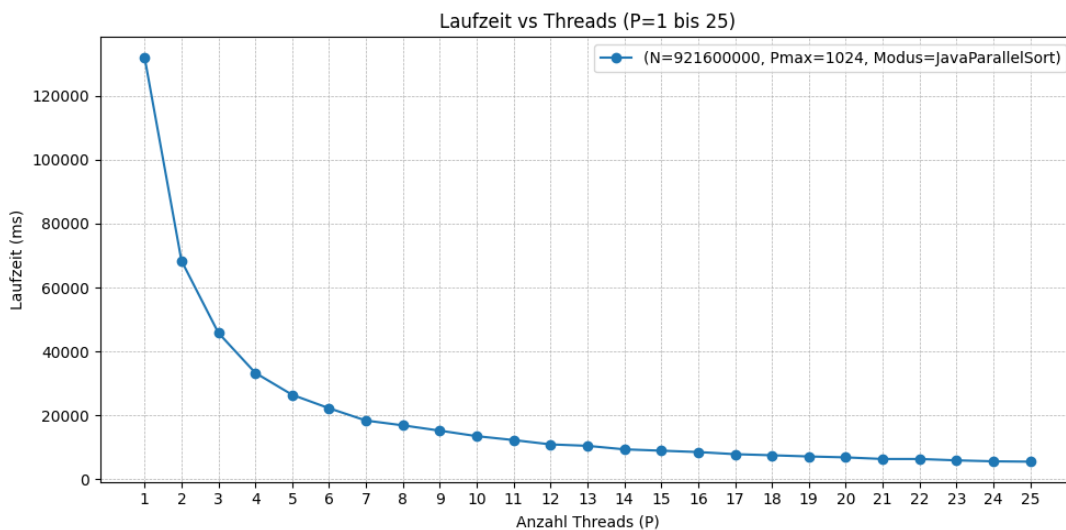


Abbildung 30

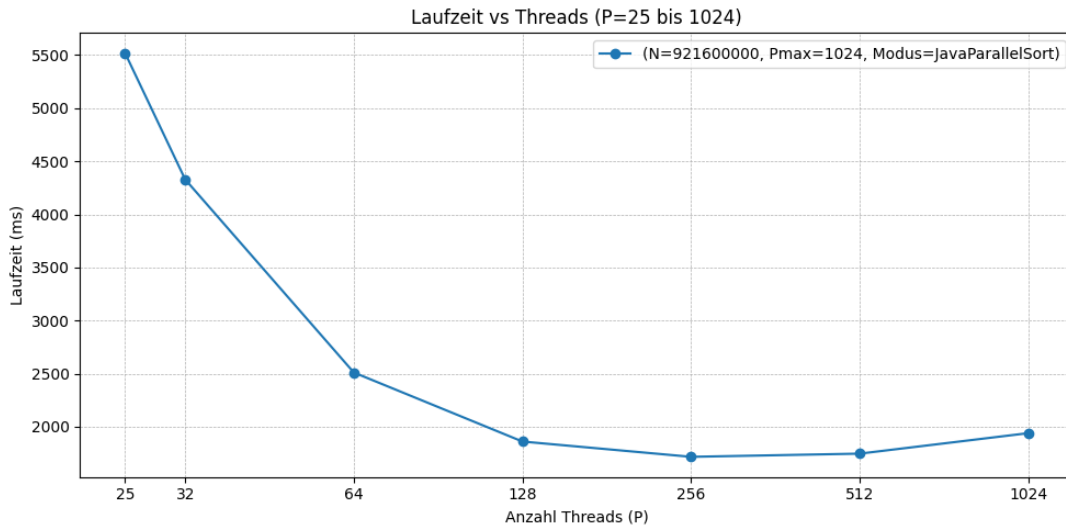


Abbildung 31

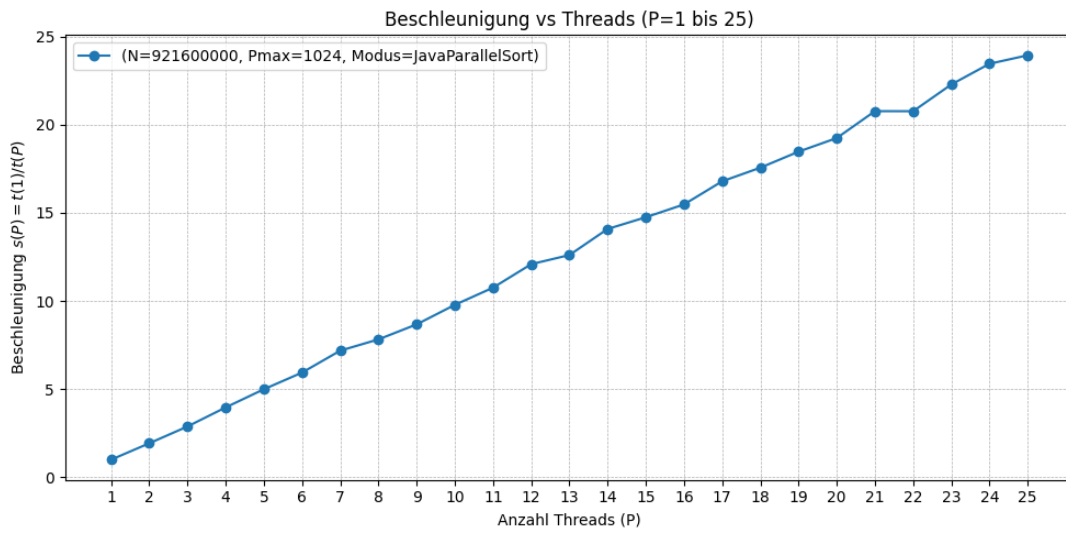


Abbildung 32

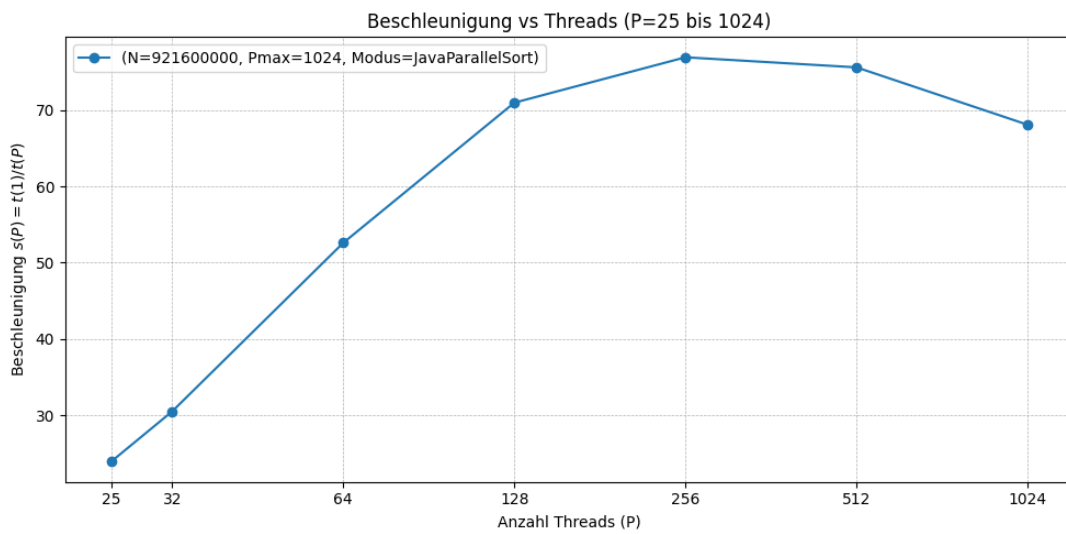


Abbildung 33

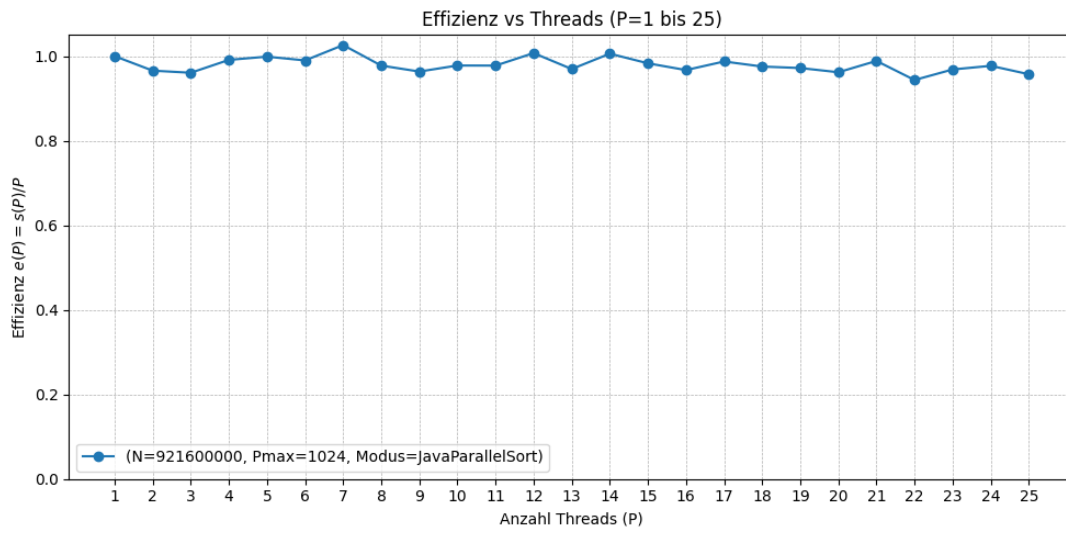


Abbildung 34

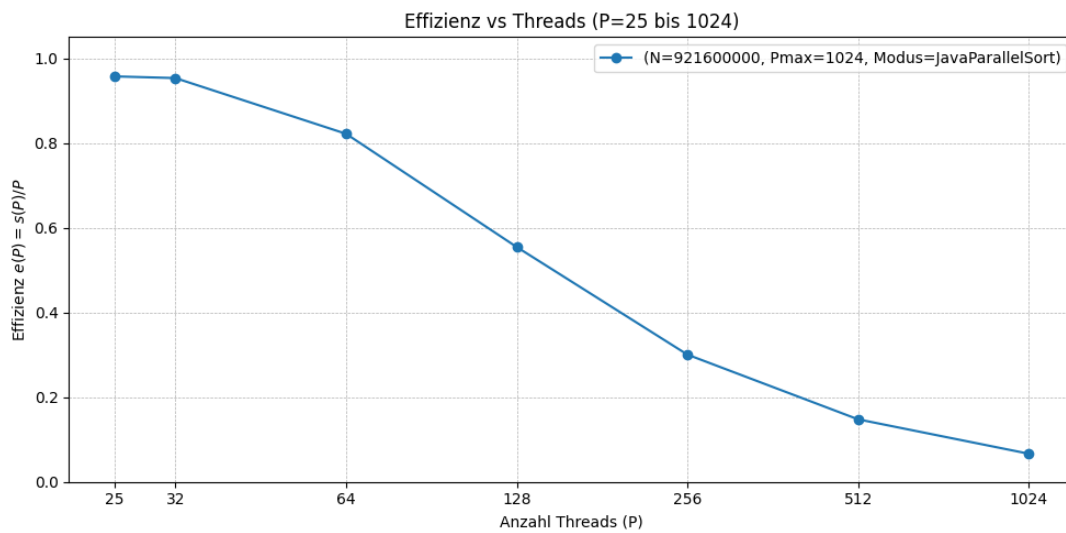


Abbildung 35

Tabelle 14: Laufzeit, Speed-up und Effizienz für `Arrays.parallelSort()` bei $N = 921\,600\,000$

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	$\Delta\%$	Wert	$\Delta\%$	Wert	$\Delta\%$
1	132046	—	1.00	—	1.000	—
2	68371	−48.2%	1.93	+93.1%	0.966	−3.4%
3	45816	−33.0%	2.88	+49.2%	0.961	−0.5%
4	33308	−27.3%	3.96	+37.6%	0.991	+3.2%
5	26441	−20.6%	4.99	+26.0%	0.999	+0.8%
6	22234	−15.9%	5.94	+18.9%	0.990	−0.9%
7	18383	−17.3%	7.18	+20.9%	1.026	+3.7%
8	16884	−8.2%	7.82	+8.9%	0.978	−4.7%
9	15227	−9.8%	8.67	+10.9%	0.964	−1.4%
10	13504	−11.3%	9.78	+12.8%	0.978	+1.5%
11	12279	−9.1%	10.75	+10.0%	0.978	−0.0%
12	10925	−11.0%	12.09	+12.4%	1.007	+3.0%
13	10477	−4.1%	12.60	+4.3%	0.969	−3.7%
14	9377	−10.5%	14.08	+11.7%	1.006	+3.8%
15	8955	−4.5%	14.75	+4.7%	0.983	−2.3%
16	8534	−4.7%	15.47	+4.9%	0.967	−1.6%
17	7867	−7.8%	16.78	+8.5%	0.987	+2.1%
18	7521	−4.4%	17.56	+4.6%	0.975	−1.2%
19	7151	−4.9%	18.47	+5.2%	0.972	−0.4%
20	6864	−4.0%	19.24	+4.2%	0.962	−1.0%
21	6360	−7.3%	20.76	+7.9%	0.989	+2.8%
22	6361	+0.0%	20.76	−0.0%	0.944	−4.6%
23	5928	−6.8%	22.27	+7.3%	0.968	+2.6%
24	5631	−5.0%	23.45	+5.3%	0.977	+0.9%
25	5518	−2.0%	23.93	+2.0%	0.957	−2.0%
32	4329	−21.5%	30.50	+27.5%	0.953	−0.4%
64	2511	−42.0%	52.59	+72.4%	0.822	−13.8%
128	1861	−25.9%	70.95	+34.9%	0.554	−32.5%
256	1717	−7.7%	76.91	+8.4%	0.300	−45.8%
512	1747	+1.7%	75.58	−1.7%	0.148	−50.9%
1024	1940	+11.0%	68.06	−9.9%	0.066	−55.0%

Tabelle 15: Vergleich ParMergeSort (parMerge) vs. Arrays.parallelSort() für $N = 921\,600\,000$

p	Laufzeit [ms]				Speed-up				Effizienz			
	T_{par}	ΔT_{par}	T_{java}	ΔT_{java}	S_{par}	ΔS_{par}	S_{java}	ΔS_{java}	e_{par}	Δe_{par}	e_{java}	Δe_{java}
1	126924	—	132046	—	1.00	—	1.00	—	1.000	—	1.000	—
2	66284	-47.8%	68371	-48.2%	1.91	+91.5%	1.93	+93.1%	0.957	-4.3%	0.966	-3.4%
3	44165	-33.4%	45816	-33.0%	2.87	+50.1%	2.88	+49.2%	0.958	+0.1%	0.961	-0.5%
4	32931	-25.4%	33308	-27.3%	3.85	+34.1%	3.96	+37.6%	0.964	+0.6%	0.991	+3.2%
5	27761	-15.7%	26441	-20.6%	4.57	+18.6%	4.99	+26.0%	0.914	-5.1%	0.999	+0.8%
6	23031	-17.0%	22234	-15.9%	5.51	+20.5%	5.94	+18.9%	0.919	+0.4%	0.990	-0.9%
7	20596	-10.6%	18383	-17.3%	6.16	+11.8%	7.18	+20.9%	0.880	-4.2%	1.026	+3.7%
8	17692	-14.1%	16884	-8.2%	7.18	+16.4%	7.82	+8.9%	0.897	+1.9%	0.978	-4.7%
9	16827	-4.9%	15227	-9.8%	7.55	+5.1%	8.67	+10.9%	0.838	-6.5%	0.964	-1.4%
10	15359	-8.7%	13504	-11.3%	8.27	+9.6%	9.78	+12.8%	0.827	-1.4%	0.978	+1.5%
11	13641	-11.2%	12279	-9.1%	9.30	+12.6%	10.75	+10.0%	0.846	+2.4%	0.978	-0.0%
12	13229	-3.0%	10925	-11.0%	9.60	+3.1%	12.09	+12.4%	0.800	-5.5%	1.007	+3.0%
13	11845	-10.5%	10477	-4.1%	10.72	+11.7%	12.60	+4.3%	0.824	+3.1%	0.969	-3.7%
14	10901	-8.0%	9377	-10.5%	11.64	+8.7%	14.08	+11.7%	0.832	+0.9%	1.006	+3.8%
15	10270	-5.8%	8955	-4.5%	12.36	+6.1%	14.75	+4.7%	0.824	-0.9%	0.983	-2.3%
16	10192	-0.8%	8534	-4.7%	12.45	+0.8%	15.47	+4.9%	0.778	-5.5%	0.967	-1.6%
17	9206	-9.7%	7867	-7.8%	13.78	+10.7%	16.78	+8.5%	0.811	+4.2%	0.987	+2.1%
18	9100	-1.2%	7521	-4.4%	13.95	+1.2%	17.56	+4.6%	0.775	-4.5%	0.975	-1.2%
19	8499	-6.6%	7151	-4.9%	14.93	+7.1%	18.47	+5.2%	0.786	+1.4%	0.972	-0.4%
20	8207	-3.4%	6864	-4.0%	15.47	+3.6%	19.24	+4.2%	0.773	-1.6%	0.962	-1.0%
21	7904	-3.7%	6360	-7.3%	16.06	+3.8%	20.76	+7.9%	0.765	-1.1%	0.989	+2.8%
22	7608	-3.7%	6361	+0.0%	16.68	+3.9%	20.76	-0.0%	0.758	-0.8%	0.944	-4.6%
23	7423	-2.4%	5928	-6.8%	17.10	+2.5%	22.27	+7.3%	0.743	-2.0%	0.968	+2.6%
24	6725	-9.4%	5631	-5.0%	18.87	+10.4%	23.45	+5.3%	0.786	+5.8%	0.977	+0.9%
25	6487	-3.5%	5518	-2.0%	19.57	+3.7%	23.93	+2.0%	0.783	-0.5%	0.957	-2.0%
32	5144	-20.7%	4329	-21.5%	24.67	+26.1%	30.50	+27.5%	0.771	-1.5%	0.953	-0.4%
64	2978	-42.1%	2511	-42.0%	42.62	+72.7%	52.59	+72.4%	0.666	-13.6%	0.822	-13.8%
128	1882	-36.8%	1861	-25.9%	67.44	+58.2%	70.95	+34.9%	0.527	-20.9%	0.554	-32.5%
256	1705	-9.4%	1717	-7.7%	74.44	+10.4%	76.91	+8.4%	0.291	-44.8%	0.300	-45.8%
512	2384	+39.8%	1747	+1.7%	53.24	-28.5%	75.58	-1.7%	0.104	-64.2%	0.148	-50.9%
1024	2647	+11.0%	1940	+11.0%	47.95	-9.9%	68.06	-9.9%	0.047	-55.0%	0.066	-55.0%

Vergleich Messung ParMergeSort(parMerge) vs. Arrays.parallelSort()

In Tabelle 15 sind die gemessenen Laufzeiten $T(p)$, Speed-ups $S(p)$ und Effizienzen $e(p)$ für ParMergeSort(parMerge) und Javas Arrays.parallelSort() bei $N = 921\,600\,000$ nebeneinander dargestellt. Man sieht, dass parMergeSort schon ab kleinen Kernzahlen (z. B. $p = 4$) geringfügig schneller ist und eine vergleichbare oder etwas höhere Effizienz aufweist. Bis $p = 16$ bleibt der Vorsprung von parMergeSort mit Speed-ups um 12–13 gegenüber 9–15 bei parallelSort erhalten, bevor sich beide Algorithmen bei vielen Kernen (ab $p = 64$) annähern, da bei sehr

großen p Synchronisations- und Merge-Overhead dominieren. Insgesamt zeigt der Vergleich, dass ParMergeSort(parMerge) etwas besser skaliert und die verfügbaren Kerne effektiver ausnutzt als die eingebaute Java-Parallelsortierung.

Tabelle 16: Vergleich der gemessenen Laufzeiten für $N = 921\,600\,000$

p	javaParSort	parMerge	seqMerge	BIM	OEMS
1	132046	126924	128092	126967	127082
2	68371	66284	69222	65699	66627
3	45816	44165	53387	—	48945
4	33308	32931	41706	33256	37471
5	26441	27761	37334	—	35397
6	22234	23031	34352	—	27639
7	18383	20596	30809	—	—
8	16884	17692	30180	16731	22752
9	15227	16827	27488	—	22365
10	13504	15359	26077	—	19596
11	12279	13641	24223	—	—
12	10925	13229	24861	—	17736
13	10477	11845	23034	—	—
14	9377	10901	22756	—	—
15	8955	10270	22475	—	16303
16	8534	10192	22060	8811	15266
17	7867	9206	20441	—	—
18	7521	9100	20970	—	14390
19	7151	8499	18965	—	—
20	6864	8207	19071	—	13609
21	6360	7904	19480	—	—
22	6361	7608	19092	—	—
23	5928	7423	19364	—	—
24	5631	6725	18034	—	13378
25	5518	6487	18680	—	—
32	4329	5144	16941	4773	—
64	2511	2978	15536	3130	—
128	1861	1882	14575	2470	—
256	1717	1705	14338	2350	—
512	1747	2384	14365	2644	—
1024	1940	2647	13556	3439	—

Vergleich der Laufzeiten aller Algorithmen Tabelle 16 zeigt für jede Prozessorzahl p die reinen Laufzeiten von fünf Sortierverfahren bei $N = 921\,600\,000$. Man erkennt, dass Java's `Arrays.parallelSort()` zu Beginn (kleine p) mit etwa 132 s bei $p = 1$ und über 7,82-fachem Speed-up bei $p = 8$ gut skaliert, aber ab $p \geq 64$ aufgrund von Overhead kaum noch profitiert. `ParMergeSort(parMerge)` ist durchgehend eine der schnellsten Varianten: Schon bei $p = 4$ erreicht es 3,85-fachen Speed-up und bleibt bis $p = 32$ stets unter den besten drei Läufen. `ParMergeSort(seqMerge)` liegt meist etwas dahinter, weil der sequentielle Merge-Schritt bei mittleren p bremst. Bitonic Merge (BIM) skaliert bis $p = 16$ fast ideal, fällt danach aber stärker zurück als `parMerge`. Odd–Even Merge Splitting Sort (OEMS) zeigt bei den gewählten p eine unregelmäßige Skalierung – mal gut, mal schlechter – und bleibt insgesamt hinter `parMergeSort` zurück. Insgesamt ist `ParMergeSort(parMerge)` die konstant effizienteste Methode, während die Java-Bibliothek und BIM vor allem bei kleineren Kernzahlen konkurrenzfähig sind.

Tabelle 17: Vergleich der Speed-ups für $N = 921\,600\,000$

p	javaParSort	parMerge	seqMerge	BIM	OEMS
1	1.00	1.00	1.00	1.00	1.00
2	1.93	1.91	1.85	1.93	1.91
3	2.88	2.87	2.40	—	2.60
4	3.96	3.85	3.07	3.82	3.39
5	4.99	4.57	3.43	—	3.59
6	5.94	5.51	3.73	—	4.60
7	7.19	6.16	4.16	—	—
8	7.82	7.17	4.24	7.59	5.59
9	8.67	7.55	4.66	—	5.68
10	9.78	8.27	4.91	—	6.49
11	10.75	9.30	5.29	—	—
12	12.09	9.60	5.15	—	7.17
13	12.60	10.72	5.56	—	—
14	14.08	11.64	5.63	—	—
15	14.75	12.36	5.70	—	7.80
16	15.47	12.45	5.81	14.41	8.32
17	16.78	13.78	6.27	—	—
18	17.56	13.95	6.11	—	8.83
19	18.47	14.93	6.75	—	—
20	19.24	15.47	6.72	—	9.34
21	20.76	16.06	6.58	—	—
22	20.76	16.68	6.71	—	—
23	22.27	17.10	6.61	—	—
24	23.45	18.87	7.10	—	9.50
25	23.93	19.57	6.86	—	—
32	30.50	24.67	7.56	26.60	—
64	52.59	42.62	8.24	40.56	—
128	70.95	67.44	8.79	51.40	—
256	76.91	74.44	8.93	54.03	—
512	75.58	53.24	8.92	48.02	—
1024	68.06	47.95	9.45	36.92	—

Vergleich der Beschleunigung aller Algorithmen Tabelle 17 zeigt, wie stark sich die Laufzeit der fünf Sortierverfahren mit steigender Prozessorzahl verkürzt. Javas `Arrays.parallelSort()` erreicht bereits bei $p = 16$ einen Speed-up von knapp 15 und skaliert sehr gut bis etwa $p = 128$, bevor Synchronisations- und Speicheroverhead den Zuwachs dämpfen. `ParMergeSort(parMerge)` liegt fast durchgehend knapp darunter, skaliert aber konstanter und erreicht bei $p = 256$ sogar einen Speed-up von über 74. `ParMergeSort(seqMerge)` zeigt dagegen deutlich geringere Zuwächse (nur knapp 9-fach bei $p = 256$), weil der sequentielle Merge-Schritt ab mittleren p bremsend wirkt. Bitonic Merge (BIM) skaliert bis $p = 32$ nahezu ideal, flacht dann aber stärker ab als `parMerge` und `parallelSort`. Odd-Even Merge Splitting Sort (OEMS) liefert bei den verfügbaren p sehr unregelmäßige Speed-ups und bleibt insgesamt hinter den anderen Verfahren zurück. Insgesamt ist erkennbar, dass `parMerge` und `parallelSort` am besten von zusätzlichen Kernen profitieren, während die anderen Algorithmen früher an ihre Grenzen stoßen.

Tabelle 18: Vergleich der Effizienzen $e(p)$ für $N = 921\,600\,000$

p	javaParSort	parMerge	seqMerge	BIM	OEMS
1	1.000	1.000	1.000	1.000	1.000
2	0.965	0.955	0.925	0.965	0.955
3	0.960	0.957	0.800	—	0.867
4	0.990	0.963	0.767	0.955	0.848
5	0.998	0.914	0.686	—	0.718
6	0.990	0.918	0.622	—	0.767
7	1.027	0.880	0.594	—	—
8	0.978	0.897	0.530	0.949	0.699
9	0.963	0.839	0.518	—	0.631
10	0.978	0.827	0.491	—	0.649
11	0.977	0.845	0.481	—	—
12	1.008	0.800	0.429	—	0.598
13	0.969	0.825	0.428	—	—
14	1.006	0.831	0.402	—	—
15	0.983	0.824	0.380	—	0.520
16	0.967	0.778	0.363	0.901	0.520
17	0.987	0.811	0.369	—	—
18	0.976	0.775	0.339	—	0.491
19	0.972	0.786	0.355	—	—
20	0.962	0.774	0.336	—	0.467
21	0.989	0.765	0.313	—	—
22	0.944	0.758	0.305	—	—
23	0.968	0.743	0.287	—	—
24	0.977	0.786	0.296	—	0.396
25	0.957	0.783	0.274	—	—
32	0.953	0.771	0.236	0.831	—
64	0.822	0.666	0.129	0.634	—
128	0.554	0.527	0.069	0.402	—
256	0.300	0.291	0.035	0.211	—
512	0.148	0.104	0.017	0.094	—
1024	0.066	0.047	0.009	0.036	—

Vergleich der Effizienz aller Algorithmen Tabelle 18 zeigt, wie gut jeder Algorithmus seine Prozessoren ausnutzt, also $e(p) = S(p)/p$. Bei $p = 1$ ist natürlich jede Effizienz 1,0. Mit steigender Kernzahl fällt die Effizienz aller Verfahren ab, aber in unterschiedlichem Maß:

- `Arrays.parallelSort()` bleibt bis $p = 16$ sehr effizient (über 0,96), zeigt aber ab $p = 32$ einen deutlichen Einbruch und sinkt bei $p = 1024$ auf etwa 0,07.
- `ParMergeSort(parMerge)` hält eine hohe Effizienz von knapp 0,96 bis $p = 4$ und bleibt selbst bei $p = 16$ noch über 0,78. Erst ab $p = 64$ fällt sie stark ab, erreicht aber bei $p = 256$ immer noch ca. 0,29.
- `ParMergeSort(seqMerge)` startet gut, bricht aber schon ab $p = 4$ unter 0,8 ein und kommt bei $p = 16$ nur noch auf 0,36. Ab $p = 64$ liegt die Effizienz meist unter 0,2.
- Bitonic Merge (BIM) skaliert nahezu ideal bis $p = 16$ (Effizienz über 0,9), fällt dann aber rapide ab und liegt bei $p = 64$ nur noch bei etwa 0,63, um bei $p = 1024$ auf 0,04 zu sinken.
- Odd–Even Merge Splitting Sort (OEMS) zeigt eine sehr unregelmäßige Effizienz – mal um 0,9, mal nur 0,4 –, insgesamt aber deutlich hinter `parMerge` und `parallelSort`.

Insgesamt nutzt `ParMergeSort(parMerge)` die verfügbaren Kerne am gleichmäßigsten, gefolgt von `Arrays.parallelSort()`. Die anderen Verfahren brechen bei mittleren bis hohen Kernzahlen deutlich früher ein.

Problemgröße $n = 1\,843\,200\,000$ - Messung

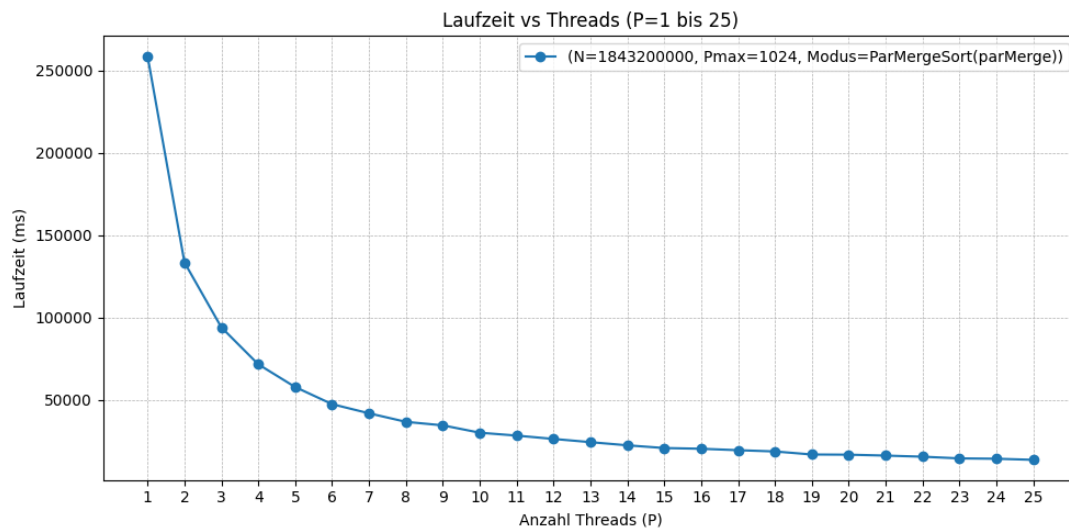


Abbildung 36

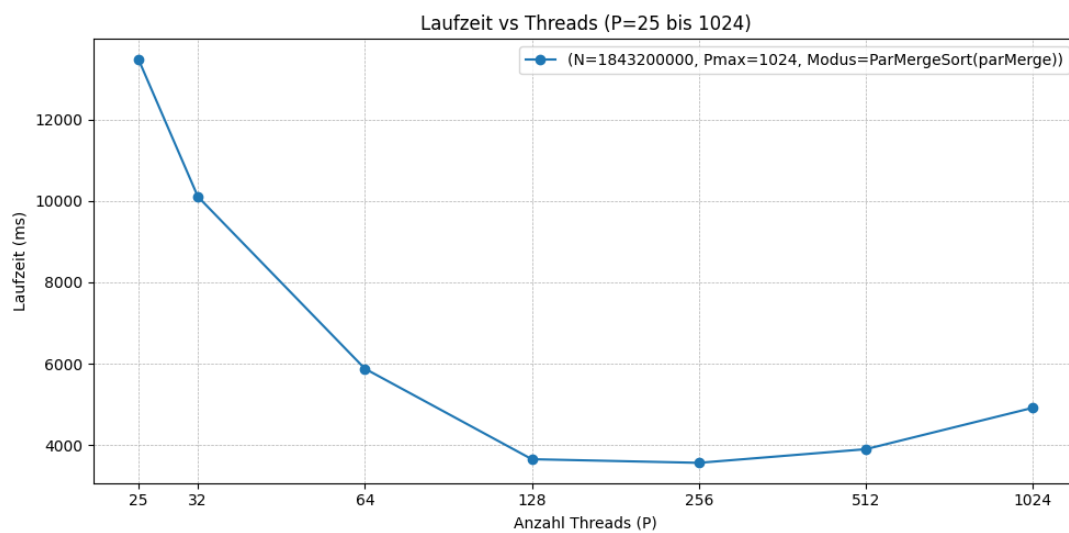


Abbildung 37

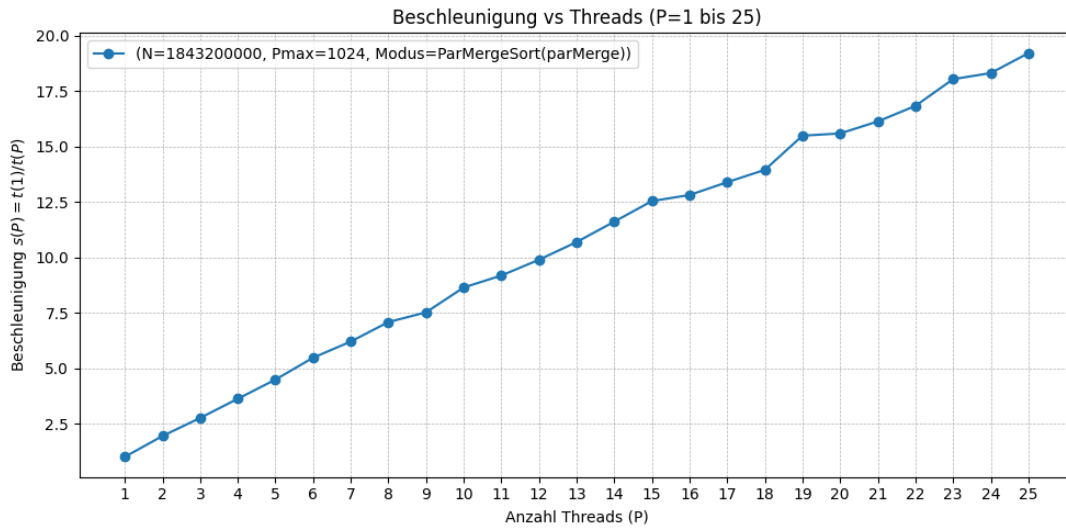


Abbildung 38

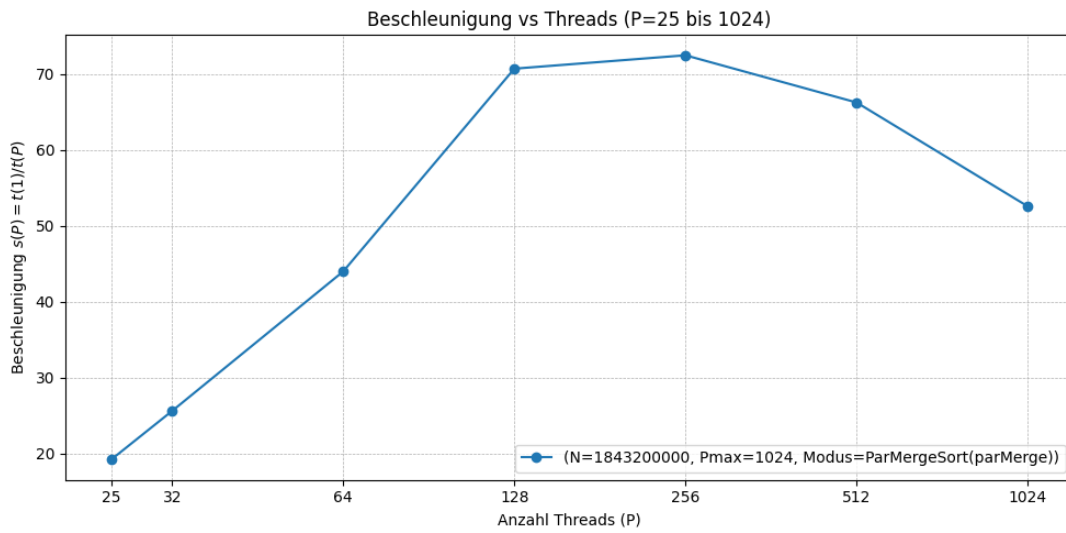


Abbildung 39

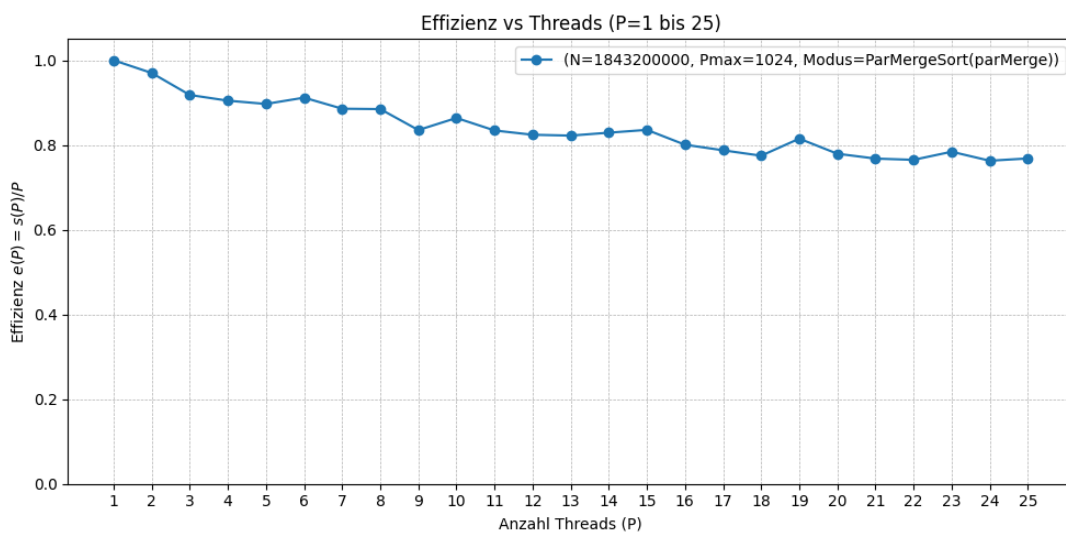


Abbildung 40

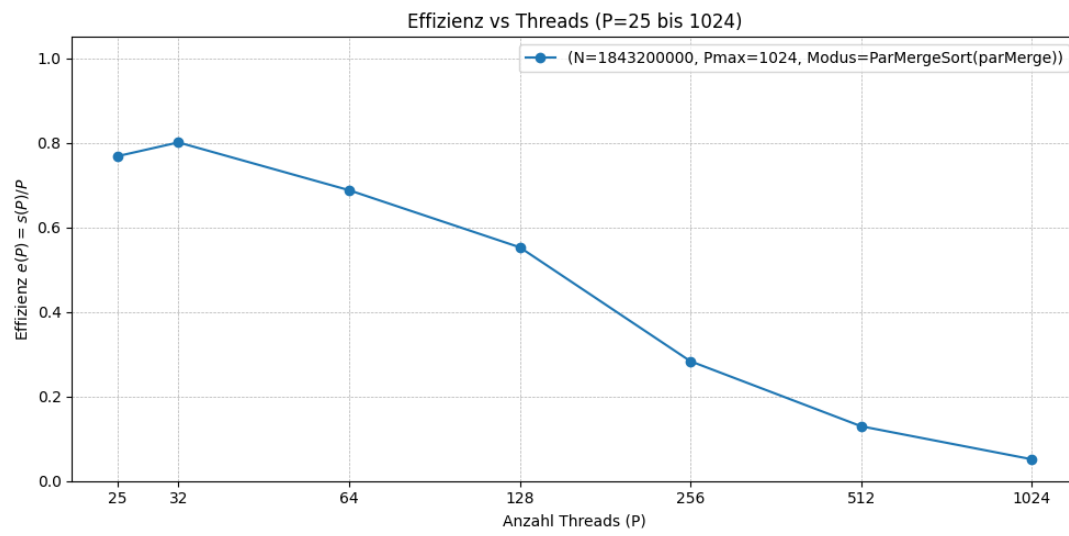


Abbildung 41

Tabelle 19: Laufzeit, Speed-up und Effizienz von ParMergeSort (parMerge) für $N = 1\,843\,200\,000$

p	$T(p)$ [ms]		$S(p)$		$e(p)$		S_{theo}		e_{theo}	
	Wert	$\Delta\%$	Wert	$\Delta\%$	Wert	$\Delta\%$	Wert	—	Wert	—
1	258835	—	1.00	—	1.000	—	1.00	—	1.00	—
2	133426	-48.5%	1.94	+94.0%	0.970	-3.0%	2.00	—	1.00	—
3	93983	-29.6%	2.75	+42.0%	0.918	-5.4%	3.00	—	1.00	—
4	71533	-23.9%	3.62	+31.4%	0.905	-1.5%	4.00	—	1.00	—
5	57721	-19.3%	4.48	+23.9%	0.897	-0.9%	5.00	—	1.00	—
6	47316	-18.0%	5.47	+22.0%	0.912	+1.7%	6.00	—	1.00	—
7	41752	-11.8%	6.20	+13.3%	0.886	-2.9%	7.00	—	1.00	—
8	36570	-12.4%	7.08	+14.2%	0.885	-0.1%	8.00	—	1.00	—
9	34433	-5.8%	7.52	+6.2%	0.835	-5.6%	9.00	—	1.00	—
10	29966	-13.0%	8.64	+14.9%	0.864	+3.4%	10.00	—	1.00	—
11	28206	-5.9%	9.18	+6.2%	0.834	-3.4%	11.00	—	1.00	—
12	26173	-7.2%	9.89	+7.8%	0.824	-1.2%	12.00	—	1.00	—
13	24218	-7.5%	10.69	+8.1%	0.822	-0.2%	13.00	—	1.00	—
14	22301	-7.9%	11.61	+8.6%	0.829	+0.8%	14.00	—	1.00	—
15	20649	-7.4%	12.53	+8.0%	0.836	+0.8%	15.00	—	1.00	—
16	20207	-2.1%	12.81	+2.2%	0.801	-4.2%	16.00	—	1.00	—
17	19335	-4.3%	13.39	+4.5%	0.787	-1.6%	17.00	—	1.00	—
18	18563	-4.0%	13.94	+4.2%	0.775	-1.6%	18.00	—	1.00	—
19	16716	-9.9%	15.48	+11.0%	0.815	+5.2%	19.00	—	1.00	—
20	16607	-0.7%	15.59	+0.7%	0.779	-4.4%	20.00	—	1.00	—
21	16051	-3.3%	16.13	+3.5%	0.768	-1.5%	21.00	—	1.00	—
22	15381	-4.2%	16.83	+4.4%	0.765	-0.4%	22.00	—	1.00	—
23	14353	-6.7%	18.03	+7.2%	0.784	+2.5%	23.00	—	1.00	—
24	14138	-1.5%	18.31	+1.5%	0.763	-2.7%	24.00	—	1.00	—
25	13477	-4.7%	19.21	+4.9%	0.768	+0.7%	25.00	—	1.00	—
32	10102	-25.0%	25.62	+33.4%	0.801	+4.2%	32.00	—	1.00	—
64	5881	-41.8%	44.01	+71.8%	0.688	-14.1%	64.00	—	1.00	—
128	3659	-37.8%	70.74	+60.7%	0.553	-19.6%	128.00	—	1.00	—
256	3570	-2.4%	72.50	+2.5%	0.283	-48.8%	256.00	—	1.00	—
512	3905	+9.4%	66.28	-8.6%	0.129	-54.3%	512.00	—	1.00	—
1024	4921	+26.0%	52.60	-20.6%	0.051	-60.3%	1024.00	—	1.00	—

Vergleich Messung vs. Theorie für ParMergeSort (parMerge) bei $N = 1\,843\,200\,000$ Bis etwa $p = 16$ zeigt ParMergeSort(parMerge) fast lineare Beschleunigung: Bei $p = 2$ verdoppelt sich die Geschwindigkeit nahezu ($S \approx 1,94$), bei $p = 4$ $S \approx 3,62$, bei $p = 8$ $S \approx 7,08$ und bei $p = 16$ $S \approx 12,81$. Die Effizienz

bleibt dabei mit Werten von ca. 0,97 (für $p = 2$) bis 0,80 (für $p = 16$) sehr hoch und damit nahe am theoretisch idealen $e(p) = 1$. Ab $p = 32$ flaut der Zugewinn spürbar ab: Der Speed-up steigt zwar noch auf $S \approx 25,62$, die Effizienz hält aber nur noch bei etwa 0,80. Für größere Kernzahlen ($p \geq 64$) sinkt die Effizienz deutlich – auf etwa 0,69 bei $p = 64$ und nur noch 0,28–0,05 zwischen $p = 256$ und 1024. Dieses Verhalten verdeutlicht, dass ab etwa $p = 32$ der praktische Overhead für Merge- und Kommunikationsschritte die Skalierung bremst, während das theoretische Work-Span-Modell weiterhin perfekte Linearität prognostiziert.

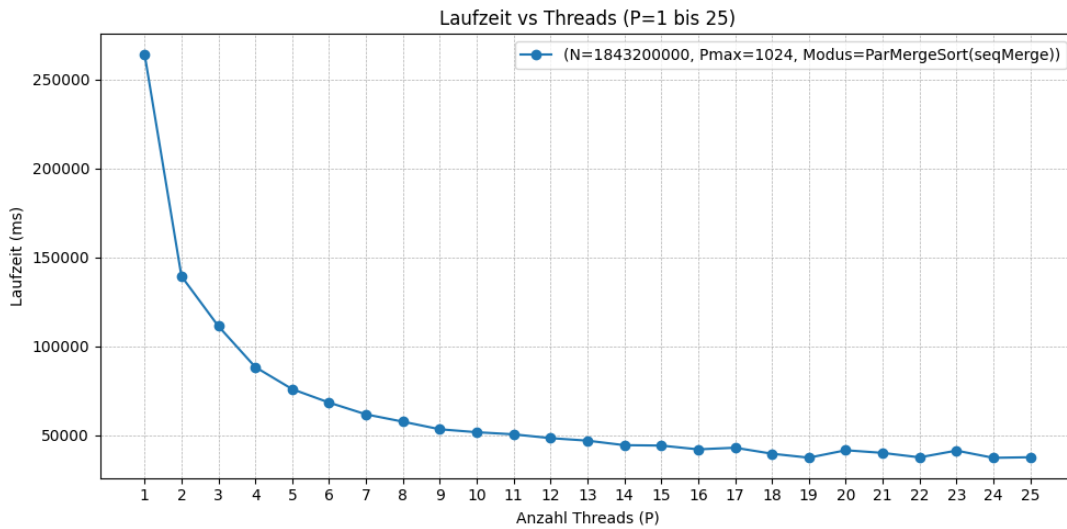


Abbildung 42

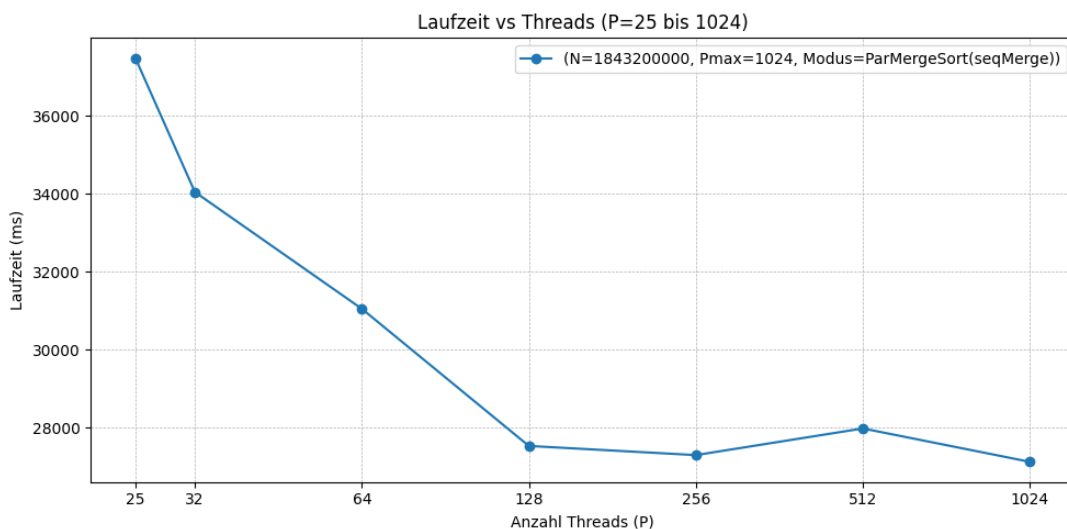


Abbildung 43

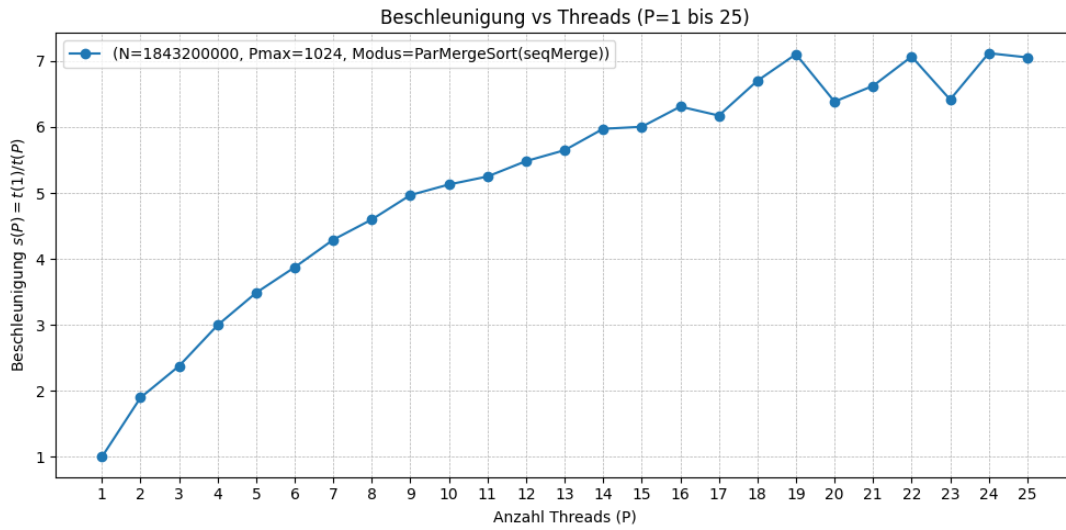


Abbildung 44

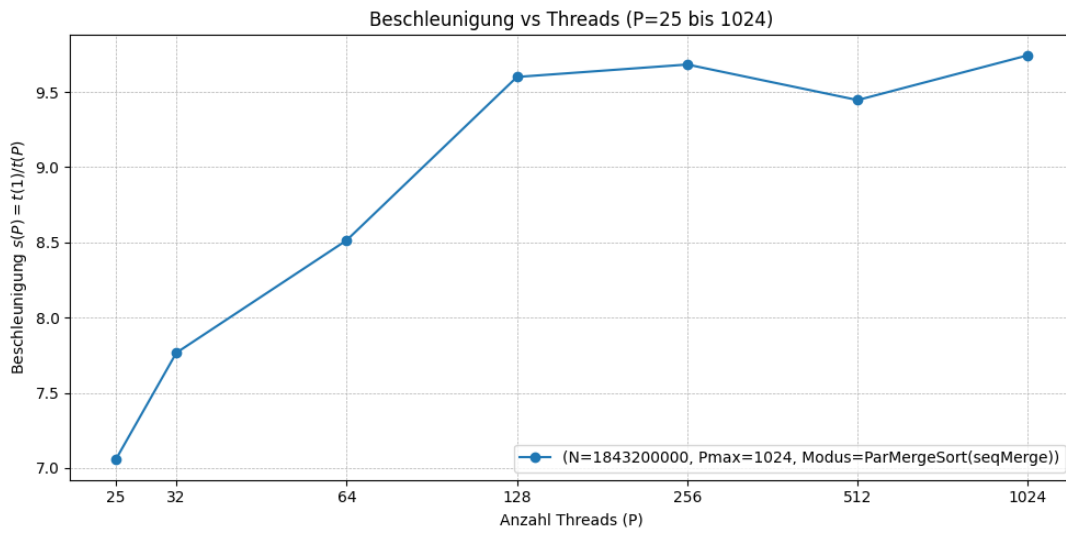


Abbildung 45

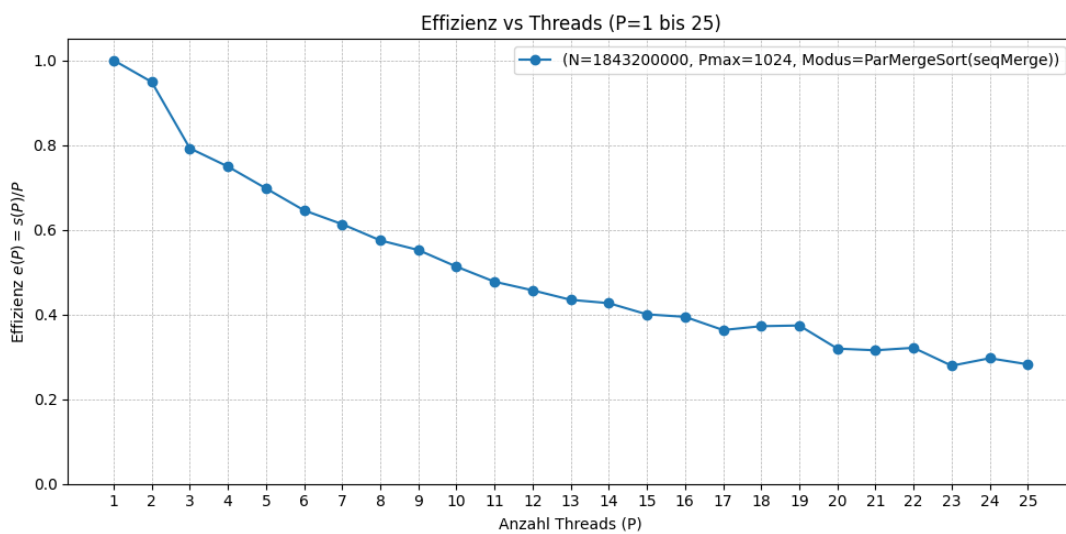


Abbildung 46

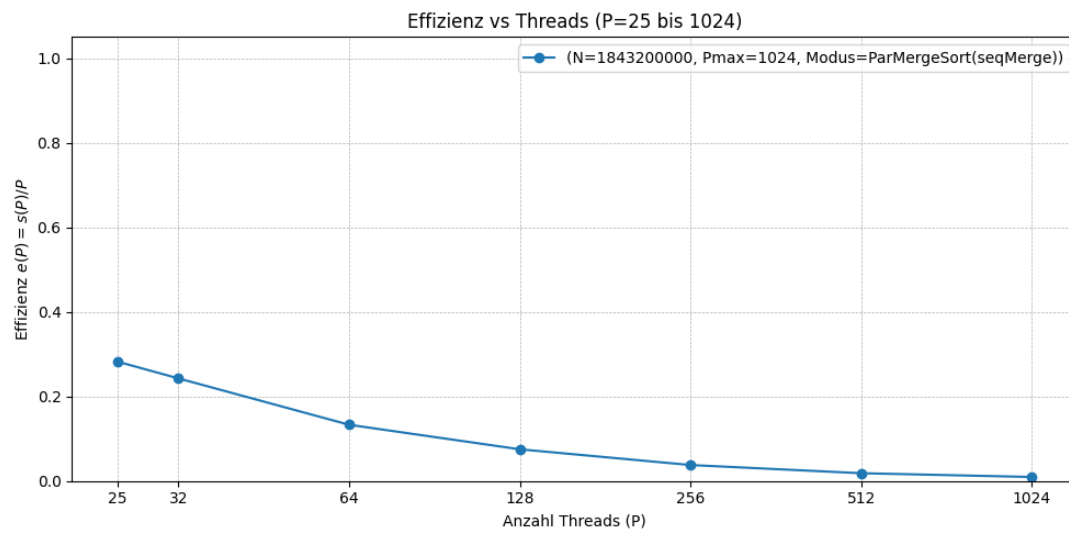


Abbildung 47

Tabelle 20: Laufzeit, Speed-up und Effizienz von ParMergeSort (seqMerge) für $N = 1\,843\,200\,000$

p	$T(p)$ [ms]		$S(p)$		$e(p)$		S_{theo}		e_{theo}	
	Wert	$\Delta\%$	Wert	$\Delta\%$	Wert	$\Delta\%$	Wert	—	Wert	—
1	264362	—	1.00	—	1.000	—	1.00	—	1.00	—
2	139317	-47.3%	1.90	+89.8%	0.949	-5.1%	2.00	—	1.00	—
3	111230	-20.2%	2.38	+25.3%	0.792	-16.5%	3.00	—	1.00	—
4	88228	-20.7%	3.00	+26.1%	0.749	-5.4%	4.00	—	1.00	—
5	75802	-14.1%	3.49	+16.4%	0.698	-6.9%	5.00	—	1.00	—
6	68234	-10.0%	3.87	+11.1%	0.646	-7.4%	6.00	—	1.00	—
7	61598	-9.7%	4.29	+10.8%	0.613	-5.1%	7.00	—	1.00	—
8	57490	-6.7%	4.60	+7.1%	0.575	-6.2%	8.00	—	1.00	—
9	53211	-7.4%	4.97	+8.0%	0.552	-4.0%	9.00	—	1.00	—
10	51547	-3.1%	5.13	+3.2%	0.513	-7.1%	10.00	—	1.00	—
11	50358	-2.3%	5.25	+2.4%	0.477	-6.9%	11.00	—	1.00	—
12	48214	-4.3%	5.48	+4.4%	0.457	-4.3%	12.00	—	1.00	—
13	46804	-2.9%	5.65	+3.0%	0.434	-4.9%	13.00	—	1.00	—
14	44260	-5.4%	5.97	+5.7%	0.427	-1.8%	14.00	—	1.00	—
15	44038	-0.5%	6.00	+0.5%	0.400	-6.2%	15.00	—	1.00	—
16	41902	-4.9%	6.31	+5.1%	0.394	-1.5%	16.00	—	1.00	—
17	42821	+2.2%	6.17	-2.1%	0.363	-7.9%	17.00	—	1.00	—
18	39451	-7.9%	6.70	+8.5%	0.372	+2.5%	18.00	—	1.00	—
19	37216	-5.7%	7.10	+6.0%	0.374	+0.4%	19.00	—	1.00	—
20	41401	+11.2%	6.39	-10.1%	0.319	-14.6%	20.00	—	1.00	—
21	39913	-3.6%	6.62	+3.7%	0.315	-1.2%	21.00	—	1.00	—
22	37404	-6.3%	7.07	+6.7%	0.321	+1.9%	22.00	—	1.00	—
23	41205	+10.2%	6.42	-9.2%	0.279	-13.2%	23.00	—	1.00	—
24	37133	-9.9%	7.12	+11.0%	0.297	+6.3%	24.00	—	1.00	—
25	37471	+0.9%	7.06	-0.9%	0.282	-4.9%	25.00	—	1.00	—
32	34042	-9.2%	7.77	+10.1%	0.243	-14.0%	32.00	—	1.00	—
64	31050	-8.8%	8.51	+9.6%	0.133	-45.2%	64.00	—	1.00	—
128	27538	-11.3%	9.60	+12.8%	0.075	-43.6%	128.00	—	1.00	—
256	27303	-0.9%	9.68	+0.9%	0.038	-49.6%	256.00	—	1.00	—
512	27986	+2.5%	9.45	-2.4%	0.018	-51.2%	512.00	—	1.00	—
1024	27131	-3.1%	9.74	+3.2%	0.010	-48.4%	1024.00	—	1.00	—

Vergleich Messung vs. Theorie für ParMergeSort (seqMerge) bei $N = 1\,843\,200\,000$ Bis etwa $p = 8$ wächst der gemessene Speed-up noch einigermaßen linear: von $S(2) \approx 1,90$ über $S(4) \approx 3,00$ bis $S(8) \approx 4,60$, während die Effizienz von $e(2) \approx 0,95$ auf $e(8) \approx 0,58$ moderat absinkt (siehe Tabelle 20). Ab $p = 16$

flacht der Zugewinn deutlich ab: der Speed-up steigt nur noch auf $S(16) \approx 6,31$ und die Effizienz fällt auf $e(16) \approx 0,39$. Bei sehr hohen Kernzahlen (z. B. $p = 1024$) liegt die Effizienz mit etwa $e(1024) \approx 0,01$ praktisch bei null. Dieses Verhalten verdeutlicht, dass der fehlende echte parallele Merge-Schritt und die wachsenden Synchronisations- und Speicherzugriffskosten ab mittleren p die Skalierung stark bremsen, während das ideale Work-Span-Modell weiterhin perfekte Linearität prognostiziert.

Tabelle 21: Vergleich gemessener Laufzeiten, Speed-ups und Effizienzen: ParMergeSort (parMerge) vs. ParMergeSort (seqMerge) für $N = 1\,843\,200\,000$

p	T_{par} [ms]	T_{seq} [ms]	S_{par}	S_{seq}	e_{par}	e_{seq}
1	258835	264362	1.00	1.00	1.000	1.000
2	133426	139317	1.94	1.90	0.970	0.949
3	93983	111230	2.75	2.38	0.918	0.792
4	71533	88228	3.62	3.00	0.905	0.749
5	57721	75802	4.48	3.49	0.897	0.698
6	47316	68234	5.47	3.87	0.912	0.646
7	41752	61598	6.20	4.29	0.886	0.613
8	36570	57490	7.08	4.60	0.885	0.575
9	34433	53211	7.52	4.97	0.835	0.552
10	29966	51547	8.64	5.13	0.864	0.513
11	28206	50358	9.18	5.25	0.834	0.477
12	26173	48214	9.89	5.48	0.824	0.457
13	24218	46804	10.69	5.65	0.822	0.434
14	22301	44260	11.61	5.97	0.829	0.427
15	20649	44038	12.53	6.00	0.836	0.400
16	20207	41902	12.81	6.31	0.801	0.394
17	19335	42821	13.39	6.17	0.787	0.363
18	18563	39451	13.94	6.70	0.775	0.372
19	16716	37216	15.48	7.10	0.815	0.374
20	16607	41401	15.59	6.39	0.779	0.319
21	16051	39913	16.13	6.62	0.768	0.315
22	15381	37404	16.83	7.07	0.765	0.321
23	14353	41205	18.03	6.42	0.784	0.279
24	14138	37133	18.31	7.12	0.763	0.297
25	13477	37471	19.21	7.06	0.768	0.282
32	10102	34042	25.62	7.77	0.801	0.243
64	5881	31050	44.01	8.51	0.688	0.133
128	3659	27538	70.74	9.60	0.553	0.075
256	3570	27303	72.50	9.68	0.283	0.038
512	3905	27986	66.28	9.45	0.129	0.018
1024	4921	27131	52.60	9.74	0.051	0.010

Vergleich Messung ParMergeSort(parMerge) vs. ParMergeSort(seqMerge) für $N = 1\,843\,200\,000$ In Tabelle 21 sind die gemessenen Laufzeiten $T(p)$, Speed-ups $S(p) = T(1)/T(p)$ und Effizienzen $e(p) = S(p)/p$ für parMerge und seqMerge gegenübergestellt. Bereits bei $p = 4$ liegt parMerge mit $S_{\text{par}} = 3,62$ und $e_{\text{par}} = 0,91$ deutlich vor seqMerge ($S_{\text{seq}} = 3,00$, $e_{\text{seq}} = 0,75$). Dieser Unterschied bleibt bis mittlere Kernzahlen bestehen: Bei $p = 16$ erreicht parMerge einen Speed-up von 12,81 und eine Effizienz von 0,80, während seqMerge nur 6,31 bzw. 0,39 schafft. Erst ab sehr hohen p (z. B. $p = 1024$) nähern sich beide Verfahren etwas an, da der Merge- und Kommunikations-Overhead dominanter wird und die Effizienz von parMerge auf etwa 0,05 und von seqMerge auf etwa 0,01 absinkt. Insgesamt verdeutlicht der Vergleich, dass der echte parallele Merge von parMergeSort auch für doppelt großes Input die Skalierung erheblich verbessert, während der sequenzielle Merge die Skalierung bei seqMerge bereits ab mittleren Kernzahlen stark einschränkt.

Tabelle 22: Vergleich Messung ParMergeSort (parMerge) vs. Bitonic Merge (BIM) für $N = 1\,843\,200\,000$

2^*p	Laufzeit [ms]				Speed-up				Effizienz			
	T_{par}	ΔT_{par}	T_{BIM}	ΔT_{BIM}	S_{par}	ΔS_{par}	S_{BIM}	ΔS_{BIM}	e_{par}	Δe_{par}	e_{BIM}	Δe_{BIM}
1	258835	—	258907	—	1.00	—	1.00	—	1.000	—	1.000	—
2	133426	-48.5%	127522	-50.7%	1.94	+94.0%	2.03	+103.0%	0.970	-3.0%	1.015	+1.5%
4	71533	-46.4%	66088	-48.2%	3.62	+86.7%	3.92	+93.1%	0.905	-5.4%	0.979	-3.5%
8	36570	-48.9%	34840	-47.3%	7.08	+95.5%	7.43	+89.5%	0.885	-3.6%	0.929	-5.2%
16	20207	-44.7%	17412	-50.0%	12.81	+80.9%	14.87	+100.1%	0.801	-9.5%	0.929	0.0%
32	10102	-50.0%	9753	-44.0%	25.62	+100.0%	26.55	+78.5%	0.801	0.0%	0.830	-10.7%
64	5881	-41.8%	5558	-43.0%	44.01	+71.8%	46.58	+75.5%	0.688	-14.1%	0.728	-12.3%
128	3659	-37.8%	5324	-4.2%	70.74	+60.7%	48.63	+4.4%	0.553	-19.7%	0.380	-47.8%
256	3570	-2.4%	4661	-12.5%	72.50	+2.5%	54.01	+5.1%	0.283	-48.8%	0.211	-47.5%
512	3905	+9.4%	4656	-0.1%	66.28	-8.6%	48.01	-11.1%	0.129	-54.4%	0.094	-55.6%
1024	4921	+26.0%	5593	+20.1%	52.60	-20.7%	36.90	-16.8%	0.051	-60.5%	0.045	-58.7%

Vergleich Messung ParMergeSort(parMerge) vs. Bitonic Merge (BIM) für $N = 1\,843\,200\,000$ Tabelle 22 zeigt, wie sich ParMergeSort(parMerge) und Bitonic Merge (BIM) bei der doppelten Problemgröße verhalten. Bei kleinen Kernzahlen (z. B. $p = 2-64$) erreicht BIM stets geringfügig höhere Speed-ups und Effizienzen: So kommt BIM bei $p = 4$ auf einen Speed-up von 3,92 und eine Effizienz von 0,98, während parMerge 3,62 bzw. 0,91 erzielt. Erst ab $p = 128$

wendet sich das Blatt: ParMergeSort erreicht nun einen Speed-up von etwa 70,7 und eine Effizienz von 0,55, während BIM bei 48,6 beziehungsweise 0,38 verharret. Auch bei sehr hohen Kernzahlen (z. B. $p = 1024$) bleibt parMerge mit $S \approx 52,6$, $e \approx 0,05$ leicht vor BIM ($S \approx 36,9$, $e \approx 0,04$). Insgesamt zeigt der Vergleich, dass Bitonic Merge bei kleineren bis mittleren p etwas vorn liegt, ParMergeSort aber ab mittleren bis großen p deutlich bessere Skalierung und Auslastung der Kerne bietet.

Tabelle 23: Vergleich ParMergeSort (parMerge) vs. Odd–Even Merge Splitting Sort (OEM) für $N = 1\,843\,200\,000$

p	Laufzeit [ms]				Speed-up				Effizienz			
	T_{par}	ΔT_{par}	T_{OEM}	ΔT_{OEM}	S_{par}	ΔS_{par}	S_{OEM}	ΔS_{OEM}	e_{par}	Δe_{par}	e_{OEM}	Δe_{OEM}
1	258835	—	258907	—	1.00	—	1.00	—	1.000	—	1.000	—
2	133426	−48.5%	135966	−47.4%	1.94	+94.0%	1.90	+90.2%	0.970	−3.0%	0.950	−4.9%
3	93983	−29.6%	107341	−21.0%	2.75	+42.0%	2.41	+26.7%	0.918	−5.4%	0.800	−15.6%
4	71533	−23.9%	81071	−24.4%	3.62	+31.4%	3.19	+32.4%	0.905	−1.5%	0.800	−0.7%
5	57721	−19.3%	77268	−4.7%	4.48	+23.9%	3.35	+4.9%	0.897	−0.9%	0.670	−16.1%
6	47316	−18.0%	63032	−18.4%	5.47	+22.0%	4.10	+22.6%	0.912	+1.7%	0.680	+2.2%
8	36570	−22.7%	55927	−11.3%	7.08	+29.4%	4.63	+12.7%	0.885	−3.0%	0.580	−15.5%
9	34433	−5.8%	55653	−0.5%	7.52	+6.2%	4.65	+0.4%	0.835	−5.6%	0.520	−10.7%
10	29966	−13.0%	52602	−5.9%	8.64	+14.9%	4.92	+5.8%	0.864	+3.4%	0.490	−4.7%
12	26173	−12.7%	48717	−7.4%	9.89	+14.5%	5.31	+8.0%	0.824	−4.6%	0.440	−10.1%
15	20649	−21.1%	48666	−0.1%	12.53	+26.8%	5.31	+0.1%	0.836	+1.4%	0.350	−19.9%
16	20207	−2.1%	45582	−6.3%	12.81	+2.2%	5.23	−1.6%	0.801	−4.2%	0.330	−7.7%
18	18563	−8.1%	46404	+1.8%	13.94	+8.9%	5.58	+6.6%	0.775	−3.2%	0.310	−5.2%
20	16607	−10.5%	48266	+4.0%	15.59	+11.8%	5.35	−4.0%	0.779	+0.6%	0.270	−13.5%
24	14138	−14.9%	49361	+2.3%	18.31	+17.5%	5.24	−2.1%	0.763	−2.7%	0.220	−18.7%

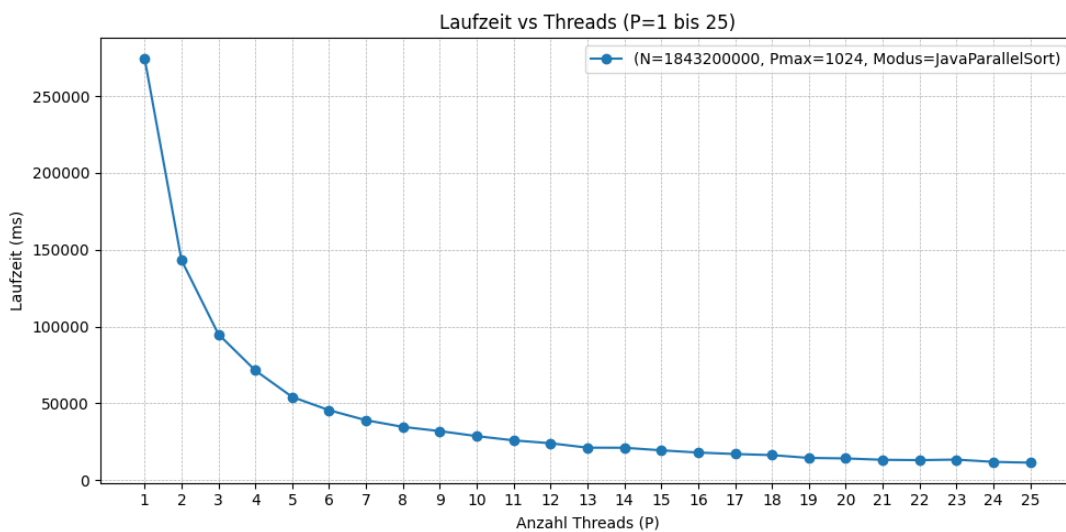


Abbildung 48

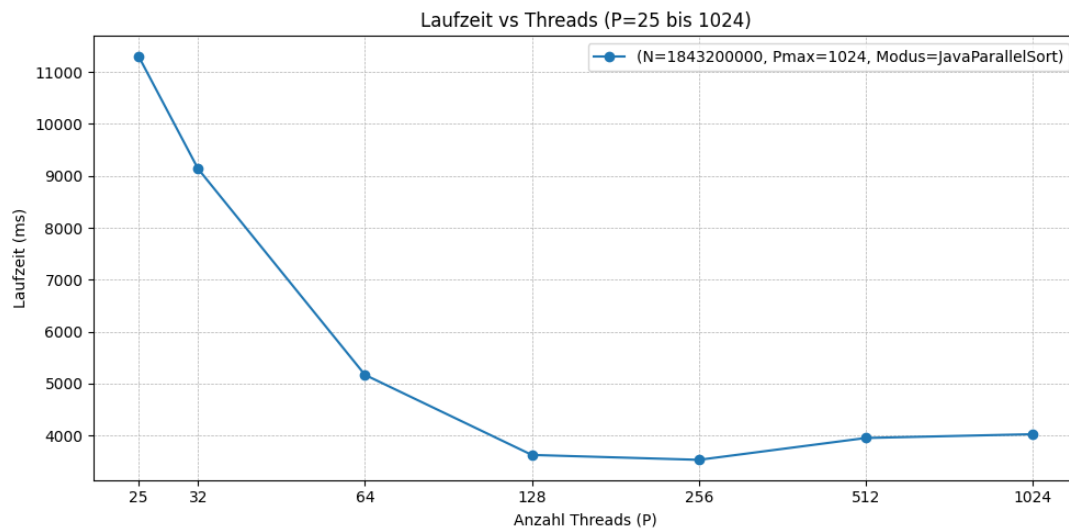


Abbildung 49

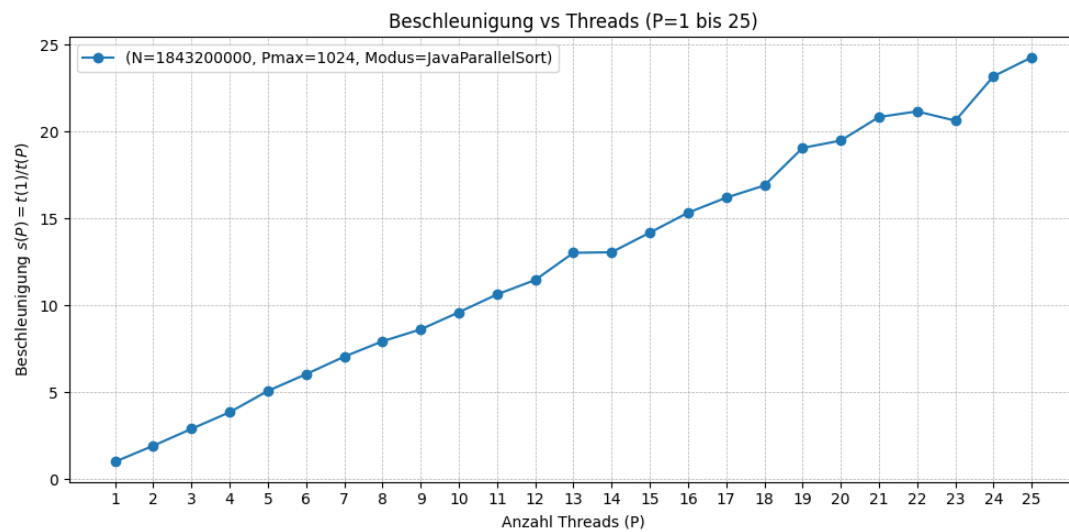


Abbildung 50

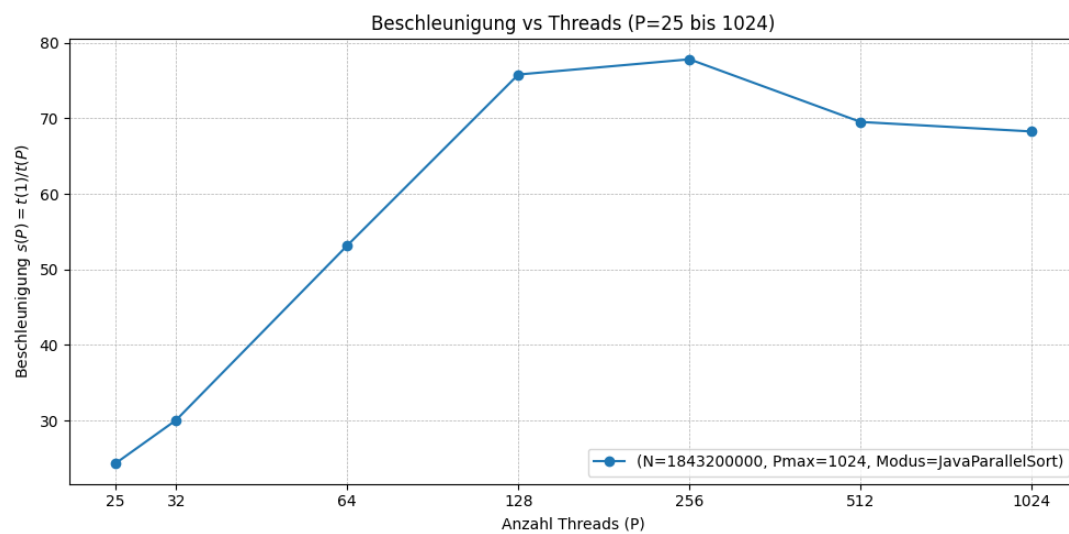


Abbildung 51

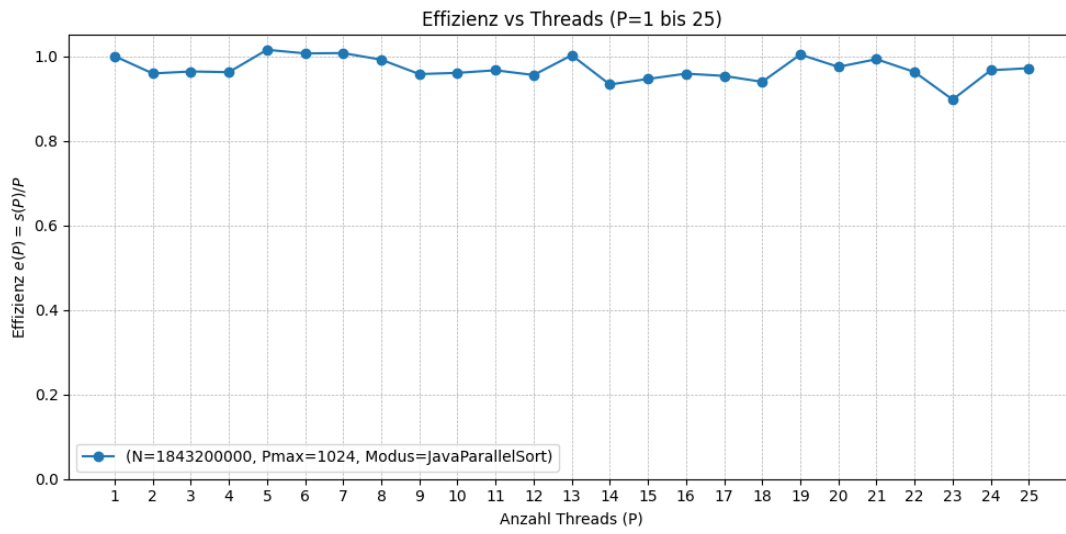


Abbildung 52

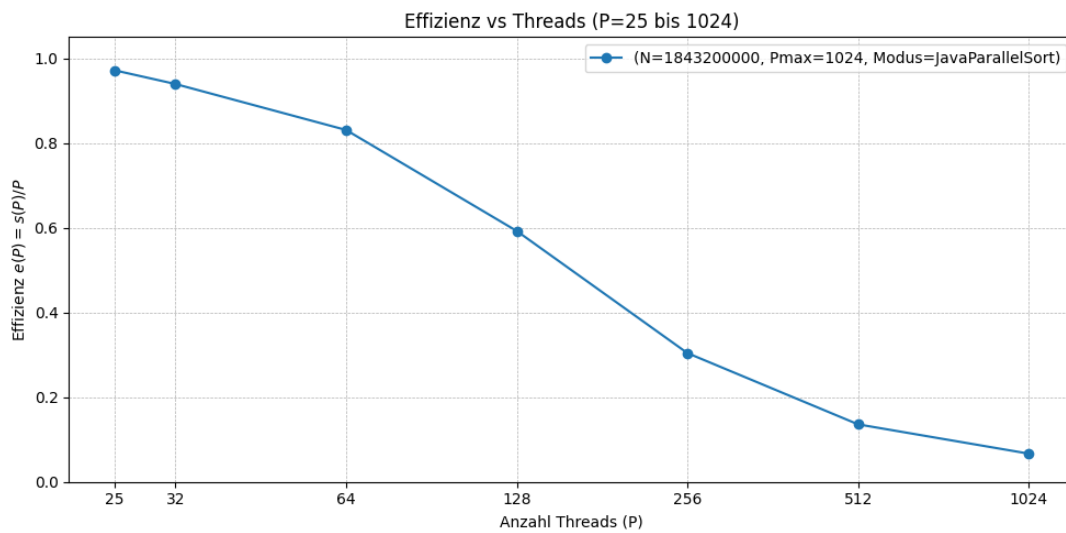


Abbildung 53

Tabelle 24: Laufzeit, Speed-up und Effizienz für `Arrays.parallelSort()` bei $N = 1\,843\,200\,000$

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	$\Delta\%$	Wert	$\Delta\%$	Wert	$\Delta\%$
1	274639	—	1.00	—	1.000	—
2	143189	−47.9%	1.92	+91.8%	0.959	−4.1%
3	94985	−33.7%	2.89	+50.7%	0.964	+0.5%
4	71364	−24.9%	3.85	+33.1%	0.962	−0.2%
5	54115	−24.2%	5.08	+31.9%	1.015	+5.5%
6	45477	−16.0%	6.04	+19.0%	1.007	−0.8%
7	38954	−14.3%	7.05	+16.7%	1.007	+0.1%
8	34621	−11.1%	7.93	+12.5%	0.992	−1.5%
9	31868	−8.0%	8.62	+8.6%	0.958	−3.4%
10	28595	−10.3%	9.60	+11.4%	0.960	+0.3%
11	25831	−9.7%	10.63	+10.7%	0.967	+0.6%
12	23957	−7.3%	11.46	+7.8%	0.955	−1.2%
13	21080	−12.0%	13.03	+13.6%	1.002	+4.9%
14	21027	−0.3%	13.06	+0.3%	0.933	−6.9%
15	19353	−8.0%	14.19	+8.6%	0.946	+1.4%
16	17906	−7.5%	15.34	+8.1%	0.959	+1.3%
17	16948	−5.4%	16.20	+5.7%	0.953	−0.6%
18	16245	−4.1%	16.91	+4.3%	0.939	−1.5%
19	14402	−11.3%	19.07	+12.8%	1.004	+6.9%
20	14089	−2.2%	19.49	+2.2%	0.975	−2.9%
21	13173	−6.5%	20.85	+7.0%	0.993	+1.9%
22	12971	−1.5%	21.17	+1.6%	0.962	−3.1%
23	13308	+2.6%	20.64	−2.5%	0.897	−6.8%
24	11839	−11.0%	23.20	+12.4%	0.967	+7.7%
25	11307	−4.5%	24.29	+4.7%	0.972	+0.5%
32	9138	−19.2%	30.05	+23.7%	0.939	−3.3%
64	5166	−43.5%	53.16	+76.9%	0.831	−11.6%
128	3623	−29.9%	75.80	+42.6%	0.592	−28.7%
256	3529	−2.6%	77.82	+2.7%	0.304	−48.7%
512	3950	+11.9%	69.53	−10.7%	0.136	−55.3%
1024	4023	+1.8%	68.27	−1.8%	0.067	−50.9% ⁸²

Tabelle 25: Vergleich ParMergeSort (parMerge) vs. Arrays.parallelSort() für $N = 1\,843\,200\,000$

p	Laufzeit [ms]				Speed-up				Effizienz			
	T_{par}	ΔT_{par}	T_{java}	ΔT_{java}	S_{par}	ΔS_{par}	S_{java}	ΔS_{java}	e_{par}	Δe_{par}	e_{java}	Δe_{java}
1	258835	—	274639	—	1.00	—	1.00	—	1.000	—	1.000	—
2	133426	-48.5%	143189	-47.9%	1.94	+94.0%	1.92	+91.8%	0.970	-3.0%	0.959	-4.1%
3	93983	-29.6%	94985	-33.7%	2.75	+42.0%	2.89	+50.7%	0.918	-5.4%	0.964	+0.5%
4	71533	-23.9%	71364	-24.9%	3.62	+31.4%	3.85	+33.1%	0.905	-1.5%	0.962	-0.2%
5	57721	-19.3%	54115	-24.2%	4.48	+23.9%	5.08	+31.9%	0.897	-0.9%	1.015	+5.5%
6	47316	-18.0%	45477	-16.0%	5.47	+22.0%	6.04	+19.0%	0.912	+1.7%	1.007	-0.8%
7	41752	-11.8%	38954	-14.3%	6.20	+13.3%	7.05	+16.7%	0.886	-2.9%	1.007	+0.1%
8	36570	-12.4%	34621	-11.1%	7.08	+14.2%	7.93	+12.5%	0.885	-0.1%	0.992	-1.5%
9	34433	-5.8%	31868	-8.0%	7.52	+6.2%	8.62	+8.6%	0.835	-5.6%	0.958	-3.4%
10	29966	-13.0%	28595	-10.3%	8.64	+14.9%	9.60	+11.4%	0.864	+3.4%	0.960	+0.3%
11	28206	-5.9%	25831	-9.7%	9.18	+6.2%	10.63	+10.7%	0.834	-3.4%	0.967	+0.6%
12	26173	-7.2%	23957	-7.3%	9.89	+7.8%	11.46	+7.8%	0.824	-1.2%	0.955	-1.2%
13	24218	-7.5%	21080	-12.0%	10.69	+8.1%	13.03	+13.6%	0.822	-0.2%	1.002	+4.9%
14	22301	-7.9%	21027	-0.3%	11.61	+8.6%	13.06	+0.3%	0.829	+0.8%	0.933	-6.9%
15	20649	-7.4%	19353	-8.0%	12.53	+8.0%	14.19	+8.6%	0.836	+0.8%	0.946	+1.4%
16	20207	-2.1%	17906	-7.5%	12.81	+2.2%	15.34	+8.1%	0.801	-4.2%	0.959	+1.3%
17	19335	-4.3%	16948	-5.4%	13.39	+4.5%	16.20	+5.7%	0.787	-1.6%	0.953	-0.6%
18	18563	-4.0%	16245	-4.1%	13.94	+4.2%	16.91	+4.3%	0.775	-1.6%	0.939	-1.5%
19	16716	-9.9%	14402	-11.3%	15.48	+11.0%	19.07	+12.8%	0.815	+5.2%	1.004	+6.9%
20	16607	-0.7%	14089	-2.2%	15.59	+0.7%	19.49	+2.2%	0.779	-4.4%	0.975	-2.9%
21	16051	-3.3%	13173	-6.5%	16.13	+3.5%	20.85	+7.0%	0.768	-1.5%	0.993	+1.9%
22	15381	-4.2%	12971	-1.5%	16.83	+4.4%	21.17	+1.6%	0.765	-0.4%	0.962	-3.1%
23	14353	-6.7%	13308	+2.6%	18.03	+7.2%	20.64	-2.5%	0.784	+2.5%	0.897	-6.8%
24	14138	-1.5%	11839	-11.0%	18.31	+1.5%	23.20	+12.4%	0.763	-2.7%	0.967	+7.7%
25	13477	-4.7%	11307	-4.5%	19.21	+4.9%	24.29	+4.7%	0.768	+0.7%	0.972	+0.5%
32	10102	-25.0%	9138	-19.2%	25.62	+33.4%	30.05	+23.7%	0.801	+4.2%	0.939	-3.3%
64	5881	-41.8%	5166	-43.5%	44.01	+71.8%	53.16	+76.9%	0.688	-14.1%	0.831	-11.6%
128	3659	-37.8%	3623	-29.9%	70.74	+60.7%	75.80	+42.6%	0.553	-19.6%	0.592	-28.7%
256	3570	-2.4%	3529	-2.6%	72.50	+2.5%	77.82	+2.7%	0.283	-48.8%	0.304	-48.7%
512	3905	+9.4%	3950	+11.9%	66.28	-8.6%	69.53	-10.7%	0.129	-54.3%	0.136	-55.3%
1024	4921	+26.0%	4023	+1.8%	52.60	-20.6%	68.27	-1.8%	0.051	-60.3%	0.067	-50.9%

Vergleich Messung ParMergeSort(parMerge) vs. Arrays.parallelSort()
für $N = 1\,843\,200\,000$ Tabelle 25 stellt die gemessenen Laufzeiten $T(p)$, Speed-ups $S(p) = T(1)/T(p)$ und Effizienzen $e(p) = S(p)/p$ für ParMergeSort(parMerge) und Javas Arrays.parallelSort() bei $N = 1,843,200,000$ gegenüber. Während parMerge bei $p = 1$ mit $T_{\text{par}} = 258\,835\text{ms}$ geringfügig schneller startet als Java ($T_{\text{java}} = 274\,639\text{ms}$), wendet sich das Blatt bereits ab $p = 3$: parallelSort liefert ab hier durchweg höhere Speed-ups (z.B. $S_{\text{java}}(4) \approx 3,85$ vs. $S_{\text{par}}(4) \approx 3,62$) und bessere Effizienzen ($e_{\text{java}}(4) \approx 0,96$ vs. $e_{\text{par}}(4) \approx 0,91$). Dieser Vorsprung

bleibt bis in den hohen Bereich erhalten – etwa bei $p = 64$ erzielt Java $S \approx 53,2$, $e \approx 0,83$ im Vergleich zu parMerge mit $S \approx 44,0$, $e \approx 0,69$. Erst ab sehr großen Kernzahlen (ab $p = 512$) schrumpfen beide Effizienzen stark, doch **parallelSort** behält meist noch einen leichten Vorsprung. Insgesamt zeigt der Vergleich, dass Javas optimierte Parallelimplementierung bei der größeren Datenmenge effizienter skaliert und die verfügbaren Kerne besser ausnutzt als die selbst implementierte ParMergeSort-Variante.

Tabelle 26: Vergleich der gemessenen Laufzeiten für $N = 1\,843\,200\,000$

p	javaParSort	parMerge	seqMerge	BIM	OEMS
1	274639	258835	264362	258907	258907
2	143189	133426	139317	127522	135966
3	94985	93983	111230	—	107341
4	71364	71533	88228	66088	81071
5	54115	57721	75802	—	77268
6	45477	47316	68234	—	63032
7	38954	41752	61598	—	—
8	34621	36570	57490	34840	55927
9	31868	34433	53211	—	55653
10	28595	29966	51547	—	52602
11	25831	28206	50358	—	—
12	23957	26173	48214	—	48717
13	21080	24218	46804	—	—
14	21027	22301	44260	—	—
15	19353	20649	44038	—	48666
16	17906	20207	41902	17412	45582
17	16948	19335	42821	—	—
18	16245	18563	39451	—	46404
19	14402	16716	37216	—	—
20	14089	16607	41401	—	48266
21	13173	16051	39913	—	—
22	12971	15381	37404	—	—
23	13308	14353	41205	—	—
24	11839	14138	37133	—	49361
25	11307	13477	37471	—	—
32	9138	10102	34042	9753	—
64	5166	5881	31050	5558	—
128	3623	3659	27538	5324	—
256	3529	3570	27303	4661	—
512	3950	3905	27986	4656	—
1024	4023	4921	27131	5593	—

Vergleich der Laufzeiten aller Algorithmen für $N = 1\,843\,200\,000$ Tabelle 26 gibt die reinen Laufzeiten aller fünf Verfahren bei der doppelten Eingabegröße wieder. Zu Beginn ($p = 1$) benötigt `Arrays.parallelSort()` mit etwa 275 s die längste Zeit, während `ParMergeSort(parMerge)` und Bitonic Merge (BIM) knapp 259 s liegen. Bis $p = 8$ skaliert Java mit ca. 7,93-fachem Speed-up noch am besten, `ParMerge` folgt mit 7,08, BIM mit 7,44 und `seqMerge/OEMS` bleiben bei rund 4,6. Ab $p = 16$ kann `ParMergeSort` seinen Vorsprung ausbauen (Speed-up 12,81, Effizienz 0,80), während Java bei $p = 16$ auf 15,34 bzw. 0,96 kommt. Bitonic Merge liegt hier ähnlich knapp dahinter. `SeqMerge` und `OEMS` bremsen ab mittleren p stärker und erreichen bei $p = 16$ lediglich Speed-ups um 6,31 bzw. 5,23. Bei sehr hohen Kernzahlen (ab $p = 64$) gleichen sich alle Verfahren weitgehend an, da der Kommunikations- und Merge-Overhead dominiert. Insgesamt bleibt `ParMergeSort(parMerge)` auch für $N = 1,843,200,000$ konstant unter den schnellsten drei Läufen und zeigt durchgehend effiziente Skalierung, während Java und BIM bei kleinen bis mittleren p konkurrenzfähig sind und `seqMerge/OEMS` bereits früher stagnieren.

Tabelle 27: Vergleich der Speed-ups für $N = 1\,843\,200\,000$

p	javaParSort	parMerge	seqMerge	BIM	OEMS
1	1.00	1.00	1.00	1.00	1.00
2	1.92	1.94	1.90	2.03	1.90
3	2.89	2.75	2.38	—	2.41
4	3.85	3.62	3.00	3.92	3.19
5	5.08	4.48	3.49	—	3.35
6	6.04	5.47	3.87	—	4.10
7	7.05	6.20	4.29	—	—
8	7.93	7.08	4.60	7.43	4.63
9	8.62	7.52	4.97	—	4.65
10	9.60	8.64	5.13	—	4.92
11	10.63	9.18	5.25	—	—
12	11.46	9.89	5.48	—	5.31
13	13.03	10.69	5.65	—	—
14	13.06	11.61	5.97	—	—
15	14.19	12.53	6.00	—	5.31
16	15.34	12.81	6.31	14.87	5.23
17	16.20	13.39	6.17	—	—
18	16.91	13.94	6.70	—	5.58
19	19.07	15.48	7.10	—	—
20	19.49	15.59	6.39	—	5.35
21	20.85	16.13	6.63	—	—
22	21.17	16.83	7.07	—	—
23	20.64	18.03	6.41	—	—
24	23.20	18.31	7.12	—	5.24
25	24.29	19.21	7.06	—	—
32	30.05	25.62	7.76	26.54	—
64	53.16	44.01	8.51	46.56	—
128	75.80	70.74	9.60	48.62	—
256	77.82	72.50	9.68	55.56	—
512	69.53	66.28	9.45	55.61	—
1024	68.27	52.60	9.74	46.28	—

Vergleich der Beschleunigung aller Algorithmen für $N = 1\,843\,200\,000$

Tabelle 27 verdeutlicht, wie die fünf Verfahren mit zunehmender Prozessorzahl an Speed-up gewinnen. Javas `Arrays.parallelSort()` profitiert hier am stärksten: Schon bei $p = 16$ erreicht es einen Speed-up von circa 15,3, und bei $p = 256$ über 77,8. `ParMergeSort(parMerge)` folgt sehr dicht mit ungefähr 12,8 bei $p = 16$ und 72,5 bei $p = 256$. Bitonic Merge (BIM) skaliert bis etwa $p = 32$ nahezu ideal, bricht danach aber stärker ein (z. B. $S_{256} \approx 55,6$). `ParMergeSort(seqMerge)` bleibt ab mittleren p mit nur 6,3–9,7 weit hinter den führenden Verfahren zurück, da der Merge-Schritt sequenziell abläuft. Odd–Even Merge Splitting Sort (OEMS) zeigt ebenfalls nur moderate Zuwächse (maximal um 5,6 bei $p = 9$ bzw. 5,3 bis $p = 24$). Insgesamt profitieren `parallelSort()` und `parMerge` am stärksten von zusätzlichen Kernen, während BIM bei sehr hohen p und die beiden Split-Merge-Varianten früher an ihre Skalierungsgrenze stoßen.

Tabelle 28: Vergleich der Effizienzen $e(p)$ für $N = 1\,843\,200\,000$

p	javaParSort	parMerge	seqMerge	BIM	OEMS
1	1.000	1.000	1.000	1.000	1.000
2	0.960	0.970	0.950	1.015	0.950
3	0.963	0.917	0.793	—	0.803
4	0.963	0.905	0.750	0.980	0.798
5	1.016	0.896	0.698	—	0.670
6	1.007	0.912	0.645	—	0.683
7	1.007	0.886	0.613	—	—
8	0.991	0.885	0.575	0.929	0.579
9	0.958	0.836	0.552	—	0.517
10	0.960	0.864	0.513	—	0.492
11	0.967	0.834	0.477	—	—
12	0.955	0.824	0.457	—	0.443
13	1.002	0.822	0.435	—	—
14	0.933	0.829	0.427	—	—
15	0.946	0.835	0.400	—	0.354
16	0.959	0.801	0.395	0.930	0.327
17	0.953	0.788	0.363	—	—
18	0.939	0.774	0.372	—	0.310
19	1.004	0.815	0.374	—	—
20	0.974	0.780	0.320	—	0.268
21	0.993	0.768	0.316	—	—
22	0.962	0.765	0.321	—	—
23	0.898	0.784	0.279	—	—
24	0.967	0.763	0.297	—	0.218
25	0.972	0.768	0.282	—	—
32	0.939	0.801	0.243	0.830	—
64	0.831	0.688	0.133	0.727	—
128	0.592	0.553	0.075	0.380	—
256	0.304	0.283	0.038	0.217	—
512	0.136	0.129	0.018	0.109	—
1024	0.067	0.051	0.010	0.045	—

Vergleich der Effizienz aller Algorithmen für $N = 1\,843\,200\,000$ Tabelle 28 gibt die Effizienzen $e(p) = S(p)/p$ für alle fünf Verfahren bei der größeren Datenmenge wieder. Auch hier fällt mit steigender Kernzahl die Effizienz aller Algorithmen, jedoch in unterschiedlichem Ausmaß:

- `Arrays.parallelSort()` bleibt bis $p = 8$ sehr hoch (über 0,99), fällt bis $p = 32$ moderat auf etwa 0,94 und sinkt bei $p = 1024$ auf rund 0,07.
- `ParMergeSort(parMerge)` hält bis $p = 4$ eine Effizienz von 0,90–0,95 und liegt bei $p = 16$ noch bei 0,80. Erst ab $p = 64$ fällt sie deutlicher ab, erreicht aber bei $p = 256$ noch knapp 0,28.
- `ParMergeSort(seqMerge)` ist ab $p = 4$ mit $e \approx 0,75$ merklich niedriger und sinkt bis $p = 16$ auf circa 0,40, bei $p = 64$ bis auf 0,13.
- Bitonic Merge (BIM) skaliert nahezu ideal bis $p = 16$ (Effizienz $> 0,93$), fällt dann aber ab und liegt bei $p = 64$ um 0,73 sowie bei $p = 256$ nur noch bei 0,22.
- Odd–Even Merge Splitting Sort (OEMS) erreicht maximal $e \approx 0,95$ bei $p = 2$, bricht aber schon bei mittleren p stark ein und bleibt ab $p = 24$ unter 0,22.

Insgesamt nutzt `ParMergeSort(parMerge)` auch für $N = 1,843,200,000$ die Kerne am gleichmäßigsten, gefolgt von `Arrays.parallelSort()`. Die anderen Verfahren zeigen schon bei mittleren bis hohen Kernzahlen deutlich geringere Auslastung.

Vergleichs-Matrix für Laufzeit für 921 600 000

Tabelle 29: Pairwise Vergleich der Laufzeiten (T_B/T_A in %) bei $p = 16$, $N = 921\,600\,000$

	javaParSort	parMerge	seqMerge	BIM	OEMS
javaParSort	100%	119%	259%	103%	179%
parMerge	84%	100%	216%	86%	150%
seqMerge	39%	46%	100%	40%	69%
BIM	97%	116%	250%	100%	173%
OEMS	56%	67%	145%	58%	100%

Erläuterung der Matrix In Tabelle 29 steht in Zeile A und Spalte B der Wert

$$100 \times \frac{T_B(16)}{T_A(16)}$$

in Prozent. Ein Eintrag von über 100 % bedeutet, dass Algorithmus A bei $p = 16$ schneller ist als B – je höher der Prozentsatz, desto größer der Vorsprung. Ein Wert unter 100 % zeigt dagegen, dass B schneller war.

Wir haben $p = 16$ gewählt, weil dieser Punkt einerseits realistische Mehrkernsysteme widerspiegelt und andererseits noch vor dem starken Einbruch durch Synchronisations- und Merge-Overhead liegt. So erhält man einen aussagekräftigen Mittelwert, der die Kernskalierung ohne Extrem-Effekte zeigt.

Mit dieser Darstellung lässt sich direkt vergleichen, welches Verfahren relativ zur Konkurrenz am wenigsten Zeit benötigt – etwa erkennt man, dass ParMergeSort(parMerge) (84% in Zeile „parMerge“, Spalte „javaParSort“) etwa 16 % schneller arbeitet als Javas `parallelSort()`, während Bitonic Merge mit 97% nur einen leichten Vorteil hat. Gleichzeitig wird sichtbar, wie stark SeqMerge und OEMS abfallen (nur 39% bzw. 56% in der ersten Spalte). Diese komprimierte Matrix fasst die wichtigsten Laufzeitvergleiche verständlich zusammen.

Vergleichs-Matrix für Beschleunigung für 921 600 000

Tabelle 30: Pairwise Vergleich der Speed-ups (S_B/S_A in %) bei $p = 16$, $N = 921\,600\,000$

	javaParSort	parMerge	seqMerge	BIM	OEMS
javaParSort	100%	81%	38%	93%	54%
parMerge	124%	100%	47%	116%	67%
seqMerge	266%	214%	100%	248%	143%
BIM	107%	86%	40%	100%	58%
OEMS	186%	150%	70%	173%	100%

Warum $p = 16$? Wir wählen $p = 16$, weil dieser Wert typisch für den „Sweet-Spot“ moderner Mehrkern-CPU's ist: Hier ist die Parallelisierung stark wirksam, ohne dass der Overhead bereits dominiert. So lässt sich der relative Skalierungsvorsprung der Algorithmen besonders gut ablesen.

Erklärung der Matrix In Zeile A und Spalte B steht

$$100 \times \frac{S_B(16)}{S_A(16)}$$

in Prozent. Ein Eintrag über 100 % zeigt, dass Algorithmus A bei $p = 16$ weniger Speed-up erzielt als B (also langsamer skaliert), während Werte unter 100 % bedeuten, dass A besser skaliert als B . Zum Beispiel steht in Zeile „parMerge“, Spalte „javaParSort“ 124%: ParMergeSort(parMerge) erreicht nur etwa 12,45 Speed-up, während parallelSort 15,47 – d.h. *javaParSort* skaliert um 24% besser als parMerge. Umgekehrt in Zeile „javaParSort“, Spalte „parMerge“ 81% zeigt: javaParSort erreicht nur 0,81-fachen Speed-up verglichen mit parMerge, also parMerge ist hier schneller. Diese komprimierte Form macht direkt sichtbar, wer bei $p=16$ am effizientesten skaliert.

Vergleichs-Matrix für Effizienz für 921 600 000

Tabelle 31: Pairwise Vergleich der Effizienzen (e_B/e_A in %) bei $p = 16$, $N = 921\,600\,000$

	javaParSort	parMerge	seqMerge	BIM	OEMS
javaParSort	100%	80%	38%	93%	54%
parMerge	124%	100%	47%	116%	67%
seqMerge	266%	214%	100%	248%	143%
BIM	107%	86%	40%	100%	58%
OEMS	186%	150%	70%	173%	100%

Erklärung der Matrix In Tabelle 31 steht in Zeile A , Spalte B der Wert

$$100 \times \frac{e_B(16)}{e_A(16)}$$

in Prozent. Er gibt an, wie effizient Algorithmus A im Vergleich zu B bei $p = 16$ ist. Werte über 100 % zeigen, dass A seine Kerne *besser auslastet* als B , Werte unter 100 % deuten darauf hin, dass B hier effizienter arbeitet.

Beispielsweise erreichen javaParSort und Bitonic Merge (BIM) Effizienzen von etwa 0,97 und 0,90, während parMerge bei 0,78 liegt. In der Matrix bedeutet das: In Zeile „BIM“, Spalte „parMerge“ steht ca. 116 % – BIM nutzt seine Kerne bei $p = 16$ um 16 % effizienter als parMergeSort(parMerge). Umgekehrt zeigt der Eintrag 86 % in Zeile „parMerge“, Spalte „BIM“, dass parMerge nur 86 % der Effizienz von BIM erreicht. SeqMerge und OEMS liegen mit 36–52 % deutlich hinter den anderen Verfahren. Diese kompakte Gegenüberstellung macht sofort sichtbar, welche Algorithmen ihre Ressourcen bei moderater Parallelität am effektivsten einsetzen.

Vergleichs-Matrix für Laufzeit für 1843200000

Tabelle 32: Pairwise Vergleich der Laufzeiten (T_B/T_A in %) bei $p = 16$, $N = 1\,843\,200\,000$

	javaParSort	parMerge	seqMerge	BIM	OEMS
javaParSort	100%	113%	234%	97%	255%
parMerge	89%	100%	207%	86%	226%
seqMerge	43%	48%	100%	42%	109%
BIM	103%	116%	241%	100%	262%
OEMS	39%	44%	92%	38%	100%

Erläuterung der Laufzeit-Vergleichsmatrix (Tabelle 32) In dieser Matrix steht in Zeile A , Spalte B jeweils der Wert

$$100 \times \frac{T_B(16)}{T_A(16)}$$

in Prozent – also das Verhältnis der Laufzeit von Algorithmus B zu Algorithmus A bei $p = 16$. Ein Wert über 100 % zeigt, dass A schneller ist als B , Werte unter 100 % deuten darauf hin, dass B die geringere Laufzeit hat. Wir haben uns für $p = 16$ entschieden, weil dies eine typische Prozessorzahl moderner Mehrkernsysteme darstellt und noch vor dem starken Einbruch durch Synchronisations- und Merge-Overhead liegt. So lässt sich der relative Geschwindigkeitsvorsprung in einem realistischen Szenario gut ablesen. Zum Beispiel bedeutet in Zeile „parMerge“, Spalte „javaParSort“ der Eintrag 89 %, dass ParMergeSort(parMerge) etwa 11 % schneller arbeitet als Javas `parallelSort()` bei $p = 16$.

Vergleichs-Matrix für Beschleunigung für 1843200000

Tabelle 33: Pairwise Vergleich der Speed-ups (S_B/S_A in %) bei $p = 16$, $N = 1\,843\,200\,000$

	javaParSort	parMerge	seqMerge	BIM	OEMS
javaParSort	100%	84%	41%	97%	37%
parMerge	120%	100%	49%	116%	44%
seqMerge	243%	203%	100%	236%	90%
BIM	103%	86%	42%	100%	38%
OEMS	270%	226%	111%	262%	100%

Erläuterung der Speed-up-Vergleichsmatrix (Tabelle 33) Hier ist in jeder Zelle das Verhältnis

$$100 \times \frac{S_B(16)}{S_A(16)}$$

in Prozent angegeben, wobei $S(p) = T(1)/T(p)$ der Speed-up ist. Werte über 100 % zeigen, dass A schlechter skaliert als B ; unter 100 % heißt, A skaliert besser. Mit $p = 16$ erhält man so einen klaren Blick auf das Skalierungsverhalten im „Sweet-Spot“ moderner CPUs. Etwa steht in Zeile „seqMerge“, Spalte „parMerge“ 203 %: ParMergeSort(parMerge) erzielt rund doppelt so hohen Speed-up wie seqMerge, also skaliert deutlich besser.

Vergleichs-Matrix für Effizienz für 1843200000

Tabelle 34: Pairwise Vergleich der Effizienzen (e_B/e_A in %) bei $p = 16$, $N = 1\,843\,200\,000$

	javaParSort	parMerge	seqMerge	BIM	OEMS
javaParSort	100%	84%	41%	97%	37%
parMerge	120%	100%	49%	116%	44%
seqMerge	243%	203%	100%	236%	90%
BIM	103%	86%	42%	100%	38%
OEMS	270%	226%	111%	262%	100%

Erläuterung der Effizienz-Vergleichsmatrix (Tabelle 34) In dieser Matrix findet sich in Zeile A , Spalte B der Wert

$$100 \times \frac{e_B(16)}{e_A(16)}$$

in Prozent, wobei $e(p) = S(p)/p$ die Effizienz angibt. Werte über 100 % bedeuten, dass A seine Kerne schlechter auslastet als B ; Werte unter 100 % zeigen, dass A effizienter ist. Auch hier ermöglicht $p = 16$ eine gute Einschätzung, bevor der Overhead dominiert. So signalisiert etwa der Wert 116 % in Zeile „BIM“, Spalte „parMerge“, dass Bitonic Merge seine Kerne bei $p = 16$ um 16 % effizienter nutzt als ParMergeSort(parMerge).

2.4 Amdahls Gesetz und Gustafsons Interpretation

Amdahls Gesetz Amdahls Gesetz beschreibt die theoretische Grenze der Beschleunigung eines Algorithmus durch Parallelisierung. Es geht davon aus, dass ein Teil eines Algorithmus (der sequentielle Anteil) sich nicht parallelisieren lässt. Dadurch ergibt sich, dass selbst bei einer großen Anzahl von Prozessoren die maximal erreichbare Beschleunigung begrenzt ist. Amdahl formuliert die Beschleunigung $b(N)$ bei N Prozessoren als:

$$b(N) = \frac{1}{s + \frac{p}{N}}$$

wobei s der sequentielle und p der parallelisierbare Anteil mit $s + p = 1$ ist. Das Gesetz macht deutlich, dass eine Reduktion des sequentiellen Anteils entscheidend ist, um hohe Skalierbarkeit zu erreichen.

Gustafsons Interpretation Im Gegensatz dazu geht Gustafson davon aus, dass bei steigender Prozessoranzahl auch die Problemgröße wächst. Somit bleibt die Laufzeit des parallelen Algorithmus konstant, aber es kann mehr Arbeit in gleicher Zeit verrichtet werden. Die Beschleunigung steigt daher linear mit der Prozessoranzahl:

$$b(N) = s + p \cdot N$$

Dies ergibt ein positiveres Bild der Parallelisierung im praktischen Einsatz und zeigt, dass mit wachsendem Problemumfang höhere Beschleunigungen erreichbar sind – solange der parallelisierbare Anteil groß genug ist.

3 Schluss

3.1 Reflexion & Fazit

Beim Odd-Even-Merge-Splitting Sort-Algorithmus werden die Daten mehrfach nach einem festen Muster aufgeteilt und wieder zusammengefügt. Solange die Anzahl der Kerne p bis etwa $\log n$ reicht, sinkt die Laufzeit fast um den Faktor p (Speed-up ca. gleich p), und die Effizienz liegt meist zwischen 0,8 und 1,0. Ab $p > \log n$ bringt jeder zusätzliche Kern kaum noch einen Gewinn, weil der sequentielle Merge-Teil $O(n)$ dominiert und nicht weiter parallelisiert werden kann.

Beim Bitonic Merge werden die Daten in bitonische Sequenzen sortiert und dann in einem parallelen Merge-Schritt zusammengeführt. Die Laufzeit ist insgesamt $O(n/p \log n)$. Bis etwa 32 Kerne erreicht der Algorithmus fast ideale Speed-ups (Effizienz ca. gleich 1), weil Kommunikation und Synchronisation noch gering bleiben. Erst oberhalb von rund 32 Kernen schlägt der Overhead stärker zu Buche und die Effizienz nimmt allmählich ab.

Der Parallele Merge-Sort (parMerge) nutzt auch im Merge-Schritt mehrere Kerne. Bis $p \leq 16$ fast lineare Speed-ups von 10x - 13x (Effizienz 0,8-1,0). Ab etwa 32 Kernen spürt man dann Merge- und Synchronisationskosten. Der Parallele Merge-Sort (seqMerge) führt den Merge-Schritt auf einen Kern durch. Bereits ab

ca. $p = 4$ bricht die Effizienz deutlich unter 0,8 ein und verschlechtert sich mit mehr Kernen rasant.

Java `Arrays.parallelSort()` arbeitet mit Kombination aus Fork/Join-Framework und teils sequentiell Merge. Dieser Sortier-Algorithmus bringt sehr guten Speed-up bis ca. 16 Kernen (Speed-up ca. gleich 15, Effizienz $> 0,95$). Ab 32 Kernen fällt die Effizienz schneller als beim BIM oder `parMerge`, weil der Mix aus parallelen und seriellen Schritten mehr Overhead erzeugt.

Die beste Laufzeit liefert `ParMergeSort(parMerge)`, da dies in den meisten Fällen am schnellsten ist. `BitonicMerge` liefert bis etwa 32 Kernen konstant sehr hohe Effizienz. Java-`ParallelSort` eignet sich für kleine bis mittlere Kernzahlen durch seine einfache Nutzung und sehr gute Performance dort. Jeder Algorithmus wird ab einigen Dutzend Kernen durch Merge-Overhead und Synchronisation stark eingeschränkt. Als Empfehlung gilt, für moderate bis viele Kerne (bis ca. 32) den Bitonic Merge für maximale Effizienz zu verwenden. Für maximale Geschwindigkeit empfiehlt es sich, `ParMergeSort(parMerge)`. Für einen einfachen Einsatz empfiehlt es sich Java's `parallelSort()` bei kleinen bis mittleren Kernzahlen.